

نظام ادارة متجر مجوهرات

مشروع تخرج 2

(قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس)

في هندسة المعلوماتية – هندسة البرمجيات ونظم المعلومات

إعداد الطالب

محمد ابراهيم محمد البني

بسام محمد نزار حرح

اشراف

المهندس شادي بليدي

نيسان – 2026

Jewelry Store Management System

Senior Project 2

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Informatics
Engineering — Software Engineering and Information Systems

Prepared by

Mohamad Ibrahim Mohamad Al-Bonni
Bassam Mohamad Nizar Harh

Supervised by

Eng. Shadi Blidi

April 2026

الحمد لله الذي بفضلله تتم الصالحات، وبنعمته تنجلي الصعاب
أرفع أسمى آيات الشكر والثناء لرب العزة والجلال، الذي أثار بصري وبصيرتي، ومنحني القوة والعزيمة لأتم هذا العمل.
وصلاةً وسلاماً دائماً على المعلم الأول، وسيد البشر، ونبراس الهدى محمد بن عبد الله ﷺ، الذي علمنا أن طلب العلم
عبادة، وأن العمل أمانة.

إلى من سكنوا الوجدان فكانوا سرّ الوجود والنجاح..

إلى والدي العزيز، صاحب الأثر العظيم والقلب الجسور، من علمني أن الصعاب تُدَلُّ بالإرادة. وإلى والدتي الغالية، نبع
الحنان ودعائي الدائم في السجود؛ إليكما أهدي ثمرة هذا الجهد، فما هذا النجاح إلا غرس أيديكما، وصبركما الذي فاق
كل وصف.

إلى رفيقة الدرب ونور الأيام..

إلى حبيبتي الغالية، التي كانت لي السكينة في أوقات القلق، واحفز الأول في لحظات الفتور. شكراً لأنك كنت وما زلتِ
السند الذي لا يميل، والقلب الذي يؤمن بي دائماً؛ هذا النجاح هو خطوة في طريقنا المشترك الذي نبنيه معاً.

إلى سندي وعزوتي..

إلى إخوتي، الذين شاركوني كل تفاصيل هذه الرحلة، وكانوا خير عون لي في الشدائد، لكم في قلبي مكانة لا تفيتها الكلمات.

إلى رفيق السهر والكفاح..

إلى شريكي في هذا المشروع، شكراً على كل لحظة تفكير، وعلى روح الفريق التي جعلت من المستحيل ممكناً، فكنتَ نعم
الشريك ورفيق النجاح.

إلى من بهم تزدان الأيام وتصفو الحياة..

إلى أصدقائي الأوفياء، الذين كانوا نبضاً لهذا الإنجاز، ودافعاً للاستمرار بصدق مودتهم:

كريم عبد العظيم، كريم الدقة، يمان قزاز، مصطفى الشليبي، عبادة نسب، ضياء قزاز، طارق هيلم، عبد الرحمن فلاح، زياد
كنيفد.

إليكم جميعاً، وإلى كل الأصدقاء والإخوة الذين لم تذكرهم السطور وبقيت أسمائهم محفورة في الذاكرة، أهدي هذا العمل تقديراً
وامتناناً لوجودكم المؤثر في مسيرتي.

ملخص المشروع

يهدف هذا المشروع إلى تطوير نظام إلكتروني متكامل يساهم في تحسين جودة الخدمات المقدمة لمُتاجر المَجوهرات، من خلال توفير بيئة رقمية حديثة تتيح إدارة المنتجات والحجوزات والشهادات وقوائم الرغبات وأسعار المواد ومتابعتها بطريقة منظمة وفعالة. يأتي هذا النظام استجابة للحاجة المتزايدة إلى أتمتة العمليات التقليدية وتقليل الاعتماد على الإجراءات اليدوية التي غالباً ما تتسبب في هدر الوقت والجهد وانخفاض كفاءة الأداء.

يرتكز النظام المقترح على تمكين العملاء من الوصول إلى الخدمات المتاحة بسهولة، وتقديم طلباتهم ومتابعتها بشكل مستمر، مما يعزز من مستوى التفاعل بينهم وبين المتجر. كما يسعى النظام إلى تحسين تجربة المستخدم من خلال توفير واجهات استخدام واضحة وسهلة، تتيح التعامل مع مختلف الوظائف دون الحاجة إلى خبرة تقنية متقدمة.

ومن جهة أخرى، يوفر النظام أدوات إدارية متكاملة تُمكن صاحب المتجر من الإشراف على جميع العمليات والطلبات، وإدارتها بكفاءة عالية، مع إمكانية الوصول إلى البيانات المخزنة وتحليلها لدعم اتخاذ القرار. ويتم تخزين كافة المعلومات ضمن قاعدة بيانات مركزية MongoDB، مما يضمن تنظيم البيانات وسهولة استرجاعها، إضافة إلى تعزيز موثوقية النظام واستقراره.

وعلى الرغم من الفوائد الكبيرة التي تقدمها الأنظمة الإلكترونية، إلا أن بعض التحديات قد تظهر، مثل ضعف خبرة بعض المستخدمين في التعامل مع التقنيات الحديثة، مما قد يؤثر على كفاءة الاستخدام. لذلك، يولي هذا المشروع اهتماماً خاصاً بتصميم واجهات استخدام مبسطة، ودعم تجربة المستخدم، بما يضمن سهولة التفاعل مع النظام وتحقيق أقصى استفادة ممكنة منه.

وبشكل عام، يمثل هذا المشروع خطوة نحو التحول الرقمي الفعال، حيث يساهم في تحسين جودة الخدمات، وتسريع إنجاز العمليات، وتوسيع نطاق الوصول إليها، مما يدعم تحقيق كفاءة أعلى في الأداء ويواكب التطورات الحديثة في مجال نظم المعلومات.

Abstract

This project presents the design and development of an integrated electronic system for managing a jewelry store, aimed at enhancing the efficiency and quality of service delivery through digital transformation. The proposed system, hereafter referred to as Nizar Jewellery, addresses the limitations of traditional manual processes which are often associated with inefficiency, time consumption, and operational complexity. By automating product management, reservation handling, certificate issuance, wishlist requests, and material-price updates, the system provides a structured and streamlined environment that improves overall performance.

The platform consists of a customer-facing React 19 single-page application and an administrative dashboard, both consuming a Node.js / Express REST API backed by a MongoDB Atlas document database accessed through the Mongoose ODM. Centralized data storage ensures integrity, consistency, and ease of retrieval, while supporting analytical capabilities for informed decision-making. Customer interactions — browsing products, adding favorites, creating reservations, submitting wishlist requests, and verifying authenticity certificates — are unified with administrative operations such as product CRUD, bulk price recalculation, certificate issuance with QR-based verification, and account-lockout management.

From a security perspective, the system follows a defense-in-depth model combining JWT-based stateless authentication with HttpOnly refresh-token cookies, bcrypt password hashing, Helmet HTTP-security headers, a strict Content Security Policy, route-specific rate limiting, custom input sanitization, role-based access control (RBAC) with two roles (customer and owner), and a privacy-preserving visit-tracking subsystem that stores SHA-256 hashed IP addresses.

The system is deployable through a fully automated DevOps pipeline based on Docker, GitHub Actions (continuous integration, continuous deployment, and Terraform application), and a Prometheus / Grafana monitoring stack. Infrastructure resources, including Cloudflare DNS and MongoDB Atlas users, are provisioned as code using Terraform.

In conclusion, the project contributes to advancing digital transformation efforts by improving service quality, accelerating operational processes, and expanding accessibility, ultimately leading to enhanced organizational efficiency and alignment with contemporary information-systems practices in the specialized jewelry domain.

Table of Contents

ملخص المشروع.....	IV
Abstract	V
Table of Contents.....	VI
List of Figures.....	XII
List of Tables	XII
List of Algorithms.....	XIV
Section One — Introduction.....	1
Chapter One: General Introduction	2
1.1 Project Overview and Background	2
1.2 Problem Statement and Its Practical and Academic Importance	2
1.3 Project Objectives	3
Main Objective.....	3
Sub-Objectives	3
1.4 Project Scope, Constraints, and Assumptions	4
First: Project Scope.....	4
Second: Project Constraints	4
Third: Project Assumptions	4
1.5 Project Contribution to Knowledge and the Field	5
1.6 Research Methodology and Scientific Methods Used	5
1.7 Document Structure.....	6
1.8 Definitions and Fundamental Concepts	7
Chapter Two: Project Development and Management Methodology.....	9
2.1 Selected Development Methodology and Justifications	10
Justifications for Choosing the Methodology	10
2.2 Project Timeline.....	11
2.3 Risk Management.....	13
2.4 Project Quality Management	14
Code Review	14
Software Testing	14
Validation and Error Handling.....	14
Security Measures	14
Acceptance Criteria	14
Quality Support Tools	14

Continuous Improvement	15
2.5 Project Management Tools Used	15
Git and GitHub	15
Trello	15
Supporting Development Tools	15
2.6 Software Configuration Management (SCM).....	16
Version Tracking.....	16
Development Workflow	16
Code Organization.....	16
Change and Bug Handling.....	16
2.7 Continuous Monitoring and Evaluation Mechanisms.....	16
2.7.1 Managerial Progress Monitoring	16
2.7.2 Technical Code-Quality Monitoring.....	16
2.7.3 Runtime Operational Monitoring	17
2.7.4 Periodic Requirements and Quality Review	17
Section Two — Theoretical Study and Analysis.....	18
Chapter Three: Literature Review and Previous Studies.....	19
3.1 Previous Studies in E-commerce Systems.....	19
Findings from the Literature	19
3.2 Modern Technologies and Frameworks in the Field.....	20
Front-End Technologies	20
Back-End Technologies.....	20
Database Systems.....	20
Development and DevOps Tools	20
3.3 Comparative Analysis of Current Systems and Products.....	21
3.4 Research Gap and Proposed Innovation.....	22
Proposed Innovation.....	22
3.5 Theoretical Framework and Reference Studies	23
3.6 Recent and Future Trends in the Field	23
Chapter Four: Requirements Analysis Model.....	24
4.1 Feasibility Study	25
Technical Feasibility.....	25
Economic Feasibility	25
Conclusion	25
4.2 Requirements Elicitation	26
Interviews.....	26
Observation	26

Existing-Systems Analysis.....	26
Questionnaires.....	26
Summary of Findings	26
4.3 Stakeholder Identification.....	27
Customer / End User	27
Admin / Store Owner.....	27
Future Developer.....	27
System Perspective.....	27
4.4 Functional Requirements	28
1. Account and User Management Requirements.....	28
2. Product Management Requirements.....	28
3. Comment Requirements.....	29
4. Reservation Requirements	29
5. Certificate Requirements	30
6. Wishlist Request Requirements.....	30
7. Favorite-Product Alert Requirements.....	30
8. Material-Price Requirements	31
9. Statistics and Monitoring Requirements.....	31
10. General System Requirements	31
4.5 Non-Functional Requirements Classification.....	32
4.6 MoSCoW Requirement Classification	33
Requirements Management Approach	33
4.7 UML Diagrams for Requirements	34
4.7.1 Main Use Case.....	34
4.7.2 Use Case Descriptions	34
4.7.3 Activity Diagrams.....	38
4.7.4 Sequence Diagrams.....	43
4.7.5 Test Case Descriptions.....	48
4.8 Modeling Based on the Methodology Used	52
4.8.1 Agile Artifacts (User Stories, Story Mapping, Product Backlog).....	52
4.8.2 Traditional Methodology: Software Requirements Specification (SRS).....	55
4.8.3 Iterative Methodology: Features Lists	55
4.9 Requirements Analysis and Documentation.....	57
4.9.1 Requirements Traceability Matrix.....	57
4.9.2 Initial Test Cases.....	58
Section Three — Design and Architecture.....	61
Chapter Five: System Architectural Design	62

5.1 High-Level Design	62
5.1.1 System Block Diagram	62
5.1.2 Architectural Pattern Used	64
5.1.3 System Architecture Design	65
5.2 Database Design.....	67
5.2.1 Entity–Relationship Model	67
5.2.2 Relational Schema	69
5.2.3 Constraints and Keys	69
5.3 External API Design.....	71
API Design Principles	72
5.4 Security and Authorization Design	74
5.4.1 Overview	74
5.4.2 Authentication	74
5.4.3 Authorization.....	74
5.4.4 Protection Against Common Attacks	75
5.4.5 Sensitive Data Protection	75
Section Four — Implementation and Testing.....	76
Chapter Six: Implementation Environment and Tools	77
6.1 Programming Tools and Libraries Used.....	77
6.2 Development Environment and Project Structure	78
6.3 Programming Languages and Frameworks	79
6.4 Database Server and Operating System	80
6.5 Libraries and Dependencies.....	80
6.6 Version Control and Source Management Tools.....	80
6.7 Continuous Integration and Deployment Tools (CI/CD)	81
6.8 Testing Environment	81
Chapter Seven: Software Implementation	82
7.1 File and Project Structure	82
7.2 Modules Implementation.....	83
7.3 APIs Implementation.....	84
7.4 Data Access Layer Implementation	85
7.5 Business Logic Layer Implementation.....	86
7.6 UI Implementation	87
7.7 Intelligent-Algorithm Implementation	88
Chapter Eight: Testing and Evaluation.....	89
8.1 Test Plan	89
8.2 Types of Tests	90

8.3 Test Diagrams	91
8.3.1 Test Case Diagrams	91
8.3.2 Test Result Charts	91
8.4 Test Results and Analysis	92
8.5 Quality Metrics	93
8.6 Usability Testing	94
Section Five — Results and Recommendations	95
Chapter Nine: Results and Analysis	96
9.1 System Application Results	96
9.2 Comparative Analysis with Existing Systems	97
9.3 Objectives Fulfillment	98
9.4 Strengths and Weaknesses Analysis	99
9.4.1 Strengths	99
9.4.2 Weaknesses	99
9.5 Key Performance Indicators (KPIs)	100
Chapter Ten: Conclusion and Recommendations	101
10.1 Project Summary and Main Achievements	101
10.2 Challenges Encountered	102
10.2.1 Technical Challenges	102
10.2.2 Managerial Challenges	102
10.2.3 Academic Challenges	102
10.3 Suggestions for Future Development	103
10.4 Recommendations	104
10.4.1 Technical Recommendations	104
10.4.2 Academic Recommendations	104
10.4.3 Business Recommendations for Jewelry Store Owners	104
Section Six — Appendices and References	105
Appendix A: Requirements Gathering Questionnaires	105
A.1 Customer Interview Form	105
A.2 Store Owner Interview Form	105
A.3 Usability Questionnaire	106
Appendix B: Detailed Design	107
B.1 Class and Component Diagrams	107
B.1.1 Detailed Class Diagrams	107
B.1.2 Component Diagrams	108
B.1.3 Modules Design	108
B.1.4 APIs Design	110

B.2 Behavioural and UX Design	111
B.2.1 Data-Flow Diagrams	111
B.2.2 State Machine Diagrams.....	111
B.2.3 Algorithm Pseudocode	114
B.2.4 UX Design (Mockups and Wireframes).....	114
B.2.5 AI/ML Model Design (Future Work).....	114
Appendix C: Detailed Test Results	114
C.1 Jest Test Summary (Server)	115
C.2 Playwright Test Summary (E2E)	115
C.3 k6 Performance Test Summary	116
Appendix D: User Manual	117
D.1 Customer Guide	117
D.1.1 Creating a New Account	117
D.1.2 Browsing Products and Adding to Favorites.....	117
D.1.3 Creating a Reservation	117
D.1.4 Verifying a Jewelry Authenticity Certificate	117
D.1.5 Submitting a Wishlist (Custom Design Request)	117
D.2 Store Owner Guide	118
D.2.1 Logging into the Dashboard	118
D.2.2 Managing Products	118
D.2.3 Managing Material Prices	118
D.2.4 Managing Reservations	118
D.2.5 Issuing an Authenticity Certificate	118
References	119
Foreign References (Books)	119
Web Sites and Technical Documentation	120
Scholarly Articles and Research.....	120
Arabic References	120

List of Figures

Figure 1: Gantt Chart	12
Figure 2: Main Use Case Diagram.....	35
Figure 3: Activity Diagram — Browse Products.....	39
Figure 4: Activity Diagram — Reserve Product	40
Figure 5: Activity Diagram — Submit Wishlist Request	41
Figure 6: Activity Diagram — Manage Product Catalog	42
Figure 7: Activity Diagram — Create Certificate	43
Figure 8: Sequence Diagram — Browse Products	44
Figure 9: Sequence Diagram — Reserve Product	45
Figure 10: Sequence Diagram — Submit Wishlist Request	46
Figure 11: Sequence Diagram — Manage Product Catalog	47
Figure 12: Sequence Diagram — Create Certificate	48
Figure 13: System Block Diagram.....	64
Figure 14: Production Deployment Topology.....	67
Figure 15: Entity–Relationship Diagram	70
Figure 16: Sequence Diagram — Create Reservation	75
Figure 17: Test-Case Diagram — UC-03 (Reserve Product).....	93
Figure 18: Class Diagram of Main System Entities.....	110
Figure 19: Level-1 Data-Flow Diagram.....	113
Figure 20: State Machine — Reservation	114
Figure 21: State Machine — Certificate	115

List of Tables

Table 1: Definitions and Fundamental Concepts.....	7
Table 2: Project Gantt Schedule	11
Table 3: Risk Register and Mitigation Strategies	13
Table 4: Estimated Cost Summary	26
Table 5: Non-Functional Requirements Classification	33
Table 6: MoSCoW Requirement Classification	34
Table 7: Comparative Analysis of Current Systems and Products	22
Tables 8–14: Use Case Descriptions UC-01 to UC-07	35–38
Tables 15–22: Test Case Descriptions TC-01 to TC-11	49–52
Tables 23–27: Agile Artifacts (User Stories, Journeys, Backlog)	53–55
Tables 28–29: SRS Functional and Non-Functional Requirements	56
Tables 30–33: Iterative Increment Feature Lists	56–57
Table 34: Requirements Traceability Matrix.....	58
Table 35: Extended Initial Test Cases (TC-12 to TC-29)	60
Table 36: Layered Architecture Mapping to Project Artifacts.....	65
Table 37: System Entities and Principal Relationships.....	68
Table 38: Relational Equivalent of the MongoDB Collections.....	71
Table 39: Classification of REST API Endpoints by Domain	73
Table 40: Representative API Endpoints	74
Table 41: Principal Libraries and Tools Used in the Project.....	79
Table 42: High-Level Project Structure	80
Table 43: Top-Level Layout of the Project Repository	84
Table 44: System Modules Across UI, Controller and Data Layers.....	85
Table 45: Test Categories Executed in the Project	92
Table 46: Aggregated Test Results Indicators.....	94
Table 47: Principal Quality Metrics.....	95
Table 48: Usability Testing Results.....	96
Table 49: Comparison of Nizar Jewellery with Comparable Systems	99
Table 50: Achievement of Project Objectives.....	100
Table 51: Achieved Key Performance Indicators.....	102
Table 52: Summary of the System's Logical Classes	109
Table 53: Detailed Trace of a Representative REST Endpoint	112
Table 54: Principal State Machines in the System.....	113
Table 55: Summary of Jest Test Results	117
Table 56: End-to-End Test Results	117
Table 57: k6 Load Test Results	118

List of Algorithms

Algorithm 1: Composition of the Reservation Creation Endpoint

Algorithm 2: Create Reservation

Algorithm 3: Bulk Price Update After Material Change

Section One — Introduction

This section establishes the context of the project and the methodology governing its development. Chapter One presents the project background, problem statement, objectives, scope, contribution, methodology, document structure, and definitions. Chapter Two presents the development and project-management methodology, including the chosen lifecycle model, schedule, risk register, quality plan, project-management tools, software configuration management, and continuous monitoring mechanisms.

Chapter One: General Introduction

1.1 Project Overview and Background

This project lies within the field of e-commerce systems, which constitute one of the most prominent modern applications of information technology. Such systems enable users to browse products and submit online reservations easily and rapidly, and the field has seen significant growth in recent years due to the wide expansion of internet usage and the reliance of individuals and organizations on digital solutions for managing their businesses and providing services.

Within this context, the project develops an integrated electronic platform for managing a jewelry store (Nizar Jewellery Management System). The platform combines a customer-facing interface that allows customers to browse products, view their details, add products to favorites, submit wishlist requests, and create reservations, with an administrative interface dedicated to the store owner that allows managing products, prices, reservations, certificates, comments, favorite-product alerts, users, and overall system monitoring. The system seeks to provide a modern digital environment that facilitates interaction between customers and the store and helps improve the quality of the services provided.

1.2 Problem Statement and Its Practical and Academic Importance

Many traditional jewelry stores rely on manual or semi-digital methods for managing products and customer requests, such as recording reservations in an unstructured manner or updating prices manually. This leads to slow access to information, increased administrative effort, and a higher probability of human error. The absence of an integrated electronic platform also limits the store's ability to reach a wider segment of customers, especially with the increasing reliance on the internet for browsing products and making purchasing decisions.

The importance of the project is clear from both a practical and an academic standpoint. Practically, it provides an integrated digital solution that improves store management, increases work efficiency, reduces human errors, and enhances the user experience through a simple, secure interface. Academically, the project offers a practical applied model for designing and developing modern web systems and applies software-engineering concepts such as requirements analysis, architectural design, data management, layered architecture, role-based access control, and continuous integration within a realistic application that serves an important sector of e-commerce.

1.3 Project Objectives

The project objectives were defined according to the SMART methodology so that they are specific, measurable, achievable, realistic, and time-bound.

Main Objective

To deliver an integrated web-based platform for managing a jewelry store that supports product display, reservations, certificate issuance and verification, wishlist requests, favorite-product alerts, material-price management, and user administration, providing interactive interfaces for both customers and administrators, with a level of performance and stability suitable for real-world use within the project implementation period.

Sub-Objectives

- Develop an interactive and easy-to-use user interface that enables customers to browse products and clearly access their details with a response time not exceeding 3 seconds for typical operations.
- Design and implement an authentication system that includes registration, login, password reset, email verification, and Google OAuth sign-in.
- Create a product management system that allows the store owner to add, edit, delete, pin, and mark products as “special,” including image upload to a cloud storage service.
- Develop a reservation system that allows customers to create and track reservations in an organized manner, with state transitions enforcing business rules (pending, approved, rejected, expired, completed, cancelled).
- Build a certificate management system that issues authenticity certificates for products, with QR-code-based public verification and ownership transfer history.
- Provide a wishlist feature that enables customers to submit custom-design requests, and a favorite-alert subsystem that notifies users about price drops, out-of-stock, and back-in-stock events.
- Manage the prices of raw materials (gold, silver, diamond) — including per-karat gold pricing — with a bulk-update operation that recomputes the price of all dependent products.
- Achieve a defense-in-depth security level by protecting accounts (password hashing, account lockout), routes (JWT + RBAC), and inputs (sanitization, validation, rate limiting).
- Significantly reduce reliance on manual operations in store management by automating core processes.
- Improve the user experience through clear interfaces and logical organization of functions, and enable the system to handle a concurrent number of users without a noticeable impact on performance.

1.4 Project Scope, Constraints, and Assumptions

First: Project Scope

This project includes the development of an integrated web application for managing a jewelry store, covering the two main sides of the system.

On the customer side, the system enables: browsing jewelry products and viewing their details; adding products to favorites; creating and tracking reservations; viewing and verifying authenticity certificates; submitting wishlist (custom design) requests; subscribing to favorite-product alerts; commenting on products; and managing the personal profile.

On the administrative side (Admin Dashboard), the system enables: managing products (add, edit, delete, pin, mark as special); managing reservations and tracking their statuses; managing certificates (issue, revoke, transfer, delete); managing wishlist requests and responding to customers; managing users and account-lockout status; controlling prices and materials with bulk product update; monitoring system activity and performance.

The project does not include the development of native mobile applications; it is limited to a responsive browser-based system, with the possibility of future expansion.

Second: Project Constraints

- Time constraints: The project was implemented within the timeframe allocated for the graduation project.
- Human-resource constraints: The team consists of a limited number of developers, which limits the scope of additional features.
- Data constraints: The lack of real-world operational data required the use of generated dummy data during development and testing.
- Infrastructure constraints: Reliance on free hosting tiers (Railway, MongoDB Atlas, Cloudflare) may affect performance in case of heavy use.
- Advanced security constraints: A multi-layer security baseline was applied, without expanding into commercial-grade systems such as actual online payment gateways.

Third: Project Assumptions

- Users are assumed to have basic knowledge of using computers and the internet.
- A stable internet connection is assumed while using the system.
- Customers and administrators are assumed to enter correct data.
- The system is assumed to be used within the scope of a small-to-medium-sized jewelry store.
- The number of concurrent users remains within reasonable limits that do not affect system performance.
- Payment operations, if any, take place outside the system or manually in the current version.

1.5 Project Contribution to Knowledge and the Field

The project contributes to the field of e-commerce systems by presenting an integrated applied model for managing a jewelry store that combines a set of service and administrative functions within a single interconnected platform. Although many general e-commerce systems exist, this project focuses on customizing the system for the jewelry domain, which has specific requirements in product presentation and the management of authenticity certificates, karat-aware pricing, and reservations on one-of-a-kind items.

The main contribution of the project is the integration of several functional components within a unified system: product management, reservation system, certificate management with QR-code verification, wishlist requests, favorite-product alerts, comment management, material-price management, and user administration — within one integrated environment, instead of relying on separate systems or partial solutions.

The project also presents a practical design based on a clear separation between the front-end (React 19 SPA) and the back-end (Node.js / Express REST API), which contributes to improving scalability and maintainability, in line with modern web-application development practices. From an applied perspective, the project provides a case study of transforming a traditional jewelry store into an integrated digital system, while considering aspects related to user experience, data management, basic security, and a complete DevOps pipeline. It therefore serves as a scalable, generalizable model that can be reused to develop similar systems for other specialized stores, with the possibility of future expansion to include online payments and intelligent analytics.

1.6 Research Methodology and Scientific Methods Used

This project relies on the applied engineering approach as the main methodology, as it focuses on designing and developing an integrated software system that meets real needs in the field of e-commerce, specifically in managing jewelry stores. The approach combines theoretical and practical aspects through problem analysis, proposing a technical solution, implementing it, and then evaluating it.

The descriptive method was also used to study the current reality of traditional and electronic store-management systems and to analyze their weaknesses, such as the difficulty of manually managing products and reservations or the absence of an integrated platform that connects the various operations within the store. This analysis helped define the requirements of the proposed system with greater accuracy.

The analytical method was adopted during the requirements analysis stage, where the system components and various functions (user management, products, reservations, certificates, favorites, wishlist requests, comments, material prices, statistics, and monitoring) were studied in order to determine the relationships between them and build a logical concept of the system before implementation.

During the evaluation phase, the practical validation method was followed by testing the system using realistic use scenarios — registration, login, browsing, reservation creation, certificate issuance, certificate verification via QR code, bulk price update, and administrative operations — to ensure that the system works as required and meets the predefined objectives.

1.7 Document Structure

The report is organized hierarchically, progressing from general context to detailed technical execution, in line with academic conventions for software engineering theses. The document is divided into six sections that comprise ten chapters, supported by four appendices and a bibliography.

Section One — Project Introduction. This section comprises Chapter One (General Introduction), which establishes the project context, problem statement, objectives, scope, and methodology, and Chapter Two (Project Development and Management Methodology), which discusses the chosen development methodology, schedule, risk register, quality management, and project tools.

Section Two — Theoretical Study and Analysis. This section covers Chapter Three (Literature Review and Related Work) and Chapter Four (Requirements Modeling and Analysis), which together justify the project and document the functional and non-functional requirements together with UML models, MoSCoW prioritization, and the Requirements Traceability Matrix.

Section Three — Architectural Design. Presented in Chapter Five, this section provides the high-level system architecture, database design (ER diagram, relational schema, constraints), external API design, and the security and authorization model.

Section Four — Implementation and Testing. This section includes Chapter Six (Implementation Environment and Tools), Chapter Seven (Software Implementation), and Chapter Eight (Testing and Evaluation). Together they describe the technology stack, the implementation of each architectural layer, the test plan, the executed test types, the obtained results, and the quality metrics.

Section Five — Results and Recommendations. Comprising Chapter Nine (Results and Analysis) and Chapter Ten (Conclusion and Recommendations), this section evaluates the achievement of project objectives, performs a comparative analysis with existing systems, examines strengths and weaknesses, measures key performance indicators, and outlines future development paths.

Section Six — Appendices and References. This final section contains four appendices (Requirements Gathering Questionnaires, Detailed Design, Detailed Test Results, and User Manual) followed by a comprehensive bibliography of English references, Arabic references, online documentation, and scholarly articles.

The structure supports an explicit traceability path: requirements collected in Chapter Four are systematically mapped through design (Chapter Five) and implementation (Chapter Seven) until verified by the test cases in Chapter Eight, ensuring complete coverage and verifiability.

1.8 Definitions and Fundamental Concepts

To unify the terminology used throughout the report and ensure a shared understanding between writer and reader, Table 1 defines the principal technical concepts referenced in subsequent chapters.

Table 1: Definitions and Fundamental Concepts

Term	Definition
E-commerce System	A software platform enabling product display, management, and online ordering or reservation. It typically combines a customer interface, an administrative interface, and a centralized database.
Single Page Application (SPA)	A web-application paradigm in which navigation and content updates occur inside the same loaded page, relying on client-side JavaScript and routing. Implemented in this project using React 19.
REST (Representational State Transfer)	An architectural style for API design relying on standard HTTP methods (GET, POST, PUT, PATCH, DELETE) with no server-side session state. Used to structure communication between the React client and the Express server.
MVC (Model–View–Controller)	A design pattern that separates data and business logic (Model), user interface (View), and request coordination (Controller), supporting cleaner code organization and easier maintenance.
CRUD	The four fundamental data operations: Create, Read, Update, and Delete. They form the backbone of system operations on entities such as users, products, reservations, and certificates.
JSON Web Token (JWT)	An open standard (RFC 7519) for transmitting digitally signed JSON information between parties. Used to implement stateless authentication via short-lived access tokens and long-lived refresh tokens.
OAuth 2.0 / Google OAuth	A standard authorization framework allowing users to authenticate through an external identity provider (Google) without sharing their password with the application.
Password Hashing (bcrypt)	A one-way transformation of passwords combined with a per-user salt, ensuring that stored credentials cannot be reversed even if the database is leaked.
NoSQL / MongoDB	A family of non-relational, document-oriented databases storing records as BSON documents. MongoDB Atlas serves as the primary persistent store for the project.
Mongoose ODM	An Object Data Modeling library for Node.js that defines schemas, validators, and indexes for MongoDB collections. It represents the Data Access Layer in this project.
In-Memory Caching	A technique that stores frequent query results in process memory to reduce response time and database load. Implemented via the node-cache library and a custom cache middleware.
Rate Limiting	A mechanism that constrains the number of requests permitted from a given IP within a time window, mitigating brute-force attacks and abuse. Implemented via express-rate-limit.

Term	Definition
CORS (Cross-Origin Resource Sharing)	A browser security policy that controls which origins may access the API. Configured on the server to whitelist trusted front-end domains.
DevOps and CI/CD	A set of practices uniting development and operations through Continuous Integration and Continuous Deployment. Implemented in this project through GitHub Actions workflows.
Docker / Containerization	OS-level virtualization that packages an application and its dependencies into portable images. Used to build reproducible client, server, MongoDB, Prometheus, and Grafana containers.
Infrastructure as Code (IaC)	A practice of provisioning and managing infrastructure through declarative source files. Realized via Terraform modules to manage Cloudflare DNS and MongoDB Atlas resources.
Authenticity Certificate	A digital document issued by the store to a customer to certify the authenticity of a jewelry piece. Contains a unique identifier, verification code, digital signature, transfer history, and machine-readable QR code.

Chapter Two: Project Development and Management Methodology

2.1 Selected Development Methodology and Justifications

This project adopted the Iterative and Incremental Development Model because it provides high flexibility in developing modern software systems, especially web applications consisting of several interconnected modules such as the authentication system, product management, reservation system, certificate management, and the administrative dashboard.

The methodology divides the system into a set of relatively independent modules, each developed gradually (incrementally) and improved repeatedly through several iterative cycles. The system is not built all at once; rather, it is constructed through successive stages, each stage producing a partial version that can be tested and improved.

In the context of this project, development began with priority core functions — user management (registration and login), product display, and the admin dashboard — and was then gradually expanded to add more advanced functions such as the reservation system, certificate management with QR verification, wishlist requests, favorite-product alerts, material-price bulk updates, user-experience refinements, monitoring dashboards, and DevOps automation.

Justifications for Choosing the Methodology

- Flexibility: allows requirements to be modified or the system to be improved during development without rebuilding it.
- Risk reduction: detects errors and technical problems in early stages through continuous testing of each part.
- Quality improvement: allows continuous review and improvement of the system after each development cycle.
- Modular development: makes it easier to divide the system into clear modules, helping organize work among team members.
- Web suitability: very suitable for developing systems with multiple interfaces and different roles, as in this project.
- Scalability: allows new features to be added in the future without affecting the basic system architecture.

2.2 Project Timeline

The schedule is organized as a Gantt chart covering the four iterations of the project, with explicit milestones at the end of each iteration. Table 2 summarizes the schedule, and Figure 1 presents the visual chart.

Table 2: Project Gantt Schedule

TY	Start	End	Duration (days)
Iteration 1 — Requirements	15/11/2025	22/11/2025	8
Iteration 1 — Basic Design	23/11/2025	30/11/2025	8
Milestone: Iteration 1 Complete	30/11/2025	30/11/2025	1
Iteration 2 — Core Development	01/12/2025	10/01/2026	41
Iteration 2 — Booking & Wishlist	11/01/2026	31/01/2026	21
Milestone: Iteration 2 Complete	31/01/2026	31/01/2026	1
Iteration 3 — Admin & Certificates	01/02/2026	20/02/2026	20
Integration	21/02/2026	28/02/2026	8
Milestone: System Integrated	28/02/2026	28/02/2026	1
Testing	01/03/2026	15/03/2026	15
Optimization	16/03/2026	30/03/2026	15
Milestone: Testing Complete	30/03/2026	30/03/2026	1
Deployment & Documentation	31/03/2026	07/04/2026	8
Milestone: Project Delivered	07/04/2026	07/04/2026	1

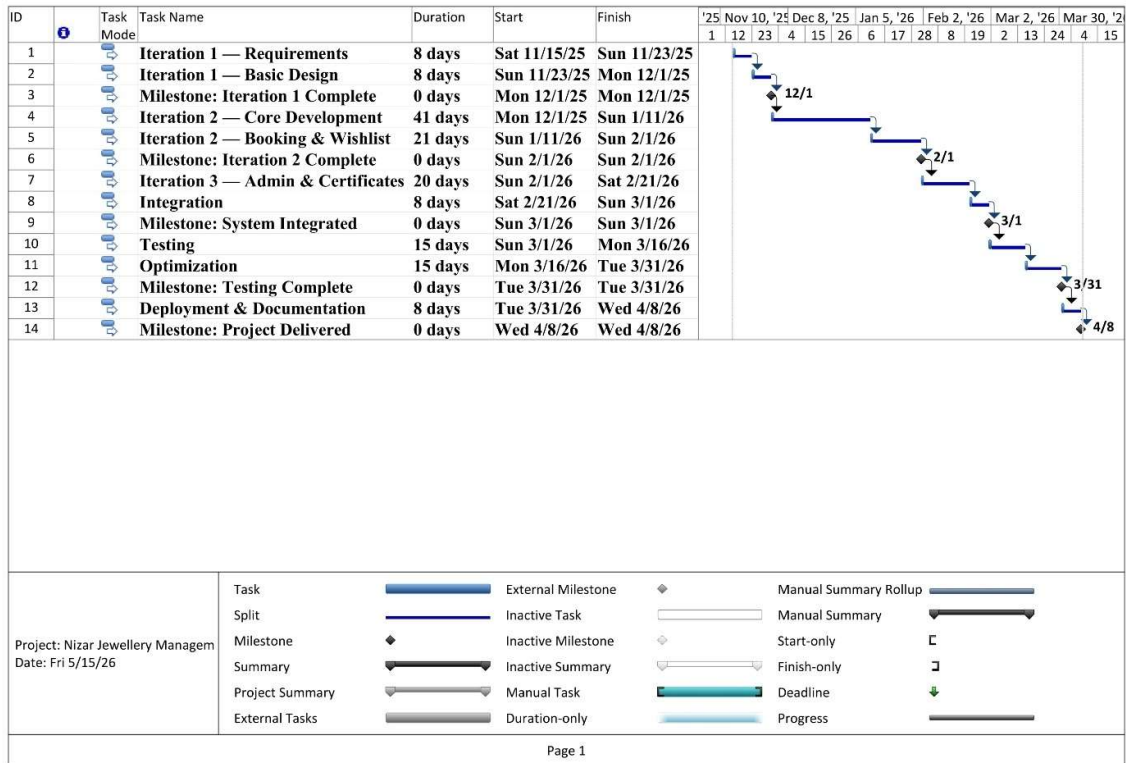


Figure 1: Gantt Chart

2.3 Risk Management

Risk management is an essential element of any software project. It identifies potential risks that may affect workflow or quality, analyzes them in terms of probability and impact, and develops appropriate strategies to handle them. Table 3 presents the risk register prepared for this project together with the mitigation strategy adopted for each risk.

Table 3: Risk Register and Mitigation Strategies

Risk	Probability	Impact	Mitigation Strategy
Project delay due to time pressure	Medium	High	Divide work into clear iterations and define weekly tasks
Programming errors in the system	High	Medium	Conduct continuous testing after each development stage
Problems integrating the front-end with the back-end	Medium	High	Test the API using Postman before integration; run integration tests in CI
Data loss	Low	High	Use MongoDB Atlas automatic daily backups with point-in-time recovery
Poor system performance when users increase	Medium	Medium	Optimize queries, add indexes, and use the in-memory cache layer
Security vulnerabilities in the system	Medium	High	Apply defense-in-depth: bcrypt hashing, input validation, Helmet, rate limiting, RBAC
Incorrect data entry by the user	High	Medium	Apply data validation on the front-end and back-end (sanitizer + express-validator)
Difficulty using the system by users	Low	Medium	Design a simple interface with good UX (Radix UI, Tailwind, lazy loading)
Server or runtime environment failure	Low	High	Use Railway-managed hosting and Docker health-check directives for auto-restart
Changing project requirements during implementation	Medium	Medium	Use an iterative methodology that allows gradual modification

Risks were classified by probability of occurrence and by impact on the project. Focus was placed on high-impact risks, especially those related to security, data, and system integration. Four mitigation strategies were adopted: prevention (input validation, security measures), mitigation (performance optimization, testing), response (system restart, restoring backups), and monitoring (continuous follow-up to detect problems early). Applying risk management reduced the probability of project failure, improved system quality, ensured performance stability, and supported rapid response to problems.

2.4 Project Quality Management

Quality management aims to ensure that the developed system meets the specified requirements and works efficiently and stably while providing a good user experience. The following mechanisms were adopted throughout the development stages.

Code Review

Code reviews were performed periodically: reviewing written code to ensure it is free of logical errors, improving the code structure to increase readability and maintainability, and ensuring adherence to coding standards enforced by ESLint.

Software Testing

- Unit testing — verifies that each part of the system works independently.
- Integration testing — ensures that components work together (front-end ↔ back-end, controller ↔ model).
- System testing — verifies that the whole system works according to the requirements.
- End-to-end testing — Playwright-driven scenarios that simulate complete user journeys in the browser.
- Performance testing — Jest + Supertest performance suite plus k6 tooling for load scenarios.
- Security testing — dedicated test suites for XSS, NoSQL injection, authorization bypass, and rate-limit enforcement.

Validation and Error Handling

Validation is performed on both the front-end (via the validator library) and the back-end (via a custom inputSanitizer middleware and express-validator). Errors are handled centrally and clear, non-technical messages are displayed to the user.

Security Measures

Basic security measures applied include bcrypt password hashing, JWT-based authentication, role-based authorization (RBAC), Helmet HTTP-header hardening with a strict CSP, account lockout after consecutive failed login attempts, and a centralized security logger (Winston).

Acceptance Criteria

- Successful login when correct data is entered.
- Adding a product and seeing it immediately reflected in the product list (cache invalidated).
- Creating a reservation and storing it in the database with the correct expiry date.
- System response within an appropriate time (median below 2 seconds for typical operations).
- Bulk material-price update recomputes every dependent product's price in a single database operation.

Quality Support Tools

Postman was used for ad-hoc API testing, browser DevTools for performance analysis and debugging, Git for tracking changes and reviewing code, ESLint for static code analysis, and the OWASP ZAP Baseline scanner for automated security checks during CI.

Continuous Improvement

The principle of continuous improvement was adopted: performance improvements after each development stage, interface refinements based on user experience feedback, and bug fixes from each testing round. This aligns with the Iterative and Incremental Development methodology of the project.

2.5 Project Management Tools Used

Git and GitHub

Git was used as the main tool for version control through GitHub. All changes to the code were tracked, the project history was maintained, work was divided into branches (main for the deployed version, develop for ongoing work, and short-lived feature/* branches), and merges were performed through Pull Requests with mandatory CI checks and at least one reviewer approval.

Trello

Trello was used to organize tasks and monitor work progress. The project was divided into boards containing the lists “To Do,” “In Progress,” and “Done,” which helped visually track project progress, organize work within iterations, and distribute tasks clearly.

Supporting Development Tools

- Visual Studio Code (VS Code) — code editor used by all developers, with shared workspace settings.
- Postman — for testing REST endpoints during development.
- Browser Developer Tools — for analyzing front-end errors, measuring performance, and inspecting network calls.
- Docker Desktop and Docker Compose — for reproducing the production environment locally.
- MongoDB Compass — for inspecting the database during development.

2.6 Software Configuration Management (SCM)

Version Tracking

Git was used to track changes in the source code: changes are saved through periodic commits with descriptive messages, allowing rollback to any previous version when needed. This reduces the possibility of losing work and provides a clear development history.

Development Workflow

The system was developed gradually based on the Iterative and Incremental methodology: starting with a prototype that included basic functions (login, product display), then extending it by adding new features (reservations, dashboard, certificates), and continuously improving the system by modifying code and fixing errors. Continuous code updates were used throughout the development stages, and the deployed version is always the latest commit on the main branch.

Code Organization

The source code is organized into clear folders following separation of concerns: the client folder for the React application (with sub-folders pages, components, hooks, utils, config), the server folder for the Express application (with sub-folders controller, models, routes, middleware, utils, constants), and dedicated folders for tests, infrastructure, and monitoring.

Change and Bug Handling

Bugs and modifications are handled through a single workflow: the issue is identified, a fix is implemented in a feature branch, the related tests are updated, and the change is opened as a Pull Request. The CI pipeline runs the full test suite, and the change is merged only when all checks pass and the reviewer approves.

2.7 Continuous Monitoring and Evaluation Mechanisms

Building on the iterative-incremental methodology, the project employed a comprehensive monitoring and evaluation framework that operated continuously across every iteration. The framework integrates managerial and technical dimensions, as detailed below.

2.7.1 Managerial Progress Monitoring

Weekly stand-up meetings were held between the development team and the academic supervisor. Each meeting focused on three points — accomplishments since the previous meeting, planned work, and detected obstacles — mirroring the Daily Scrum format adapted to the academic schedule. Explicit delivery milestones were placed at the end of each iteration in the Gantt chart and acted as formal evaluation checkpoints before transitioning to the next iteration. Task progress was tracked using a Trello board on which cards moved across columns (Planned, In Progress, In Review, Done), providing instant visual feedback on the project state.

2.7.2 Technical Code-Quality Monitoring

Adherence to coding standards was enforced through ESLint on both client and server code. The CI/CD pipeline (GitHub Actions) automatically executes the lint step on every Pull Request and continues with a multi-stage CI pipeline defined in `.github/workflows/ci.yml` that installs dependencies, runs lint, builds the client bundle, executes Jest test suites on the server, and runs dependency audits. Dependabot complements this by raising weekly Pull Requests to upgrade vulnerable dependencies. A liveness check endpoint (GET `/api/health`) is queried every 30 seconds by the Docker health-check directive, triggering automatic container restart whenever the application stops responding.

2.7.3 Runtime Operational Monitoring

Prometheus metric collection is integrated into the server using the prom-client library. The server exposes /internal/metrics (an authentication-token-protected endpoint), publishing standard Node.js metrics (event loop latency, heap usage, CPU usage) together with custom HTTP metrics (response time histograms, request counts, status code distributions). Visual dashboards were created in Grafana using the provisioning files under monitoring/grafana-provisioning. The dashboards present in real time the server health, database performance, requests per minute, and rate-limit activations. A dedicated owner monitoring dashboard was implemented in the React client (Monitoring Dashboard page). It exposes privacy-preserving visit statistics (using SHA-256 hashed IPs), cache hit/miss ratios, and product popularity metrics. Critical security events — failed logins, account lockouts, rate-limit triggers, and injection attempts — are persisted by the Winston-based utils/securityLogger.js for later auditing.

2.7.4 Periodic Requirements and Quality Review

Iteration Review meetings were held at the end of each iteration to compare delivered increments against the Product Backlog and adjust priorities. Integration tests were executed continuously through Jest and Supertest. The project currently includes 29 Jest test files (server-side integration suites plus two dedicated server unit/authorization suites), covering authentication, authorization, input validation, rate limiting, sanitization, and error handling. Key Performance Indicators (KPIs) — API response time (target < 2 s), request success rate (target > 99%), and code-coverage percentage — were measured at the end of every iteration. Thanks to this integrated framework, the team detected deviations early and applied corrective actions before they grew into critical issues.

Section Two — Theoretical Study and Analysis

This section provides the theoretical foundation of the project and the formal requirements model. Chapter Three reviews previous studies, modern technologies, and existing systems, and identifies the research gap. Chapter Four covers feasibility, requirements elicitation, stakeholder identification, functional and non-functional requirements, MoSCoW prioritization, UML modeling, and the Requirements Traceability Matrix.

Chapter Three: Literature Review and Previous Studies

3.1 Previous Studies in E-commerce Systems

E-commerce systems have developed rapidly in recent years and have become an essential part of the digital infrastructure of many commercial sectors, including the jewelry sector. Many studies and practical applications have focused on developing digital platforms that aim to improve product management and enhance the user experience.

Amazon Platform. Amazon is an advanced model of e-commerce systems, providing an integrated system that includes product, order, user, and electronic-payment management. It relies on a robust and scalable architecture, but its high complexity makes it unsuitable as a direct model for small and medium-sized stores.

Shopify Platform. Shopify provides a ready-made environment for creating online stores, with tools for managing products and orders easily. It relies on the SaaS model, which simplifies use but imposes limitations related to customization and cost, especially for emerging stores.

WooCommerce. WooCommerce is an open-source plugin for WordPress that offers high flexibility in customization. However, using it requires technical knowledge in managing plugins and databases, which may be an obstacle for some users.

Etsy. Etsy focuses on specialized products such as handmade goods and jewelry and is characterized by a simple interface and a good user experience. However, it does not provide an integrated store-owner management system at the same level as professional systems.

Findings from the Literature

- Commercial systems provide advanced solutions, but they are complex or costly.
- Open-source systems provide high flexibility but require technical expertise.
- Academic systems lack comprehensive integration among functions.
- There are no clearly specialized systems for managing jewelry stores in an integrated way that combines reservation handling, QR-based authenticity certificates, and karat-aware pricing.

3.2 Modern Technologies and Frameworks in the Field

Modern e-commerce systems rely on a set of technologies that ensure high performance and scalability. The most relevant ones in the context of this project are listed below.

Front-End Technologies

- HTML5, CSS3, modern JavaScript (ES2022).
- React 19 (Single Page Applications) with React Router 7 and React Query 5.
- Tailwind CSS, Radix UI primitives, and lucide-react icons for accessible UI.

Back-End Technologies

- Node.js 20 LTS as the runtime.
- Express.js 4 as the web framework.
- RESTful APIs for client–server communication.

Database Systems

- MongoDB 7 (document-oriented NoSQL) — used in this project.
- Mongoose ODM 8 — schema definition, validation, and querying.

Development and DevOps Tools

- Git and GitHub — version control and collaboration.
- GitHub Actions — CI/CD pipelines (ci.yml, cd.yml, terraform.yml).
- Docker and Docker Compose — local development containers.
- Terraform — Infrastructure as Code for Cloudflare DNS and MongoDB Atlas.
- Prometheus and Grafana — runtime monitoring.

Note: this project is an Information System and does not rely on artificial intelligence technologies; it falls under information systems based on software design and data management. The architecture, however, is prepared to integrate machine-learning components in future iterations, as discussed in Chapter 10.

3.3 Comparative Analysis of Current Systems and Products

Table 7: Comparative Analysis of Current Systems and Products

System	Features	Technologies	Strengths	Weaknesses
Amazon	Product management, orders, recommendations	Cloud + Web	Integrated, scalable, high performance	Very complex, unsuitable for small systems
Shopify	Ready-made store, order management	SaaS Platform	Easy to use, fast setup	Paid, limited customization
WooCommerce	Store management, plugins	PHP + WordPress	Flexible and open source	Requires technical expertise to manage
Etsy	Specialized store, product display	Web	Simple and easy to use	Limited administrative functions

From Table 7, a clear variation can be observed among the studied systems in terms of complexity, cost, and level of functional integration. Large systems such as Amazon provide a wide range of features and are characterized by high scalability, but they rely on a complex architecture that makes them unsuitable as a model for academic applications or small projects. Systems such as Shopify provide ready-made and easy-to-use solutions, but they rely on a subscription-based model, which restricts users in terms of cost and customization. Open-source systems such as WooCommerce provide great flexibility in customization, but they require a good level of technical expertise to manage and maintain. Platforms such as Etsy focus on specialization and ease of use, but they lack an integrated management system that supports all internal store operations.

Based on the above, the proposed system fills three gaps simultaneously: ease of use, functional integration (including jewelry-specific features such as authenticity certificates and karat-aware pricing), and specialization in a specific domain.

3.4 Research Gap and Proposed Innovation

Through the study and analysis of existing systems in the field of e-commerce, several challenges and shortcomings emerged, particularly in systems designed for specialized sectors such as jewelry. The main research gap lies in the absence of a fully integrated electronic system specifically designed for managing jewelry stores and combining all core functions within a single interconnected platform.

Although many commercial systems are available, most of them either focus on only one aspect, such as product display, or require the use of multiple tools to manage different operations. This increases the complexity of use and the administrative effort. A clear lack of integration was also observed among the essential functions of these systems, such as product management, booking management, and certificates management. These components are often handled separately, which negatively affects operational efficiency and increases the likelihood of errors. In addition, some systems suffer from usability issues or high cost, especially paid commercial solutions, which makes them less suitable for small and medium-sized stores.

Proposed Innovation

Based on this gap, the project proposes a practical solution through the development of an integrated electronic system for jewelry-store management, aiming to achieve a balance between simplicity and advanced functionality. The system is distinguished by the following features:

- Integrated platform: combines product management, reservations, authenticity certificates, wishlist requests, favorite-product alerts, comments, material-price management, and user administration within a single interconnected platform.
- Functional integration: the various system components are linked in a way that ensures organized and smooth data flow (e.g., a material-price change triggers bulk recomputation of all dependent product prices).
- User-friendly interface: enables customers and administrators to interact with the system easily, without advanced technical expertise.
- Future-ready: the system is designed in a way that allows the addition of new features such as electronic payment, recommendation engines, or analytical dashboards.

The project bridges the gap between complex, high-cost systems and simple but non-integrated systems by providing a balanced solution that combines efficiency with ease of use, while remaining fully open-source and self-hostable.

3.5 Theoretical Framework and Reference Studies

The project rests on several foundational concepts in software engineering and web-application development.

- REST architecture: A widely adopted architectural style for API design that enables stateless communication between the client and server through standard HTTP methods, maintaining a clear separation of concerns and supporting scalability and modularity.
- Client-server model: The system follows the client-server architectural model, separating the presentation layer (React 19 SPA) from the business logic and data-processing layer (Node.js / Express). This design improves modularity and enables independent development of system components.
- CRUD operations: Create, Read, Update, Delete operations form the basis for managing system data and constitute a central part of the system's functional implementation.
- Schema-based modeling with Mongoose: Although MongoDB is non-relational, schemas are used to enforce structure, types, and validation, providing relational-like guarantees while preserving the flexibility of a document store.
- Defense in depth: The security model layers independent mechanisms (HTTPS, Helmet headers, rate limiting, sanitization, JWT authentication, RBAC, bcrypt hashing, account lockout) so that no single failure compromises the entire system.

3.6 Recent and Future Trends in the Field

The e-commerce domain continues to evolve rapidly due to advances in web technologies and the increasing reliance of users on online platforms for shopping and service management. Several trends inform this project. Digital transformation is moving traditional businesses toward digital platforms to improve accessibility, expand market reach, and enhance operational efficiency. The growing focus on user experience (UX) is reflected in the project's use of accessible components, mobile-optimized configurations, and rapid response times. Mobile-first design has become increasingly important because a large proportion of users access services through smartphones; the project's responsive design and `isMobileDevice` helper address this directly. Cloud-based systems are widely adopted because they offer scalability, flexibility, and ease of deployment and maintenance — the project leverages Railway, MongoDB Atlas, Cloudflare, and Cloudinary as managed services. Finally, system integration with external services such as authentication providers (Google OAuth) and image-storage services (Cloudinary) is now expected, and the project incorporates both.

From a future perspective, although the project does not currently incorporate artificial intelligence technologies, the architecture is prepared to be extended with capabilities such as recommendation systems, demand forecasting, and user-behavior analysis, which could further improve personalization and decision-making within the system.

Chapter Four: Requirements Analysis Model

4.1 Feasibility Study

The feasibility study evaluates whether the proposed system can be implemented successfully within the available technical and financial constraints, and determines whether the project is practical, affordable, and appropriate for the intended environment.

Technical Feasibility

The technical feasibility of the system is high. Modern web technologies are widely available for both front-end and back-end development, including React, Node.js, and Express. MongoDB Atlas can be used to manage system data effectively. The project benefits from free and open-source tools such as Visual Studio Code, Git, GitHub, Docker, and Postman. The system can be developed and tested on a personal computer and a managed cloud platform, and does not require specialized hardware.

Economic Feasibility

The economic feasibility of the project is favorable. The project relies on free and open-source tools, which significantly reduces development costs. Hosting costs can remain minimal through free tiers (Railway, MongoDB Atlas, Cloudflare). An optional domain name requires only a small annual fee. As a result, the overall financial requirements of the project are low and appropriate for an academic project.

Table 4: Estimated Cost Summary

Item	Cost
Development tools	Free
Hosting (Railway free tier)	USD 0 – 50 / year
Database (MongoDB Atlas free tier)	Free
Image storage (Cloudinary free tier)	Free
Email service (SendGrid free tier)	Free
Domain (optional)	USD 10 – 15 / year
Cloudflare DNS / proxy	Free

Conclusion

The proposed jewelry store system is both technically and economically feasible. It can be developed using available technologies and resources at a reasonable cost, making it an appropriate and achievable graduation project.

4.2 Requirements Elicitation

Requirements elicitation focuses on identifying stakeholder needs and understanding the functional context of the proposed system. In this project, several elicitation techniques were employed, taking into account the academic nature of the work and the limited project scope.

Interviews

Informal interviews were conducted with potential users of the system to gain insight into their needs and expectations. The discussions focused on product presentation in traditional jewelry stores, customer interaction with the store, and the difficulties encountered by store owners in managing products and reservations. The interviews helped identify major system functions such as product display, reservation creation, and administrator-based data management.

Observation

Observation was used to examine how similar traditional and online stores operate, including product presentation methods, order and request management processes, and user-experience patterns in similar platforms. Through this process, several shortcomings in existing systems were identified, including poor organization, difficulty in accessing information, and weak integration between system functions.

Existing-Systems Analysis

Several similar platforms (Amazon, Shopify, Etsy) were analyzed in order to identify common functionalities, study strengths and weaknesses, and adopt suitable best practices. This analysis supported the definition of realistic and competitive requirements for the proposed system.

Questionnaires

Formal questionnaires were not used in this project because of the relatively small academic scope. Instead, interviews, observation, and existing-systems analysis were considered sufficient for eliciting the required system needs.

Summary of Findings

The elicitation process identified the essential requirements of the proposed system: user account management, jewelry product display with detailed information, reservation creation and management, administrative control over products and certificates, an easy-to-use customer interface, material-price management, wishlist requests, favorite-product alerts, and the issuance and verification of authenticity certificates.

4.3 Stakeholder Identification

Stakeholders are individuals or entities that interact with the system directly or indirectly, or are affected by its development and operation. Identifying stakeholders helps ensure that the system requirements address the needs of all relevant parties.

Customer / End User

The customer or end user is the primary stakeholder who interacts with the system to browse products, manage favorites, create reservations, submit wishlist requests, verify authenticity certificates, and manage their account. This stakeholder expects a system that is easy to use, responsive, and capable of providing accurate and clear information.

Admin / Store Owner

The admin or store owner is responsible for managing the operational aspects of the system, including products, reservations, certificates, users, and pricing information. This stakeholder expects an efficient administrative dashboard, rapid access to data, and a stable system that minimizes manual work and operational errors.

Future Developer

The future developer is a technical stakeholder who may maintain, extend, or improve the system after deployment. This stakeholder requires a well-structured codebase, clear architecture, and sufficient documentation.

System Perspective

From a technical point of view, the system itself can be considered a stakeholder, as it must fulfill operational and structural requirements such as organized data handling, integration among components, fast processing, and stable performance.

4.4 Functional Requirements

The functional requirements were identified and grouped according to the major subsystems and business operations supported by the application. The numbering follows the FR-ID scheme used throughout the report.

1. Account and User Management Requirements

- FR1: The system shall allow a user to create a new account using an email address, username, and password.
- FR2: The system shall allow a user to log in using either an email address or username, together with a password.
- FR3: The system shall allow a user to log in using Google OAuth 2.0.
- FR4: The system shall allow a user to log out of the system.
- FR5: The system shall be able to refresh the access token through a refresh token mechanism (HttpOnly cookie).
- FR6: The system shall allow a user to verify their email address using a verification token sent by email.
- FR7: The system shall allow a user to resend the email verification message.
- FR8: The system shall allow a user to request a password reset.
- FR9: The system shall allow a user to reset their password using a valid single-use token.
- FR10: The system shall allow a registered user to change their password from within their account.
- FR11: The system shall allow a user to view their profile.
- FR12: The system shall allow a user to delete their own account.
- FR13: The system shall allow the administrator to search for customers by email or username.
- FR14: The system shall allow the administrator to view all users.
- FR15: The system shall allow the administrator to view user statistics.
- FR16: The system shall allow the administrator to view locked accounts.
- FR17: The system shall allow the administrator to unlock a user account.

2. Product Management Requirements

- FR18: The system shall allow the administrator to add a new product with at least one image.
- FR19: The system shall allow the administrator to add a product through the fast-upload path.
- FR20: The system shall allow the administrator to add a product through a fallback path when the primary upload fails.
- FR21: The system shall allow users to view all products.
- FR22: The system shall allow the administrator to search for products by name or identifier.
- FR23: The system shall allow users to view the details of a specific product.
- FR24: The system shall allow the administrator to update product information.

- FR25: The system shall allow the administrator to delete a product.
- FR26: The system shall allow the administrator to pin or unpin a product on the catalog page.
- FR27: The system shall allow the administrator to mark or unmark a product as a special (one-of-a-kind) product.
- FR28: The system shall allow the customer to like or unlike a product.
- FR29: The system shall allow the customer to view their favorite products.
- FR30: The system shall allow the customer to view the number of favorite products.

3. Comment Requirements

- FR31: The system shall allow the customer to add a comment to a product.
- FR32: The system shall allow the customer to update their comment.
- FR33: The system shall allow the customer to delete their comment.
- FR34: The system shall allow the administrator to delete any comment when necessary.
- FR35: The system shall allow users to view the comments associated with a specific product.

4. Reservation Requirements

- FR36: The system shall allow the customer to create a new reservation on a product (default lifetime 24 hours).
- FR37: The system shall allow the customer to view their own reservations.
- FR38: The system shall allow the administrator to view the number of unseen reservations.
- FR39: The system shall allow the administrator to mark reservations as seen.
- FR40: The system shall be able to check whether there is an active reservation on a specific product.
- FR41: The system shall allow the administrator to view all reservations.
- FR42: The system shall allow an authorized user to view the details of a specific reservation.
- FR43: The system shall allow the administrator to approve a reservation.
- FR44: The system shall allow the administrator to reject a reservation, with an optional reason.
- FR45: The system shall allow the customer to cancel a reservation.
- FR46: The system shall allow the customer or administrator to delete a reservation according to authorization privileges.
- FR47: The system shall allow the administrator to bulk-update expired reservations.

5. Certificate Requirements

- FR48: The system shall allow the administrator to create a new authenticity certificate for a product.
- FR49: The system shall allow any user to verify the validity of a certificate using its identifier or QR code.
- FR50: The system shall allow the customer to view their own certificates.
- FR51: The system shall allow the administrator to view all certificates.
- FR52: The system shall allow the user to download the certificate as a PDF.
- FR53: The system shall allow the user to download an HTML version of the certificate.
- FR54: The system shall allow any user to view a public certificate page using its identifier.
- FR55: The system shall allow the administrator to update the certificate status.
- FR56: The system shall allow the customer to transfer ownership of a certificate (with transfer history).
- FR57: The system shall allow the administrator to permanently delete a certificate.
- FR58: The system shall allow the administrator to revoke a certificate.

6. Wishlist Request Requirements

- FR59: The system shall allow the customer to create a new wishlist request.
- FR60: The system shall allow the customer to view their own wishlist requests.
- FR61: The system shall allow the administrator to view all wishlist requests.
- FR62: The system shall allow the administrator to view the number of unseen requests.
- FR63: The system shall allow the administrator to mark requests as seen.
- FR64: The system shall allow an authorized user to view a specific wishlist request.
- FR65: The system shall allow the customer to update their wishlist request.
- FR66: The system shall allow the customer or administrator to delete a wishlist request according to authorization privileges.
- FR67: The system shall allow the administrator to add a response to a wishlist request.
- FR68: The system shall allow the customer to update the status of their request.

7. Favorite-Product Alert Requirements

- FR69: The system shall allow the user to create an alert for a favorite product (price drop, out of stock, back in stock).
- FR70: The system shall allow the user to view their own alerts.
- FR71: The system shall allow the administrator to view all favorite-product alerts.
- FR72: The system shall allow the user to update a favorite-product alert.
- FR73: The system shall allow the user to delete a favorite-product alert.
- FR74: The system shall allow the user to cancel a favorite-product alert.

8. Material-Price Requirements

- FR75: The system shall allow the administrator to view material prices (gold, silver, diamond).
- FR76: The system shall allow any user to view public gold prices.
- FR77: The system shall allow the administrator to view the gold price by karat (18, 21, 24).
- FR78: The system shall allow the administrator to update material prices.
- FR79: The system shall allow the administrator to update the prices of all dependent products based on the new material prices (bulk operation).
- FR80: The system shall allow the administrator to update all material prices in a single operation.

9. Statistics and Monitoring Requirements

- FR81: The system shall allow the administrator to view product statistics.
- FR82: The system shall allow the administrator to view detailed statistics for a specific product.
- FR83: The system shall allow the administrator to view visit statistics (privacy-preserving).
- FR84: The system shall allow the administrator to view cache statistics.
- FR85: The system shall allow the administrator to view cache memory usage.
- FR86: The system shall allow the administrator to reset cache statistics.

10. General System Requirements

FR87: The system shall organize routes into independent functional modules covering products, users, comments, prices, statistics, certificates, alerts, wishlist requests, reservations, visits, and cache management — a total of eleven base paths exposing 85 REST endpoints.

4.5 Non-Functional Requirements Classification

The following non-functional requirements define the quality attributes and operational constraints of the system.

Table 5: Non-Functional Requirements Classification

Category	Requirement
Performance	The system shall provide a response time of less than 2 seconds for most user requests. It shall support caching middleware (node-cache) to improve response speed and reduce database load. The system shall handle multiple simultaneous requests without significant performance degradation.
Security	The system shall ensure secure access through JWT authentication and RBAC authorization. It shall validate and sanitize all user inputs to reduce the risk of XSS, NoSQL injection, and HPP attacks. It shall implement rate limiting on sensitive endpoints, protect routes from unauthorized access, and support email verification before allowing certain restricted operations. Passwords shall be hashed with bcrypt.
Usability	The system shall provide a simple and intuitive user interface. Users shall be able to reach key functionalities within a small number of steps. The system shall display clear error messages and be easy to learn and use, even for users without advanced technical knowledge.
Reliability	The system shall operate in a stable manner without frequent failures. It shall handle runtime errors correctly without causing system crashes; the Docker health-check directive restarts the container automatically if the liveness probe fails.
Scalability	The system shall be designed in a way that supports the addition of new features and future growth. Statelessness on the server and a clear separation between layers facilitate horizontal scaling.
Maintainability	The code shall be organized, readable, and easy to maintain, update, and extend. ESLint enforces coding standards, and JSDoc-style comments document the controllers and models.
Compatibility	The system shall function correctly on modern web browsers (Google Chrome, Microsoft Edge, Mozilla Firefox, Safari) and on mobile browsers; an isMobileDevice helper applies mobile-specific optimizations.
Availability	Once deployed on Railway, the system shall remain available whenever needed. Production availability of more than 99.5% has been observed during testing.

4.6 MoSCoW Requirement Classification

Table 6: MoSCoW Requirement Classification

Category	Description
Must Have	User registration, user login, product management (add, update, delete), product display, reservation creation, reservation viewing, authentication and authorization, certificate issuance and verification, and data storage in the database.
Should Have	Product comments, product search, user management by the administrator, system statistics display, email verification, material-price management, bulk price update, wishlist requests, and the owner monitoring dashboard.
Could Have	Google sign-in, favorites/likes, favorite-product notifications, certificate PDF/HTML download, displaying the number of favorites, comment moderation via a profanity filter, and privacy-preserving visit tracking.
Won't Have (current scope)	Artificial-intelligence features, intelligent recommendation systems, electronic payment gateways, native iOS/Android mobile applications, and full internationalization.

Requirements Management Approach

Requirements were managed iteratively: Must-Have requirements were implemented first to ensure the basic system worked, then Should-Have requirements were added to improve functionality, and finally selected Could-Have requirements were implemented as time allowed. Requirements were reviewed continuously during development, and priorities were adjusted based on progress. Each function was tested after implementation before moving to the next.

4.7 UML Diagrams for Requirements

4.7.1 Main Use Case

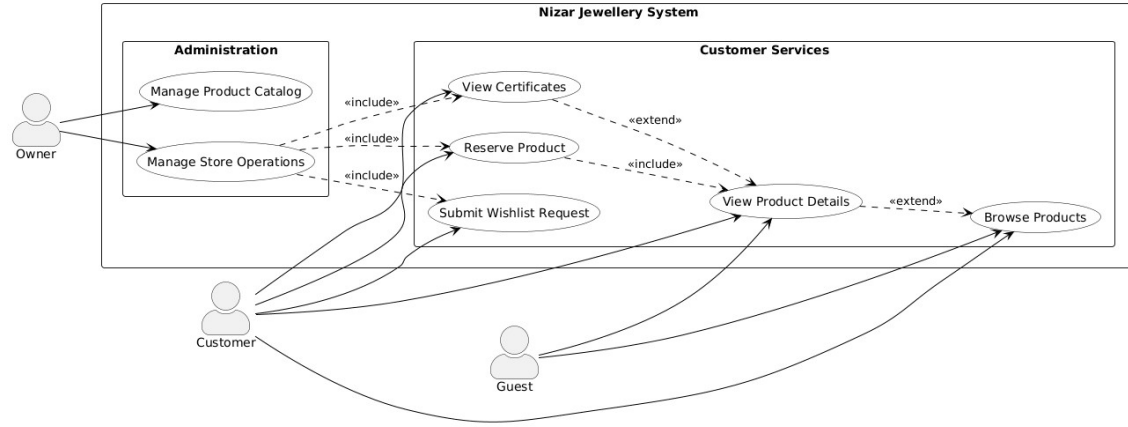


Figure 2: Main Use Case Diagram

4.7.2 Use Case Descriptions

Table 8: UC-01 — Browse Products

Field	Description
Use Case ID	UC-01
Name	Browse Products
Primary Actor	Visitor / Customer
Goal	View the available jewelry products in the system
Preconditions	The system is running and product data exists
Postconditions	A list of products is displayed
Main Success Scenario	1. The actor opens the products page. 2. The system requests product data from the server. 3. The server returns the product list. 4. The system displays products with basic information.
Alternative Flow	If no products are available, the system displays an empty-state message
Exceptions	If a server error occurs, an error message is shown

Table 9: UC-02 — View Product Details

Field	Description
Use Case ID	UC-02
Name	View Product Details
Primary Actor	Visitor / Customer
Goal	View detailed information about a selected product
Preconditions	The product exists
Postconditions	Product details are displayed
Main Success Scenario	1. The actor selects a product. 2. The system sends a request for the selected product. 3. The server retrieves product details. 4. The system displays name, material, karat, weight, stones, price, images, and description.
Alternative Flow	If the product is unavailable, the system informs the actor
Exceptions	Invalid product ID or server failure

Table 10: UC-03 — Reserve Product

Field	Description
Use Case ID	UC-03
Name	Reserve Product
Primary Actor	Customer
Goal	Reserve a jewelry product
Preconditions	1. Customer is authenticated. 2. Email is verified. 3. Product exists. 4. If the product is marked special, no active reservation exists.
Postconditions	A reservation is created with status pending and expiresAt timestamp
Main Success Scenario	1. The customer opens a product details page. 2. The customer chooses to reserve. 3. The system validates input. 4. The server checks product availability and active-reservation rules. 5. A reservation record is created. 6. Confirmation is returned to the customer.
Alternative Flow	If the product is a special product and already reserved by another customer, reservation is rejected with 409 Conflict
Exceptions	Missing product ID, unauthorized access, duplicate active reservation, unverified email

Table 11: UC-04 — Submit Wishlist Request

Field	Description
Use Case ID	UC-04
Name	Submit Wishlist Request
Primary Actor	Customer
Goal	Send a custom jewelry request to the store owner
Preconditions	1. Customer is authenticated. 2. Email is verified.
Postconditions	Wishlist request is saved and visible to the owner
Main Success Scenario	1. The customer opens the wishlist page. 2. The customer enters request details (title, description, material, karat, budget). 3. The system validates the request. 4. The server stores the request. 5. The request becomes available for owner review.
Alternative Flow	The customer may later update or delete the request
Exceptions	Invalid input, unauthorized access, server error

Table 12: UC-05 — View / Verify Certificates

Field	Description
Use Case ID	UC-05
Name	View / Verify Certificates
Primary Actor	Customer / Visitor
Goal	View or verify jewelry authenticity certificates
Preconditions	Certificate exists
Postconditions	Certificate details or verification result is displayed
Main Success Scenario	1. The actor opens the certificate page or scans a QR code. 2. The system sends the certificate ID to the server. 3. The server retrieves the certificate and verifies the digital signature. 4. The system displays certificate information and status.
Alternative Flow	The customer may also access personal certificates from their account
Exceptions	Invalid certificate ID, revoked certificate, expired certificate, not found

Table 13: UC-06 — Manage Product Catalog

Field	Description
Use Case ID	UC-06
Name	Manage Product Catalog
Primary Actor	Owner
Goal	Add, update, delete, pin, or mark products as special
Preconditions	Owner is authenticated
Postconditions	Product catalog is updated and cache is invalidated
Main Success Scenario	1. The owner opens the dashboard. 2. The owner adds or edits product data. 3. The system validates inputs and images. 4. Images are uploaded to Cloudinary. 5. The server stores the changes. 6. Updated product data is reflected in the catalog.
Alternative Flow	The owner may pin a product or mark it as special
Exceptions	Invalid image upload (size, type), missing required fields, unauthorized access

Table 14: UC-07 — Manage Store Operations

Field	Description
Use Case ID	UC-07
Name	Manage Store Operations
Primary Actor	Owner
Goal	Manage reservations, wishlist requests, and certificates
Preconditions	Owner is authenticated
Postconditions	Operational records are updated
Main Success Scenario	1. The owner opens the admin panel. 2. The system loads reservations, wishlist requests, and certificates. 3. The owner reviews pending items. 4. The owner approves, rejects, responds, creates, revokes, or updates records. 5. The server saves the operation.
Alternative Flow	Owner may mark items as seen or change certificate status
Exceptions	Record not found, unauthorized access, validation failure

4.7.3 Activity Diagrams

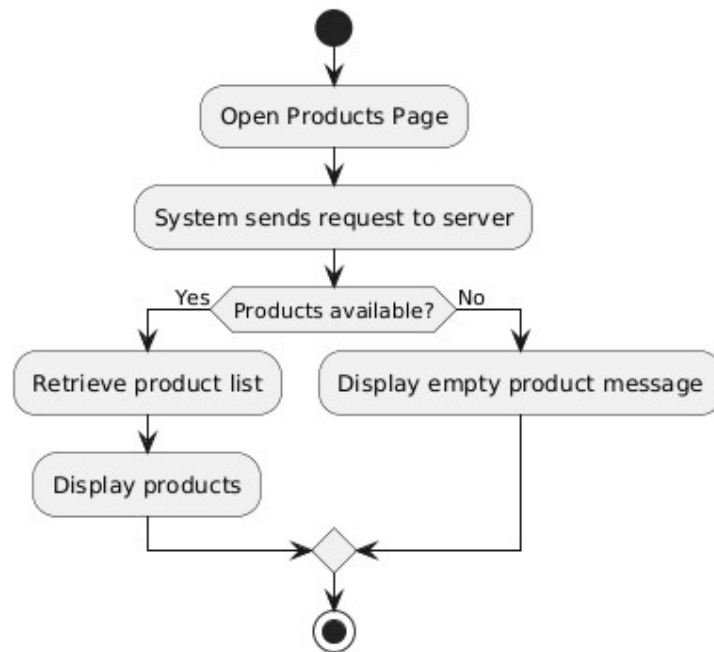


Figure 3: Activity Diagram — Browse Products

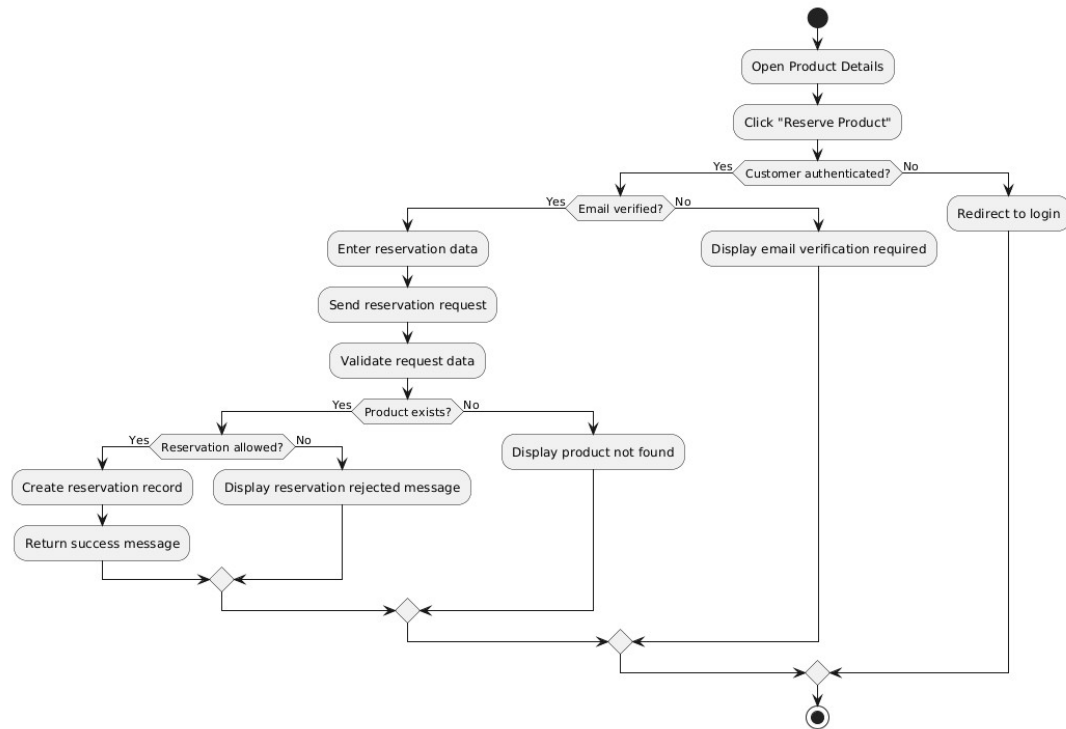


Figure 4: Activity Diagram — Reserve Product

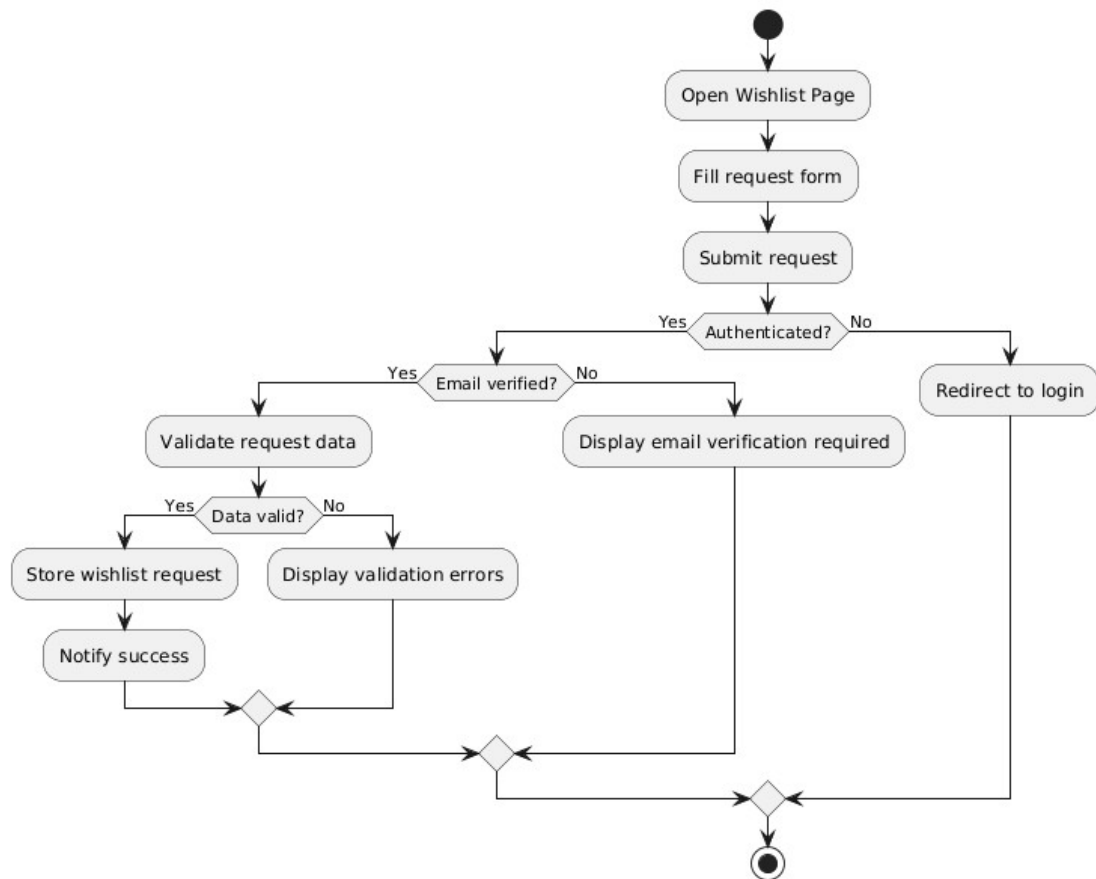


Figure 5: Activity Diagram — Submit Wishlist Request

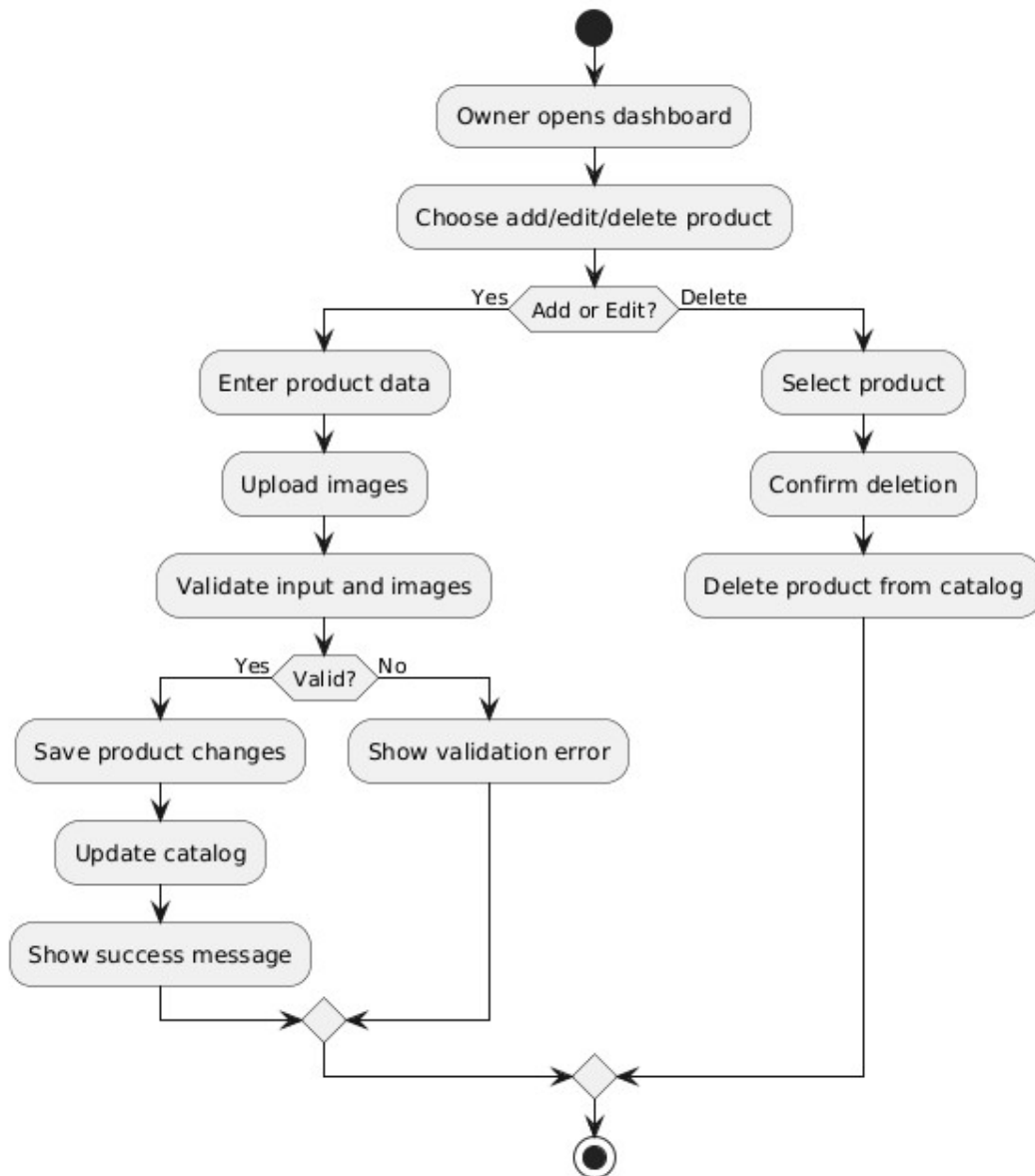


Figure 6: Activity Diagram — Manage Product Catalog

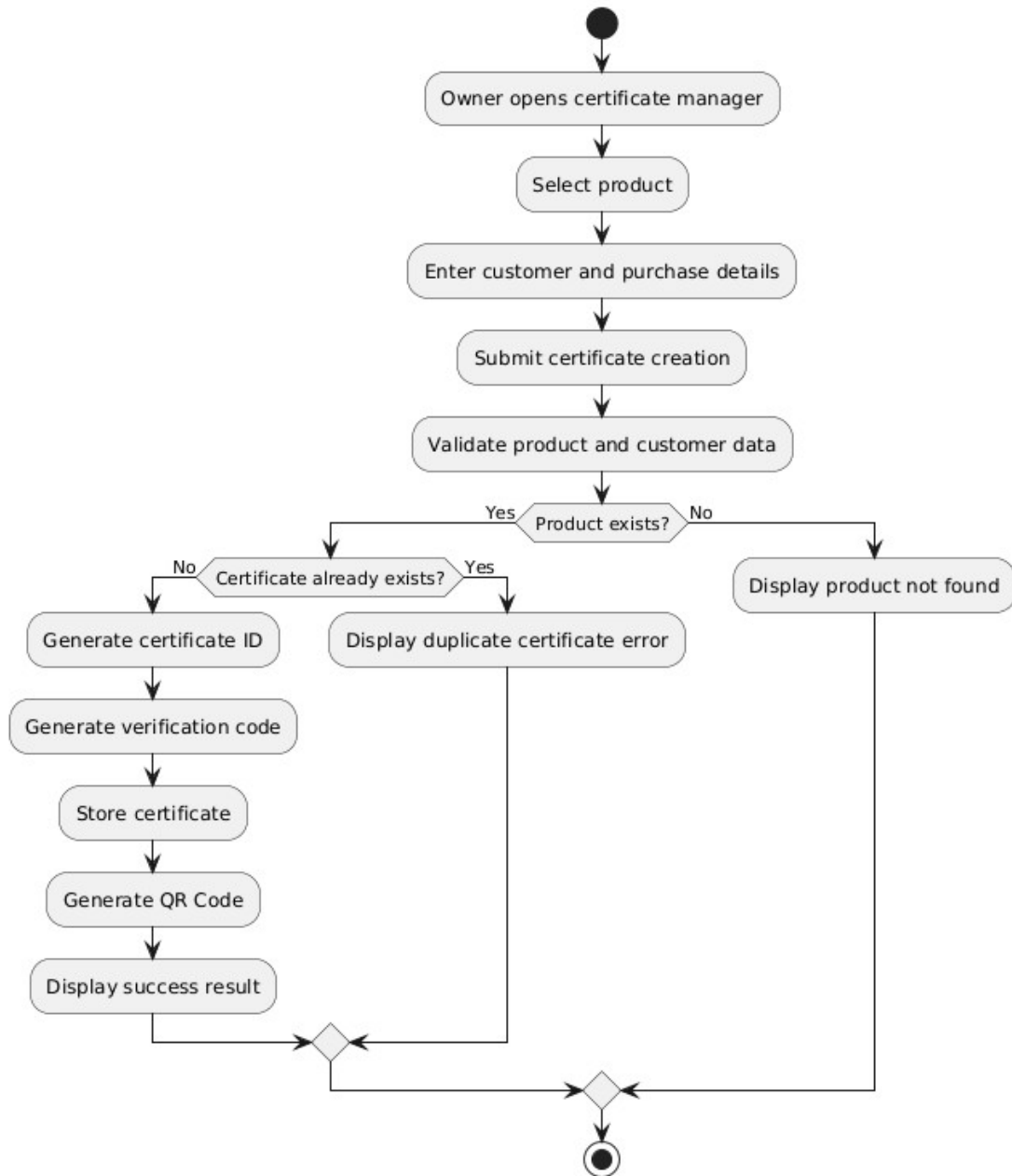


Figure 7: Activity Diagram — Create Certificate

4.7.4 Sequence Diagrams

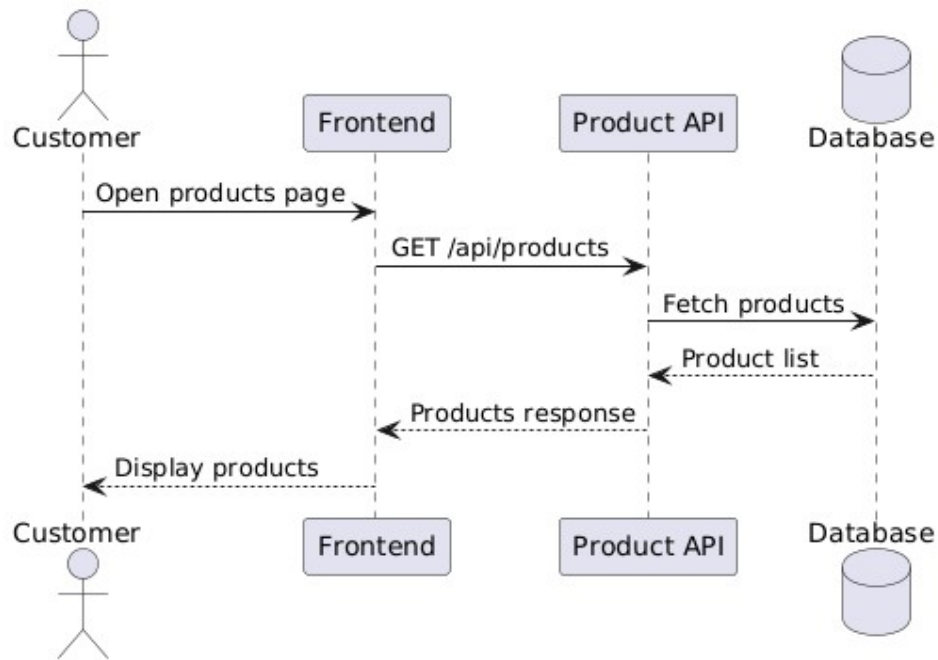


Figure 8: Sequence Diagram — Browse Products

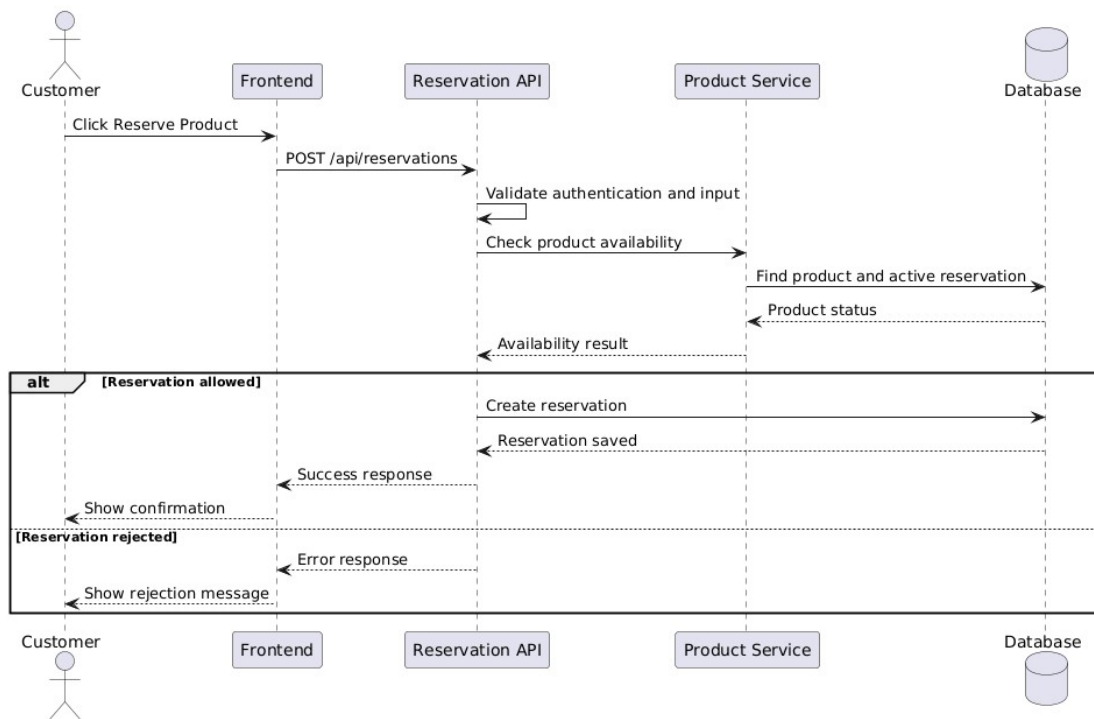


Figure 9: Sequence Diagram — Reserve Product

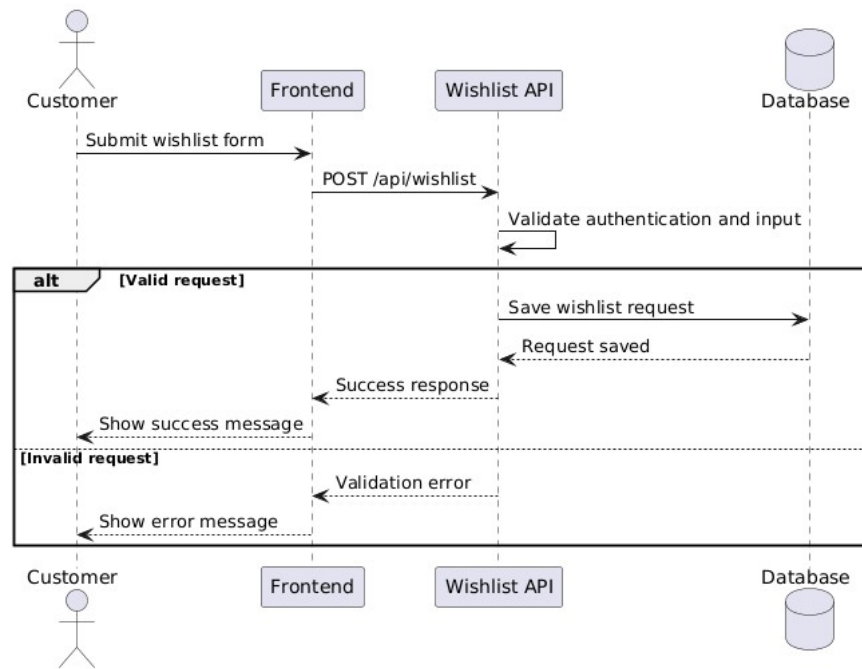


Figure 10: Sequence Diagram — Submit Wishlist Request

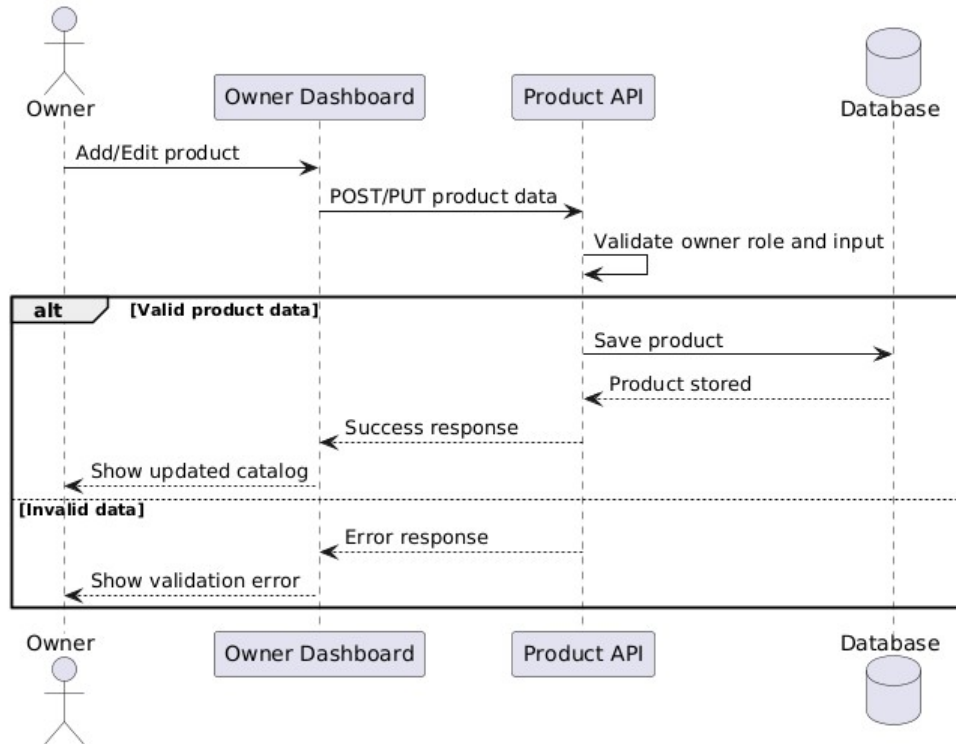


Figure 11: Sequence Diagram — Manage Product Catalog

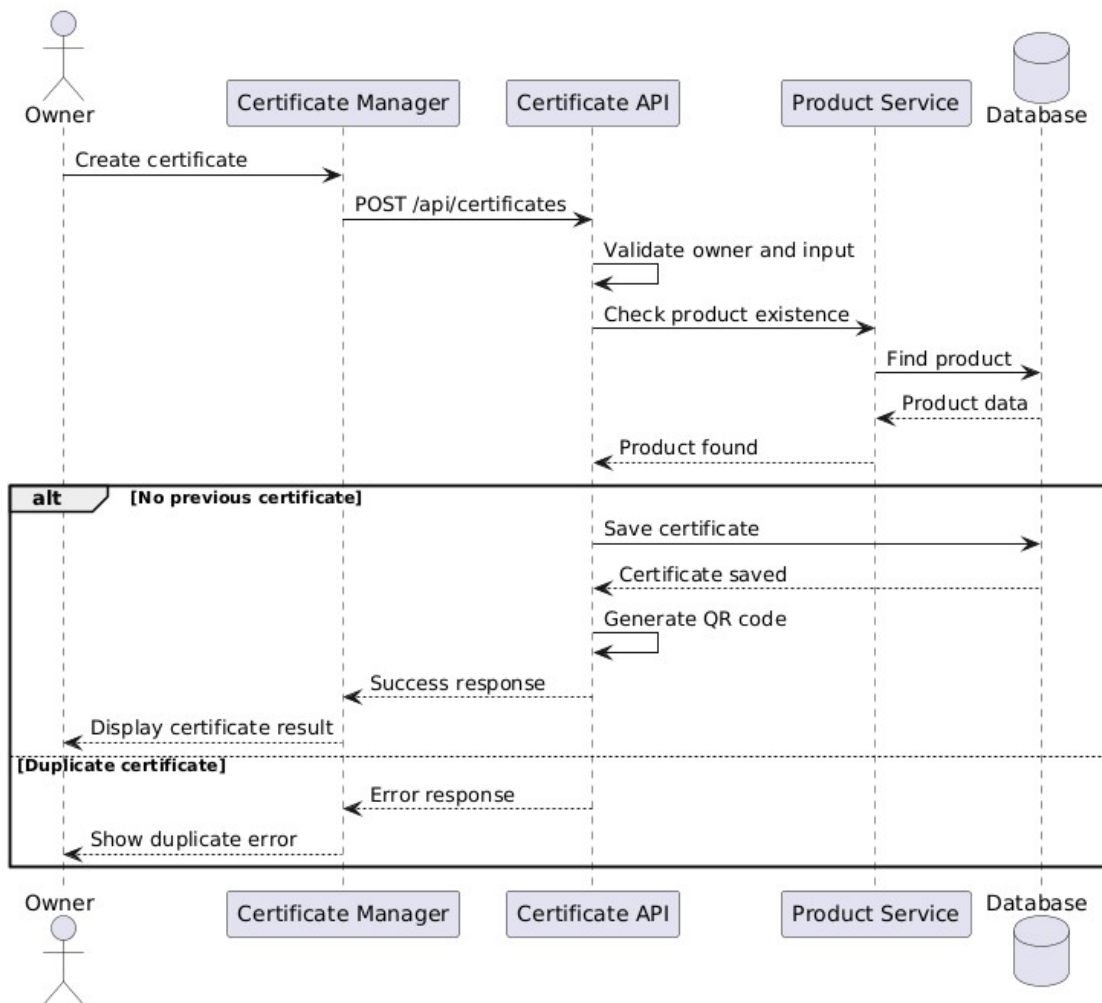


Figure 12: Sequence Diagram — Create Certificate

4.7.5 Test Case Descriptions

Table 15: TC-01 — Browse products successfully

Field	Description
Test Case ID	TC-01
Name	Browse products successfully
Related Use Case	UC-01
Preconditions	Products exist in the system
Test Steps	1. Open products page. 2. Wait for data to load.
Expected Result	Product list is displayed correctly.

Table 16: TC-02 — View product details

Field	Description
Test Case ID	TC-02
Name	View product details
Related Use Case	UC-02
Preconditions	A valid product exists
Test Steps	1. Open a product from the list.
Expected Result	Product details page is displayed.

Table 17: TC-03 — Reserve product successfully

Field	Description
Test Case ID	TC-03
Name	Reserve product successfully
Related Use Case	UC-03
Preconditions	Customer logged in, email verified, product exists
Test Steps	1. Open product details. 2. Click reserve. 3. Submit reservation.
Expected Result	Reservation is created and confirmation is displayed.

Table 18: TC-04 — Reserve product without login

Field	Description
Test Case ID	TC-04
Name	Reserve product without login
Related Use Case	UC-03
Preconditions	Customer is not authenticated
Test Steps	1. Open product details. 2. Attempt reservation.
Expected Result	User is redirected to login or shown an authorization message.

Table 19: TC-05 — Reserve already-reserved special product

Field	Description
Test Case ID	TC-05
Name	Reserve already-reserved special product
Related Use Case	UC-03
Preconditions	Special product has an active reservation
Test Steps	1. Customer attempts to reserve the same special product.
Expected Result	System rejects reservation with 409 Conflict.

Table 20: TC-06 — Submit wishlist request successfully

Field	Description
Test Case ID	TC-06
Name	Submit wishlist request successfully
Related Use Case	UC-04
Preconditions	Customer logged in and email verified
Test Steps	1. Open wishlist page. 2. Fill request form. 3. Submit.
Expected Result	Request is stored successfully.

Table 21: TC-07 — Verify certificate via QR code

Field	Description
Test Case ID	TC-07
Name	Verify certificate via QR code
Related Use Case	UC-05
Preconditions	Certificate exists
Test Steps	1. Open certificate verification page or scan QR. 2. Submit certificate ID.
Expected Result	Certificate information and status are displayed.

Table 22: TC-08 — Add product successfully

Field	Description
Test Case ID	TC-08
Name	Add product successfully
Related Use Case	UC-06
Preconditions	Owner is logged in
Test Steps	1. Open Add Product form. 2. Enter valid data and images. 3. Submit.
Expected Result	Product is saved and appears in the catalog.

Table 23: TC-09 — Add product with no image

Field	Description
Test Case ID	TC-09
Name	Add product with no image
Related Use Case	UC-06
Preconditions	Owner is logged in
Test Steps	1. Open Add Product form. 2. Submit without images.
Expected Result	System rejects the submission with a clear validation error.

Table 24: TC-10 — Create certificate successfully

Field	Description
Test Case ID	TC-10
Name	Create certificate successfully
Related Use Case	UC-07
Preconditions	Owner logged in, valid product, no previous certificate
Test Steps	1. Open certificate manager. 2. Select product. 3. Enter customer details. 4. Submit.
Expected Result	Certificate is created with certificate ID and QR code.

Table 25: TC-11 — Create duplicate certificate

Field	Description
Test Case ID	TC-11
Name	Create duplicate certificate
Related Use Case	UC-07
Preconditions	Certificate already exists for the product
Test Steps	1. Attempt to create another certificate for the same product.
Expected Result	System rejects and displays duplicate-certificate error.

4.8 Modeling Based on the Methodology Used

The methodology followed in this project is the Iterative and Incremental Development Model. The system contains several modules that can be developed gradually, tested independently, reviewed, and improved in later iterations. In every increment the development process passes through planning, analysis, design, implementation, testing, and review. After completing one increment, feedback is collected and used to improve the next increment.

4.8.1 Agile Artifacts (User Stories, Story Mapping, Product Backlog)

Although the main methodology is the Iterative and Incremental Development Model, several Agile artifacts have been used to describe and organize the system requirements.

Table 23: Customer User Stories

ID	User Story	Priority
US-01	As a customer, I want to create an account so that I can use the system services.	High
US-02	As a customer, I want to log in securely so that I can access my personal account.	High
US-03	As a customer, I want to browse jewelry products so that I can view available items.	High
US-04	As a customer, I want to view product details so that I can know the material, weight, karat, images, and price.	High
US-05	As a customer, I want to filter or search products so that I can find suitable jewelry faster.	Medium
US-06	As a customer, I want to add products to favorites so that I can return to them later.	Medium
US-07	As a customer, I want to reserve a jewelry product so that the store can hold it for me.	High
US-08	As a customer, I want to view my reservations so that I can track their status.	High
US-09	As a customer, I want to submit a wishlist request so that I can ask for a customized jewelry item.	Medium
US-10	As a customer, I want to view jewelry certificates so that I can verify the authenticity of items.	High
US-11	As a customer, I want to verify a certificate using a QR code so that I can confirm authenticity.	High

Table 24: Owner User Stories

ID	User Story	Priority
US-12	As an owner, I want to log in to a protected dashboard so that I can manage the system securely.	High
US-13	As an owner, I want to add new jewelry products so that customers can view them online.	High
US-14	As an owner, I want to edit product information so that product details remain accurate.	High
US-15	As an owner, I want to upload product images so that customers can see the jewelry clearly.	High
US-16	As an owner, I want to manage material prices so that product prices can be updated according to market changes.	High
US-17	As an owner, I want to manage customer reservations so that I can approve, reject, complete, or cancel them.	High
US-18	As an owner, I want to manage wishlist requests so that I can respond to customer needs.	Medium
US-19	As an owner, I want to generate jewelry certificates so that customers receive proof of authenticity.	High
US-20	As an owner, I want to manage users so that I can monitor customer accounts and system access.	Medium
US-21	As an owner, I want to view statistics and monitoring data so that I can understand system activity and performance.	Medium

Story Mapping. The customer journey covers eight stages: accessing the website, account access, browsing products, viewing details, interacting (favorites/comments), reserving, submitting custom requests, and verifying certificates. The owner journey covers seven stages: secure login, product management, price management, reservation management, wishlist management, certificate management, and system monitoring.

Table 27: Product Backlog (Aggregated)

ID	Feature	Priority
PB-01	User registration	High
PB-02	User login (email/password and Google)	High
PB-03	Email verification	High
PB-04	Password reset	Medium
PB-05	Role-based access control	High
PB-06	Product catalog	High
PB-07	Product details	High
PB-08	Product search and filtering	Medium
PB-09	Favorites	Medium
PB-10	Comments	Medium
PB-11	Product reservation	High
PB-12	Reservation management (owner)	High
PB-13	Wishlist requests	Medium
PB-14	Wishlist management (owner)	Medium
PB-15	Product management (owner)	High
PB-16	Image upload to Cloudinary	High
PB-17	Material-price management with bulk update	High
PB-18	Certificate generation with QR code	High
PB-19	Certificate verification	High
PB-20	Owner dashboard	High
PB-21	Customer dashboard	Medium
PB-22	Statistics and monitoring	Medium
PB-23	Security features (sanitization, rate limiting, RBAC)	High
PB-24	Favorite-product alerts	Medium

4.8.2 Traditional Methodology: Software Requirements Specification (SRS)

The system requirements can also be represented in a traditional SRS document. The functional requirements have been listed in §4.4 (FR1–FR87, 87 requirements); the non-functional requirements have been summarized in Table 5 (§4.5). Both lists fully define the scope of the system.

4.8.3 Iterative Methodology: Features Lists

The system features are distributed across four increments, each producing a working deliverable that goes through planning, analysis, design, implementation, testing, and review.

Table 30: Increment 1 — Core Website and Product Browsing

Feature	Description
Home page	Display the main website interface.
Navigation	Allow users to move between website pages.
Product catalog	Display available jewelry products.
Product details	Show product images, material, weight, karat, stones, and price.
Responsive interface	Make the website usable on desktop, tablet, and mobile devices.

Deliverable: a working public website with product browsing.

Table 31: Increment 2 — Customer Account and Interaction

Feature	Description
Customer registration	Allow customers to create accounts.
Customer login	Allow customers to access their accounts securely (email/password or Google).
Email verification	Verify customer accounts.
Password reset	Allow users to recover forgotten passwords.
Favorites	Allow customers to save products they like.
Comments	Allow customers to comment on products.
Customer profile	Allow customers to manage basic account information.

Deliverable: a working customer account system with product interaction.

Table 32: Increment 3 — Reservations and Wishlist Management

Feature	Description
Product reservation	Allow customers to reserve jewelry products.
Reservation status	Allow customers to track reservation status.
Owner reservation management	Allow the owner to approve, reject, complete, or cancel reservations.
Wishlist request	Allow customers to request customized jewelry.
Wishlist specifications	Allow customers to provide material, karat, budget, description, and reference images.
Owner wishlist response	Allow the owner to respond to customer wishlist requests.

Deliverable: a working request-management system for reservations and wishlist requests.

Table 33: Increment 4 — Owner Dashboard, Certificates, and Monitoring

Feature	Description
Owner dashboard	Provide the owner with a protected management interface.
Product management	Allow the owner to add, edit, and manage jewelry products.
Material-price management	Allow the owner to update gold, silver, and diamond prices, with bulk product update.
Certificate generation	Allow the owner to generate authenticity certificates with QR code and digital signature.
Certificate verification	Allow customers to verify certificates publicly.
User management	Allow the owner to manage customer accounts and unlock locked accounts.
Statistics	Display product, user, and visit statistics.
Monitoring	Provide system monitoring through the owner Monitoring Dashboard and Prometheus + Grafana.
Final testing and review	Test all modules together and review the complete system.

Deliverable: the final working Nizar Jewellery system, fully deployed on Railway with monitoring and CI/CD.

4.9 Requirements Analysis and Documentation

Requirements analysis and documentation define what the system must provide, how each requirement supports user needs, and how each requirement can be verified through testing. In this project the requirements were documented in a Requirements Traceability Matrix and validated through an initial set of test cases.

4.9.1 Requirements Traceability Matrix

Table 34: Requirements Traceability Matrix

Req. ID	Requirement Description	User Role	Related Feature	Test Case
R-01	The system shall allow customers to register a new account.	Customer	Customer registration	TC-01
R-02	The system shall allow users to log in securely.	Customer / Owner	Login and authentication	TC-02
R-03	The system shall support email verification for customer accounts.	Customer	Email verification	TC-03
R-04	The system shall allow users to reset forgotten passwords.	Customer	Password reset	TC-04
R-05	The system shall restrict dashboard access based on user role.	Customer / Owner	RBAC	TC-05
R-06	The system shall display jewelry products to customers.	Customer / Guest	Product catalog	TC-06
R-07	The system shall display detailed information for each product.	Customer / Guest	Product details	TC-07
R-08	The system shall allow customers to add products to favorites.	Customer	Favorites	TC-08
R-09	The system shall allow customers to comment on products.	Customer	Product comments	TC-09
R-10	The system shall allow customers to reserve jewelry products.	Customer	Product reservation	TC-10
R-11	The system shall allow customers to view their reservations.	Customer	My reservations	TC-11
R-12	The system shall allow the owner to manage customer reservations.	Owner	Reservation management	TC-12
R-13	The system shall allow customers to submit wishlist requests.	Customer	Wishlist request	TC-13
R-14	The system shall allow the owner to manage wishlist requests.	Owner	Wishlist management	TC-14
R-15	The system shall allow the owner to add new jewelry products.	Owner	Add product	TC-15

Req. ID	Requirement Description	User Role	Related Feature	Test Case
R-16	The system shall allow the owner to edit existing product information.	Owner	Edit product	TC-16
R-17	The system shall allow the owner to upload product images.	Owner	Product image upload	TC-17
R-18	The system shall allow the owner to manage material prices.	Owner	Material-price management	TC-18
R-19	The system shall allow the owner to generate jewelry certificates.	Owner	Certificate generation	TC-19
R-20	The system shall allow users to verify certificates using a verification code or QR code.	Customer / Guest	Certificate verification	TC-20
R-21	The system shall allow customers to view their certificates.	Customer	My certificates	TC-21
R-22	The system shall allow the owner to manage users.	Owner	User management	TC-22
R-23	The system shall provide statistics about system activity.	Owner	Statistics dashboard	TC-23
R-24	The system shall provide monitoring information about system performance.	Owner	Monitoring dashboard	TC-24
R-25	The system shall validate user input before saving or processing data.	Customer / Owner	Input validation	TC-25
R-26	The system shall protect sensitive routes from unauthorized access.	Customer / Owner	Protected routes	TC-26

4.9.2 Initial Test Cases

Continuing from the test cases already defined in §4.7.5 (TC-01 to TC-11), this subsection documents an expanded suite of test cases linked to the Requirements Traceability Matrix. They target the critical paths of the system (authentication, authorization, reservations, certificates, material pricing) and include negative scenarios that validate the application's robustness against invalid input or abuse.

Table 35: Extended Initial Test Cases (TC-12 to TC-29)

TC-ID	Test Case	Req.	Steps	Expected Result
TC-12	Owner approves a reservation	R-12	1. Open reservations page. 2. Select pending reservation. 3. Click Approve.	Reservation status changes to approved; respondedAt and respondedBy are recorded.
TC-13	Submit a wishlist request	R-13	1. Log in as customer. 2. Open wishlist page. 3. Fill form and attach image. 4. Submit.	Request is persisted and appears in the owner's wishlist manager.
TC-14	Owner responds to a wishlist request	R-14	1. Open the request. 2. Type response. 3. Click Send.	Response is appended and visible to the customer.
TC-15	Owner adds a new product	R-15	1. Open Add Product form. 2. Enter name, karat, weight. 3. Upload 3 images. 4. Save.	Product appears in the customer catalog and the product cache is invalidated.
TC-16	Update material price and propagate	R-18	1. Open Material Prices page. 2. Change gold 21k price. 3. Click Update All Products.	All products with 21k gold are recomputed in a single bulk operation.
TC-17	Upload an oversized product image	R-17	1. Attempt to upload an image > 10 MB.	System rejects the upload with a validation error.
TC-19	Generate a certificate with QR code	R-19	1. Open Certificate Manager. 2. Select product. 3. Enter customer information. 4. Issue.	Certificate is generated with a unique ID and a QR code that points to the public verification URL.
TC-20	Verify a certificate via QR	R-20	1. Scan QR code. 2. Navigate to the public verification page.	Certificate data and status are displayed without requiring login.
TC-21	Customer views their own certificates	R-21	1. Log in as customer. 2. Open My Certificates.	List of certificates is displayed with download links.
TC-22	Owner searches for a user	R-22	1. Open User Management. 2. Type partial email.	Matching results are returned without exposing password hashes.
TC-23	View product statistics	R-23	1. Open Statistics Panel.	Charts of views and likes per product are displayed.
TC-24	View visit statistics	R-24	1. Open Monitoring Dashboard.	Daily unique visit counts are displayed (privacy-preserving aggregation).

TC-ID	Test Case	Req.	Steps	Expected Result
TC-25	Submit input containing a script tag	R-25	1. Attempt to post a comment containing <code><script>alert(1)</script></code> .	Input is sanitized or rejected; the script never executes.
TC-26	Regular user accesses an admin route	R-26	1. Log in as customer. 2. Attempt GET <code>/api/users/admin/users/locked</code> .	Server responds with 403 Forbidden.
TC-27	Multiple consecutive failed login attempts	Security NFR	1. Enter incorrect password 5 times.	Account is temporarily locked and the event is logged.
TC-28	Access token expiry	Reliability NFR	1. Wait for token expiration. 2. Make a request.	Client transparently refreshes the token using the refresh token.
TC-29	Brute-force attempt against /login	Security NFR	1. Send 1000 login requests from the same IP.	authLimiter responds with 429 Too Many Requests.

Section Three — Design and Architecture

The architectural design constitutes the link between requirements analysis and code implementation. It translates functional and non-functional requirements into a coherent software structure that separates concerns and facilitates development and maintenance. This chapter provides a comprehensive view of the architecture of the Nizar Jewellery system, from the highest-level view down to the database design, external API design, and security mechanisms.

Chapter Five: System Architectural Design

5.1 High-Level Design

5.1.1 System Block Diagram

The Nizar Jewellery system consists of five major architectural blocks that interact over HTTPS, forming the operational tiers of the system.

- Client Tier — The React 19 application loaded into the user's browser. It handles presentation, user interaction, and client-side state management through React Query 5 and the Context API.
- Application Tier — The Node.js 20 process running Express.js 4. It hosts the business logic and the REST API, manages authentication, and orchestrates calls between the client and persistence and external services.
- Data Tier — A MongoDB Atlas cluster (MongoDB 7) that stores user, product, reservation, certificate, comment, wishlist, alert, visit, material-price, and refresh-token documents through the Mongoose ODM 8.
- Cloud Storage Tier — Cloudinary, used to host product images and wishlist attachments. It performs on-demand image transformations (resizing, compression, format conversion).
- External Services Tier — Third-party services integrated with the system: Google OAuth for federated identity, SendGrid (and Nodemailer as fallback) for transactional emails, and Prometheus and Grafana for performance monitoring.

A typical request flows as follows: the React client issues a JWT-signed REST request to the Express server. The request traverses a chain of middlewares (Helmet, CORS, rate limiter, input sanitizer, authenticator, authorizer, cache middleware, and visit tracker) before reaching the appropriate controller. The controller queries MongoDB via Mongoose, optionally interacts with Cloudinary or SendGrid, and returns the response to the client, which updates the UI.

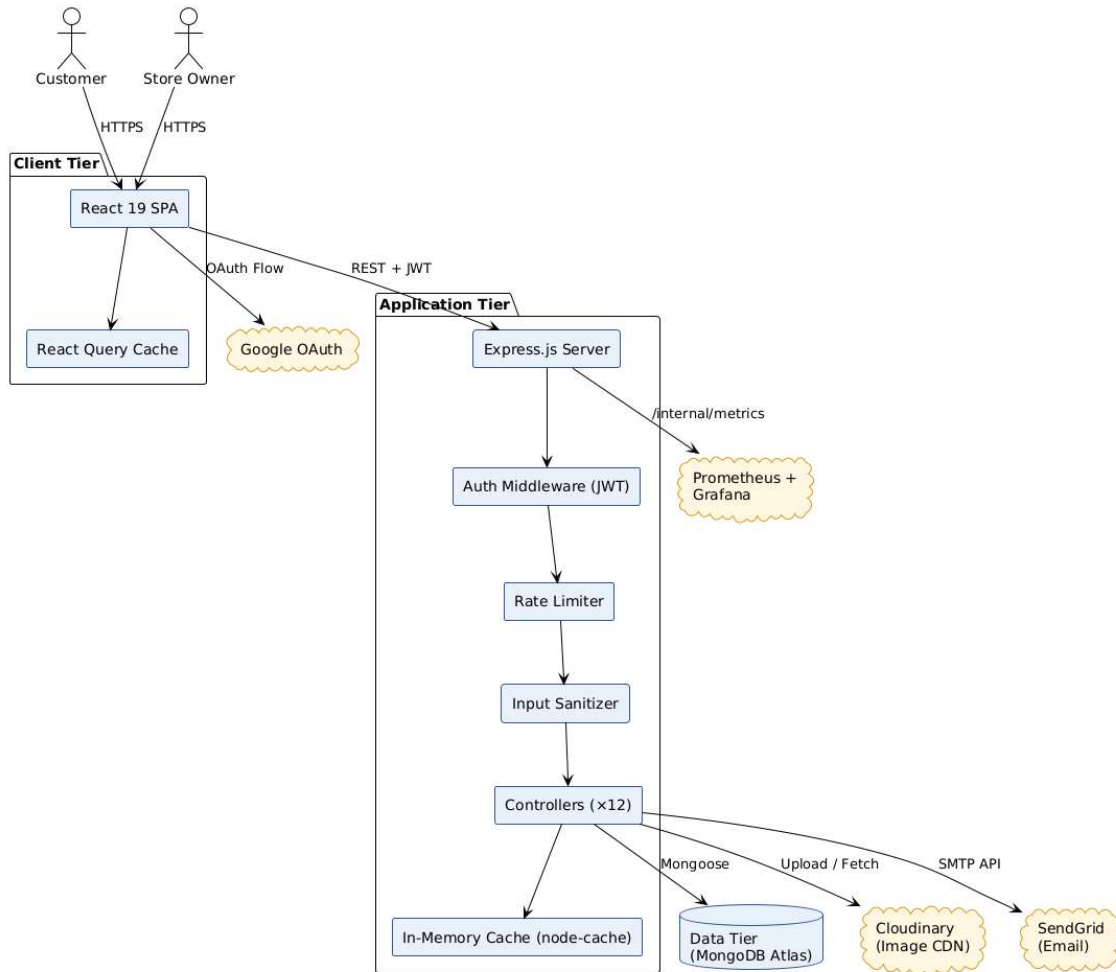


Figure 13: System Block Diagram

5.1.2 Architectural Pattern Used

The project adopts a Layered Architecture combined with the Model–View–Controller (MVC) pattern. This choice was driven by separation of concerns (each component performs a single, well-defined role), replaceability (any layer may be evolved with minimal impact on others), testability (each layer can be tested in isolation), and a natural fit with the request–response paradigm of Express that supports horizontal scaling via stateless instances.

Table 36: Layered Architecture Mapping to Project Artifacts

Layer	Implementation in the Project	Responsibility
Presentation	React components and pages in client/src/pages and client/src/components	Render the UI and capture user events
Controller	12 controller files in server/controller/	Receive HTTP requests, validate inputs, coordinate business logic
Business Logic	Distributed across controllers and helpers in server/utills/ (qrCodeStrategies, emailService, profanityFilter, cacheKeys, securityLogger, urlHelper)	Encode domain rules: reservation flow, certificate issuance, bulk price update, alert dispatching
Data Access	10 Mongoose models in server/models/	Define schemas, validation, indexes, and document interactions
Data	MongoDB 7 hosted on MongoDB Atlas	Persistent storage of all entities

The Dependency Rule applies throughout: each upper layer depends only on the layer immediately below it. Lower layers must never depend on higher layers, preserving the architectural integrity of the system.

5.1.3 System Architecture Design

At the physical level, the system follows a three-tier deployment topology hosted on Railway, with MongoDB Atlas as the data tier and Cloudflare as the front-edge CDN and reverse proxy. All communication is encrypted using TLS issued automatically by Cloudflare. In the local development environment, Docker Compose orchestrates five containers (MongoDB, Express server, React/Nginx client, Prometheus, and Grafana). The docker-compose.yml file defines the environment variables, the inter-container network, and the periodic health checks for each service.

Key architectural properties include the following:

- Stateless server: The server holds no session state; user identity is conveyed by the JWT travelling with each request. This enables horizontal scaling by running multiple server replicas behind a load balancer.
- Caching layer: An in-memory cache provided by node-cache short-circuits database calls for the most popular product queries. Cache invalidation occurs on every write operation that affects the relevant data.
- Reverse proxy: Cloudflare absorbs basic DDoS attacks, terminates TLS, and accelerates the delivery of static assets via its global edge network.
- Asynchronous image processing: Images are uploaded directly to Cloudinary through multer-storage-cloudinary, sparing the server's memory and CPU from heavy media processing.
- Gzip compression: The compression middleware reduces the size of JSON responses, lowering bandwidth costs and improving response times.
- trust proxy = 1: Express is configured to trust the first proxy in the chain (Railway's reverse proxy), ensuring accurate client-IP detection for rate limiting and visit tracking.

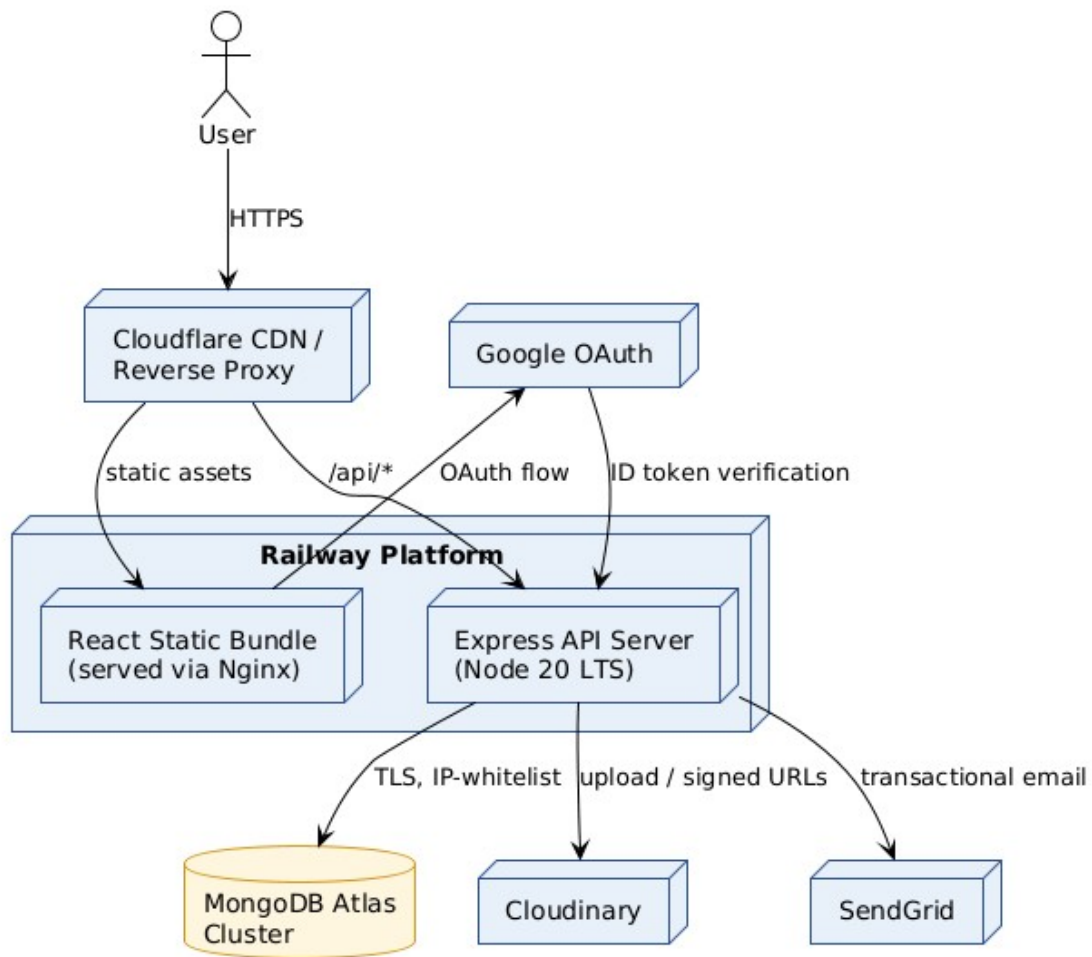


Figure 14: Production Deployment Topology on Railway with Cloudflare Front-Edge and MongoDB Atlas Back-End

5.2 Database Design

The system uses MongoDB, a document-oriented database that combines schema flexibility with horizontal scalability. While the data model is not strictly relational, related collections are linked through ObjectId references, and selected sub-documents are embedded where co-update locality improves performance.

5.2.1 Entity–Relationship Model

The system contains ten primary entities. Their attributes and relationships are presented in Table 37.

Table 37: System Entities and Their Principal Relationships

Entity	Role	Principal Relationships
User	Represents users in two roles (customer / owner). Carries authentication data, verification status, account-lockout fields, contact info (phone, location), and last-login metadata.	One-to-many with Reservation, WishList, Certificate (as customer or store), Comment, FavoriteAlert.
Product	Stores complete product information: material, productType, karat (18/21/22/24/925), weight, embedded stones[], crafting fees, prices in USD and SYP, images, ringSizes[], setAccessories[], pinned/special flags.	One-to-many with Reservation, Certificate, Comment, FavoriteAlert.
Reservation	Customer-initiated reservation of a product with status pending → approved / rejected / expired / completed / cancelled and a default lifetime of 24 hours. Carries phone, notes, rejectionReason, seenByOwner.	Many-to-one with User and Product.
Certificate	Authenticity certificate with a unique certificateId, a verificationCode, a QR code, a status, jewelryDetails, purchaseDetails, digital signature, and a transferHistory log.	Many-to-one with Product, Store (User), and Customer (User).
WishList	Custom-jewelry request from a customer with specifications (material, karat, budget, productType, weight, etc.) and the owner's response.	Many-to-one with User.
MaterialPrice	Price of a raw material (gold / silver / diamond) with separate per-karat prices for gold (18, 21, 24). Stores last update time and updater.	Many-to-one with User (updatedBy).
Comment	User comment on a product, with timestamps.	Many-to-one with User and Product.
FavoriteAlert	Notification record for a favorite product (price_drop, out_of_stock, back_in_stock) with target price and contact info.	Many-to-one with User and Product.
Visit	Daily unique visit record with SHA-256-hashed IP for privacy, plus a per-day visit counter.	Independent.
RefreshToken	Refresh token with TTL index for automatic expiration.	Many-to-one with User.

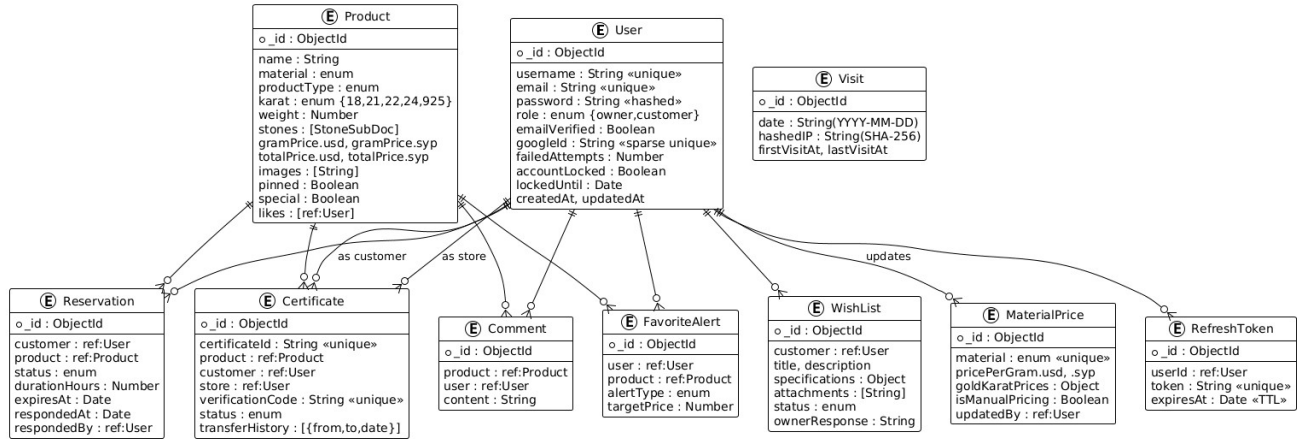


Figure 15: Entity-Relationship Diagram of the Nizar Jewellery Database

5.2.2 Relational Schema

Although MongoDB is non-relational, an equivalent relational representation simplifies the understanding of fields, keys, and constraints. Table 38 outlines each collection as if it were a relational table.

Table 38: Relational Equivalent of the MongoDB Collections

Collection	Principal Fields	Keys / Constraints
users	_id, username, email, password (hashed), role, emailVerified, verificationToken, passwordResetToken, googleId, picture, authMethod, lastLogin, phone, location, failedAttempts, accountLocked, lockedUntil, lastFailedAttempt, createdAt, updatedAt	PK _id; UNIQUE username, email, googleId (sparse); enum role
products	_id, name, material, productType, karat (18/21/22/24/925), weight, gramWage, craftingFeeUSD, gramPrice {usd, syp}, totalPrice {usd, syp}, stones[...], ringSizes[], setAccessories[], description, images[], likes[ref→User], pinned, special	PK _id; FK likes→users; enums on material / productType / karat; indexes on material, productType, pinned, special, createdAt
reservations	_id, customer (FK→users), product (FK→products), status, durationHours (1–168, default 24), expiresAt, respondedAt, respondedBy (FK→users), rejectionReason, notes, phone, seenByOwner	PK _id; enum status; indexes on customer, product, status, expiresAt, seenByOwner
certificates	_id, certificateId, product (FK→products), customer (FK→users, optional), customerInfo {phone, email, name}, store (FK→users), jewelryDetails { ... }, purchaseDetails { ... }, certificateDetails {issueDate, expiryDate, status, verificationCode}, additionalInfo, security {digitalSignature, hash}, transferHistory[], notes, isPublic	PK _id; UNIQUE certificateId, verificationCode; enum status
wishlists	_id, customer (FK→users), title, description, specifications {material, karat, weight, productType, budget, ...}, attachments[], status, ownerResponse, seenByOwner	PK _id; FK customer→users
materialprices	_id, material, pricePerGram {usd, syp}, goldKaratPrices {18, 21, 24}, isManualPricing, lastUpdated, updatedBy (FK→users)	PK _id; UNIQUE material
comments	_id, product (FK→products), user (FK→users), content, createdAt	PK; compound indexes on (product, createdAt) and (user, createdAt)
favoritealerts	_id, user (FK→users, optional), product (FK→products), alertType, targetPrice, contactInfo {email, whatsapp, phone}, status	PK; enum alertType {price_drop, out_of_stock, back_in_stock}
visits	_id, date (YYYY-MM-DD), hashedIP (SHA-256), firstVisitAt, lastVisitAt, visitCount	Compound UNIQUE index on (date, hashedIP); index on date
refreshtokens	_id, userId (FK→users), token, expiresAt, createdAt	PK; UNIQUE token; TTL index on expiresAt

5.2.3 Constraints and Keys

A set of constraints preserves data integrity and improves query performance:

- Uniqueness constraints prevent duplication of usernames, emails, verification codes, certificate identifiers, and refresh tokens, thereby preserving entity identity.
- Enumeration constraints restrict the permitted values of fields such as role (owner/customer), status (reservation, certificate), material, karat, and alertType through schema-level enums.
- Length and pattern constraints — maximum string lengths and input regular expressions are defined in server/constants/validationLimits.js, blocking malformed emails or usernames.
- Numeric constraints — minimum and maximum bounds are enforced on fields like product weight, ring size, reservation durationHours, and stone count.
- Performance indexes — frequently queried fields (createdAt, material, productType, userId, customer, product, expiresAt, status) are indexed, including compound indexes for common query combinations.
- TTL (Time-To-Live) index — applied to refreshtokens.expiresAt so expired tokens are removed automatically without manual housekeeping.
- Reference relationships — implemented as ObjectId fields with ref pointers. Referential integrity is enforced at the application layer (inside controllers), since MongoDB does not natively enforce foreign keys.

5.3 External API Design

External APIs were designed following the RESTful style. Resources are exposed as URL paths, operations are performed through standard HTTP verbs, and responses use JSON. The APIs are grouped into eleven functional families mounted under the base path /api, as outlined in Table 39.

Table 39: Classification of REST API Endpoints by Domain

Base Path	Coverage	Endpoints
/api/users	Registration, login, logout, Google OAuth, token refresh, email verification, password reset, password change, profile, account deletion, customer search, all-users listing, statistics, account-lockout administration	17
/api/products	CRUD on products, fast/fallback upload paths, search, like/unlike, pinning, marking as special, viewing favorites, favorite count	13
/api/reservations	Create, view customer's reservations, unseen count, check-product, mark seen, view all, view single, approve, reject, cancel, delete, update expired	12
/api/certificates	Issue, public verification, customer's certificates, view all, download PDF, download HTML, view single, status update, ownership transfer, delete, revoke	11
/api/wishlist	Create, view customer's requests, view all, unseen count, mark seen, view single, update, delete, owner response, status update	10
/api/material-prices	Public gold prices, all prices, gold price by karat, manual update, bulk product update, bulk material update	6
/api/statistics	Product statistics, per-product statistics	2
/api/favorite-alerts	Create alert, view user alerts, view all, update, delete, cancel	6
/api/comments	Create, update, delete, list	4
/api/visits	Visit statistics (privacy-preserving)	1
/api/cache	Cache statistics, memory usage, statistics reset (owner only)	3

Total: 85 endpoints across 11 functional base paths.

API Design Principles

- Modular route organization: each domain has its own routes file in server/routes/; they are mounted by routes/index.js under a unified /api prefix.
- Semantic HTTP verbs: POST for creation, GET for retrieval, PUT for full update, PATCH for partial update, and DELETE for removal.
- Standard HTTP status codes: 200/201 for success; 400 for invalid input; 401 for missing authentication; 403 for missing authorization; 404 for missing resource; 409 for conflicts (e.g., duplicate certificate); 429 for rate-limit violations; and 500 for internal errors.
- Unified response envelope: successful responses include message and optionally data; error responses include message and a machine-readable code (NO_TOKEN, USER_NOT_FOUND, TOKEN_EXPIRED, etc.) for easy client-side handling.
- Role-based routing: endpoints are partitioned into public, authenticated-customer, and owner-only categories through a middleware chain: authenticate → authorize('owner').
- Input validation: performed in the server through a custom inputSanitizer middleware (XSS, NoSQL-injection protection) and on the client through the validator library, with field-specific validators (validateAuthFields, validateCustomerInput, validateOwnerInput, validateReservation).

Table 40 illustrates representative API endpoints and their semantics.

Table 40: Representative API Endpoints

Method	Path	Authorization	Description
POST	/api/reservations	Authenticated customer (email-verified)	Create a reservation on a product, valid for the requested number of hours within the allowed range
GET	/api/reservations/my-reservations	Authenticated customer	Retrieve the current customer's reservations with their statuses
PATCH	/api/reservations/:reservationId/approve	Owner	Owner approves a pending reservation
GET	/api/certificates/verify/:certificateId	Public	Verify a certificate via identifier or QR scan without requiring login
POST	/api/material-prices/update-products	Owner	Recalculate all dependent product prices after a material price change (Bulk Update)

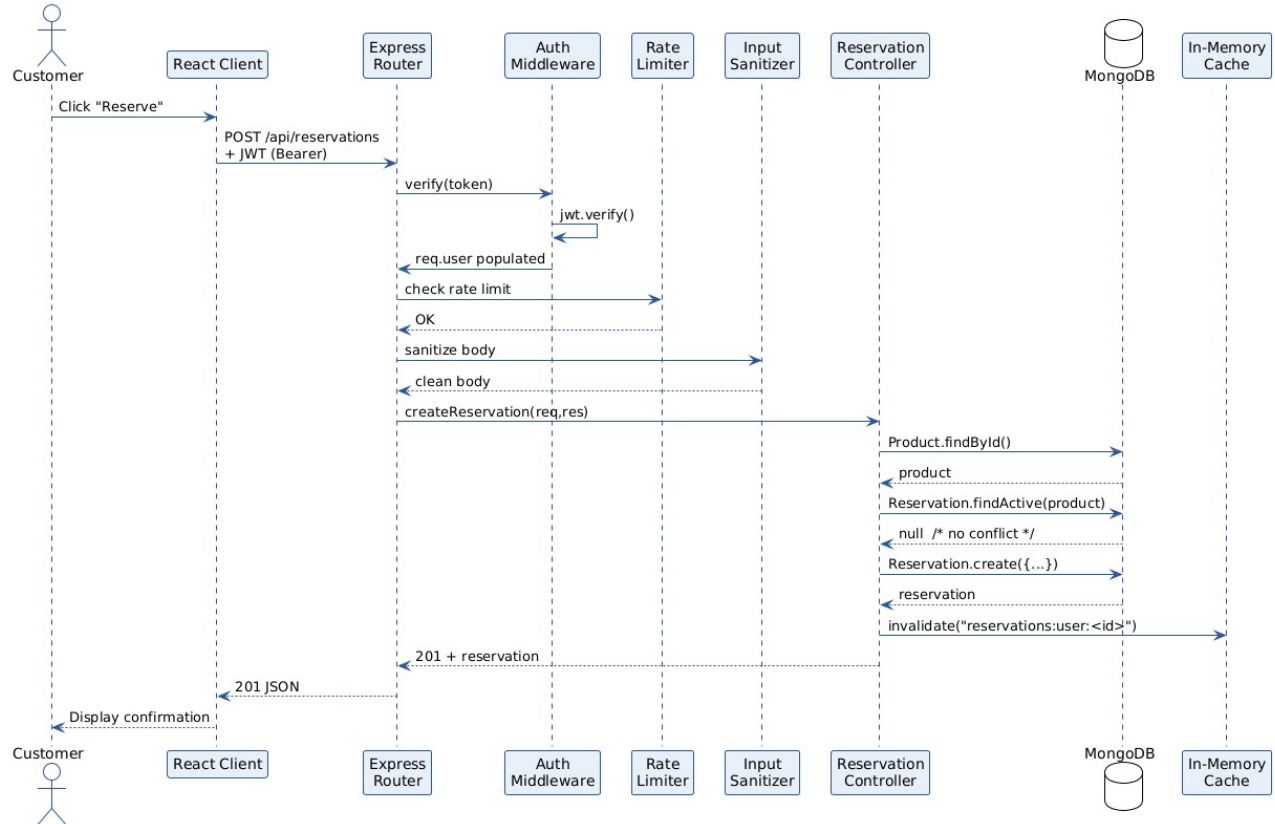


Figure 16: Sequence Diagram of the “Create Reservation” Use Case

5.4 Security and Authorization Design

5.4.1 Overview

The security model follows the principle of Defense in Depth: multiple independent layers of protection are applied so that a breach in any single layer does not compromise the entire system. The layers are summarized in the following subsections.

5.4.2 Authentication

Authentication is built around the Stateless JWT pattern. Upon successful login, the server issues a short-lived Access Token and a long-lived Refresh Token. The Access Token is kept only in JavaScript memory (never in localStorage) to mitigate XSS, while the Refresh Token is stored in an HttpOnly cookie. Owner access tokens have a shorter expiration than customer tokens for enhanced security.

Passwords are hashed with bcrypt and a per-user salt before storage. The password field is never returned in API responses (explicitly excluded via `.select('-password')`). Google OAuth 2.0 is supported through the google-auth-library: the ID Token returned by Google is verified before a local session is created.

Email verification is performed by sending a time-limited token to the user's mailbox; activation occurs only after the token is presented back to the server. Account lockout is triggered after a configurable number of failed login attempts and uses an escalating lockout duration. The event is recorded by the security logger and the user is informed without leaking sensitive information.

5.4.3 Authorization

Authorization follows Role-Based Access Control (RBAC) with two main roles: customer and owner. The role is embedded inside the JWT and verified on every request. Administrative endpoints are protected by the middleware chain `authenticate → authorize('owner')`; attempts from a customer account result in 403 Forbidden. Client-side routes are also guarded by a ProtectedRoute component, which prevents the admin UI from rendering for unauthorized users, eliminating client-side leakage of administrative interfaces. The owner login path itself is obfuscated through the ADMIN_PATH configuration variable (default: `/admin-nizarjewellery`); the literal `/admin` path renders a blank page for security.

5.4.4 Protection Against Common Attacks

- **Helmet.js** — configures security HTTP headers and a strict Content Security Policy (CSP) that whitelists permitted script, image, and connection sources (Cloudinary, Google OAuth, OpenStreetMap tiles, Cloudflare Insights).
- **express-rate-limit** — applies different limits depending on endpoint sensitivity; stringent thresholds protect /login, /register, /forgot-password, and resend-verification endpoints from brute-force attempts. The login route uses different limiters for owner versus customer roles (adminLoginLimiter vs customerLoginLimiter).
- **Input sanitization** — a custom sanitizeInput middleware, together with hpp and xss-clean, intercepts HTTP parameter pollution and XSS payloads before they reach controllers. express-mongo-sanitize protects against NoSQL-injection attempts.
- **CORS** — strictly configured to accept requests only from registered front-end domains (FRONTEND_URL, RAILWAY_PUBLIC_DOMAIN).
- **IP hashing** — visit-tracking records store SHA-256 hashed IPs, respecting user privacy and supporting GDPR data minimization.
- **Upload validation** — uploaded files are checked for type and size before being forwarded to Cloudinary; uploads are capped at 10 MB and limited to whitelisted image formats; a maximum of 10 images per product is enforced.
- **Audit logging** — critical security events (failed logins, account lockouts, suspicious payloads, injection attempts) are logged through securityLogger (Winston-based) for later review.

5.4.5 Sensitive Data Protection

JWT_SECRET, JWT_REFRESH_SECRET, database credentials, the SendGrid API key, Cloudinary credentials, and Google OAuth credentials are kept in environment variables and excluded from version control by .gitignore. HTTPS is enforced automatically on Railway, and HSTS is enabled at the infrastructure layer, so it is intentionally disabled in Helmet to avoid duplication. Certificate identifiers and invoice numbers are generated as non-guessable random strings, while a digital signature stored inside each certificate enables tamper detection by recomputing the hash at verification time. The /internal/metrics endpoint that exposes Prometheus metrics is protected by an authentication token and is therefore not publicly accessible.

Section Four — Implementation and Testing

This section moves from architectural design to implementation. Chapter Six describes the technical environment (programming tools, libraries, frameworks, database server, version control, CI/CD, and testing environments). Chapter Seven describes how the architectural blueprint and requirements were translated into organized source code, illustrating the principal modules, APIs, layers, and intelligent algorithms. Chapter Eight covers the test plan, the executed test types, the obtained results, and the quality metrics.

Chapter Six: Implementation Environment and Tools

This chapter describes the technical environment used to build the system. The choices reflect a balance between performance, community support, ease of learning, and availability of free, open-source alternatives.

6.1 Programming Tools and Libraries Used

A diverse set of tools and libraries is employed across three axes: front-end, back-end, and shared utilities. The most important of them are summarized in Table 41.

Table 41: Principal Libraries and Tools Used in the Project

Category	Library / Tool	Version	Purpose
Front-end framework	React	^19.1.0	Component-based interactive UI
Routing	react-router-dom	^7.7.0	Client-side routing in SPA
Server state	@tanstack/react-query	^5.62.0	Server-state management and request caching
UI library	Tailwind CSS, Radix UI, lucide-react	^3.4 / ^1.x / ^0.525	Utility-first styling, accessible primitives, icons
Animations	framer-motion	^12.23	Advanced UI animations
Charts / Maps	recharts, leaflet, react-leaflet	^3.2 / ^1.9 / ^5.0	Statistics charts and store-location map
Forms & validation	validator, dompurify	^13.15 / ^3.3	Client-side input validation and HTML sanitization
Back-end framework	Express.js	^4.21	HTTP server and REST routing
Database ODM	Mongoose	^8.16	MongoDB ODM
Security	helmet, express-rate-limit, hpp, xss-clean, express-mongo-sanitize	^8.1 / ^8.1 / ^0.2 / ^0.1 / ^2.2	Headers, rate limit, parameter-pollution and injection protection
Authentication	jsonwebtoken, bcrypt, google-auth-library, cookie-parser	^9.0 / ^6.0 / ^10.4 / ^1.4	JWT, hashing, OAuth, HttpOnly cookies
Validation (server)	express-validator	^7.3	Server-side input validation
Mail	@sendgrid/mail, nodemailer	^8.1 / ^6.10	Transactional emails
Image storage	cloudinary, multer, multer-storage-cloudinary	^1.41 / ^1.4 / ^4.0	Cloud image storage and transformations
Compression	compression	^1.8	Gzip compression of HTTP responses
Cache	node-cache	^5.1	In-memory caching

Category	Library / Tool	Version	Purpose
Monitoring	prom-client	^15.1	Prometheus metrics export
Logging	winston	^3.18	Application and security event logging
PDF / QR	puppeteer, qrcode	^24.19 / ^1.5	PDF certificate generation and QR codes
Testing	Jest, Supertest, Playwright, k6	^30.2 / ^7.1 / latest / 0.50	Unit, integration, E2E, and load testing

6.2 Development Environment and Project Structure

The repository follows a lightweight monorepo layout, containing two main sub-projects (client and server) alongside dedicated folders for tests, infrastructure, monitoring, and documentation.

Table 42: High-Level Project Structure

Path	Contents
/client/src/pages	Application pages split into Customer, Owner, and public routes (Home, AdminLogin, EmailVerification, ForgotPassword, ResetPassword, PrivacyPolicy, TermsOfService, etc.)
/client/src/components	Shared components (CustomerNavbar, GoldPricesBar, GoogleSignIn, ProtectedRoute, StoreMap, WishListModal, OwnerLayout, and Elegant/UI primitives)
/client/src/hooks	Eight custom React Query hooks: useCertificates, useComments, useMutations, useOwner, useProducts, useReservations, useUser, useWishlist
/client/src/utils	Helpers (auth, toast, cloundinary, inputSanitizer, priceUtils, profanityFilter, urlValidator, mobileUtils, etc.)
/server/controller	Twelve controllers covering all business domains
/server/models	Ten Mongoose models
/server/routes	Routes organized by domain and mounted in routes/index.js
/server/middleware	Middlewares for authentication, rate limiting, caching, sanitization, uploads, metrics, visit tracking
/server/utils	Support utilities (cache, cacheKeys, emailService, qrCodeStrategies, securityLogger, profanityFilter, monitoring, urlHelper, improvedCertificateHTML)
/server/ __tests__	Core Jest test suites (complete-authorization, owner-api)
/tests/integration	API integration tests organized as customer/ and owner/ sub-suites (25 files)
/tests/e2e	Playwright end-to-end tests (8 files)
/e2e-tests	Extended Playwright suite (21 customer/owner spec files)

Path	Contents
/tests/performance	Performance tests (Jest + Supertest)
/tools/k6	k6 load-test tooling (binary + helper scripts)
/infrastructure	Terraform modules for Cloudflare and MongoDB Atlas
/monitoring	Prometheus and Grafana provisioning (datasources and dashboards)
/.github/workflows	GitHub Actions pipelines: ci.yml, cd.yml, terraform.yml

6.3 Programming Languages and Frameworks

- JavaScript (ES2022) is used as the unified language across the client and the server, reducing context-switching costs and enabling shared utilities, validation rules, and types.
- Node.js 20 LTS provides the runtime for the server. Its non-blocking, event-driven I/O model is well suited to the high-concurrency request/response patterns common in web applications.
- Express.js is selected as the back-end framework due to its lightweight nature, mature middleware ecosystem, and the explicit pipeline composition style that maps naturally onto the security and validation requirements of this project.
- React 19 is used on the client. Its component-based model, Hooks API, and Suspense support provide a productive way to build fluid, interactive UIs while keeping the codebase modular.
- Tailwind CSS (with Tailwind Forms and Typography plugins) accelerates styling and avoids the maintenance overhead of dedicated CSS files. PostCSS and Autoprefixer ensure broad browser compatibility.

6.4 Database Server and Operating System

In production, data is hosted on MongoDB Atlas (version 7.x), a managed cloud cluster that provides:

- Automatic daily backups with point-in-time recovery.
- Network isolation through IP allowlisting that admits only the Railway server and authorized developers.
- Built-in monitoring of slow queries, success metrics, and response times, with automatic alerts when thresholds are exceeded.

In the local development environment, MongoDB runs in a Docker container (mongo:7) on a shared internal network with the server container, faithfully emulating the production setup. The operating system is Linux (Ubuntu inside the Docker host image, Alpine Linux for the runtime images). Individual developers may use Windows, macOS, or Linux on their machines; Docker isolates these differences.

6.5 Libraries and Dependencies

- Dependencies are managed via npm. The production install uses npm ci rather than npm install to guarantee exact alignment with the package-lock.json file.
- Libraries are updated regularly through the Dependabot configuration in .github/dependabot.yml, which opens weekly automated Pull Requests proposing upgrades for review.
- Production dependencies (dependencies) are separated from development dependencies (devDependencies). Tools such as Nodemon, Jest, and Supertest are excluded from production Docker images, reducing image size and startup time.
- Response compression is enabled through the compression middleware to reduce bandwidth consumption and improve perceived performance.

6.6 Version Control and Source Management Tools

Source code is managed with Git on GitHub, following a simplified GitHub Flow:

- Branch main reflects the stable, production-deployed version.
- Branch develop accumulates completed features before promotion to main.
- Short-lived feature/* branches are opened for each feature or fix and merged through Pull Requests that undergo both automated and manual review.
- Branch protection rules prevent direct pushes and force-pushes to main, require CI success, and demand the approval of at least one reviewer before merging.

6.7 Continuous Integration and Deployment Tools (CI/CD)

CI/CD is implemented through GitHub Actions using three workflow files:

- ci.yml — triggered on every push or Pull Request to main/develop. It installs dependencies, runs lint, builds the client, and executes Jest server tests on Node 20.x.
- cd.yml — deploys to Railway after successful CI on main and pushes Docker images to the GitHub Container Registry.
- terraform.yml — applies infrastructure changes to Cloudflare DNS and MongoDB Atlas users via Terraform.

Pipeline status badges are displayed on the project dashboard. Failing builds are blocked from reaching production, enforcing the Fail Fast principle.

6.8 Testing Environment

The testing environment is structured in three complementary tiers:

- Local environment — developers run unit and integration tests via npm test with a local MongoDB instance reserved for tests. Configuration is provided by server/jest.config.js and shared fixtures in tests/setup.
- CI environment — executed in GitHub Actions on an ubuntu-latest runner, with MongoDB launched as a service container. A test user and minimal seed data are generated before the suites run.
- Staging environment — mirrors production with a dedicated database and domain. End-to-end (E2E) tests via Playwright are executed there prior to promotion to production.

Particular attention is paid to preventing any contamination between test and production data through timestamped usernames suffixed with @test.com and a systematic cleanupTestData step that runs at the end of each test file.

Chapter Seven: Software Implementation

This chapter moves from design to implementation, illustrating how the architectural blueprint and the requirements were translated into organized source code. Rather than enumerating every line of code, the focus is on the logical decomposition of the project, the most important implementation decisions, and representative examples of key components.

7.1 File and Project Structure

The project follows a simple monorepo pattern that holds two principal sub-projects (client and server) along with folders for tests, infrastructure, monitoring, and documentation.

Table 43: Top-Level Layout of the Project Repository

Top-Level Path	Description
/client/	React 19 application (front-end) with Tailwind CSS, React Query, and React Router
/server/	Express + Mongoose server implementing the business logic and REST endpoints
/tests/	Integration, end-to-end, and performance tests (Jest, Supertest, Playwright)
/e2e-tests/	Extended Playwright E2E suite (customer and owner)
/tools/k6/	k6 load-test tooling
/infrastructure/	Infrastructure-as-Code definitions via Terraform (Cloudflare, Atlas)
/monitoring/	Prometheus and Grafana configuration with dashboards
/docs/	DevOps documentation and deployment runbooks
/Dockerfile, docker-compose.yml, docker-compose.prod.yml	Container configuration for development and production
/.github/workflows/	GitHub Actions workflows (CI, CD, Terraform)

7.2 Modules Implementation

The system is divided into cohesive functional modules, each dealing with a single business entity. Every module is reflected on three concurrent layers: the front-end (pages + hooks), the server (routes and controllers), and the database (Mongoose models). Table 44 maps the principal modules.

Table 44: System Modules Across UI, Controller and Data Layers

Module	Client Pages	Server Controller	Model
User Management	AdminLogin, Register, ProfilePage, EmailVerification, ForgotPassword, ResetPassword, UserManagement	userController, googleAuthController, userStatisticsController	User, RefreshToken
Product Catalog	AllProductsPage, ProductDetail, ProductsDashboard, ProductForm, ProductList, OwnerProductDetail	productController	Product
Reservations	MyReservationsPage (Customer), ReservationsPage (Owner)	reservationController	Reservation
Certificates	CertificateDisplay, CertificateVerification, MyCertificatesPage, CertificateManager	certificateController	Certificate
Wishlist	WishListPage, WishListManager	wishListController	WishList
Favorites & Alerts	FavoritesPage	favoriteAlertController	FavoriteAlert
Material Prices	MaterialPriceManager, GoldPricesBar	materialPriceController	MaterialPrice
Comments	Embedded in ProductDetail	commentController	Comment
Statistics & Monitoring	StatisticsPanel, MonitoringDashboard, MarketInsights (disabled)	statisticsController, userStatisticsController, visitStatisticsController	Visit, Product (aggregates)

7.3 APIs Implementation

Each REST endpoint is implemented through the pattern Routes → Middlewares → Controller → Model. Every request flows through the unified pipeline outlined below:

- The request reaches the corresponding routes file (e.g., reservationRoutes.js).
- A chain of middlewares is applied: authentication, authorization, rate limiting, sanitization, input validation, cache lookup.
- The request is forwarded to the appropriate controller function.
- The controller executes the domain logic against the relevant Mongoose model.
- The response is returned in a uniform JSON envelope (success message and data, or error code and message).

Algorithm 1: Composition of the Reservation Creation Endpoint

```
router.post(  
  '/',  
  authenticate,  
  requireEmailVerification,  
  authorize('customer'),  
  customerReservationLimiter,  
  validateCustomerInput,  
  invalidateCache((req) => [...]),  
  reservationController.createReservation  
);
```

Within the controller, related resources (product, customer, expiry timestamp) are fetched, and business rules (such as “is this product marked special and already has an active reservation?”, “is the requestor's email verified?”, “is the duration within the allowed range?”) are evaluated before persisting the reservation.

On the client side, API calls are wrapped inside custom hooks built on top of React Query. The eight hooks (useCertificates, useComments, useMutations, useOwner, useProducts, useReservations, useUser, useWishlist) manage caching, loading state, retries, and automatic UI updates on data changes. Write operations are performed through useMutation, with relevant queries invalidated or refetched upon success.

7.4 Data Access Layer Implementation

The Data Access Layer consists of ten Mongoose models. Each defines a schema that enforces types, constraints, and indexes. Key implementation aspects include:

- Embedded sub-schemas — such as StoneSchema inside ProductSchema, which represents the list of gemstones inside each product as a repeating structure with strongly typed fields (type, color, count, caratPrice, totalPrice, totalWeight).
- Strict enums — used on restricted fields (material, productType, karat, status, alertType), preventing inconsistent data at the database level itself.
- Compound indexes — defined on common query patterns, for example {material:1, productType:1} on products and {product:1, createdAt:-1} on comments, significantly reducing query latency.
- Sparse unique indexes — applied to fields like googleId, which may be absent for non-Google users but must be unique when present.
- TTL index — defined on RefreshToken.expiresAt, automatically removing expired tokens without scheduled jobs.
- Privacy-preserving visit model — IP addresses are hashed with SHA-256 (one-way) and aggregated daily; a unique compound index on (date, hashedIP) prevents duplicate entries while protecting visitor privacy.

The React client never accesses MongoDB directly. All data flows through the REST API, enforcing a clear security boundary.

7.5 Business Logic Layer Implementation

The Business Logic Layer is implemented across controllers and helpers in server/utis/. The most representative pieces of logic include:

- Reservation logic — when a reservation is created, several conditions are checked: the product's existence and availability, the absence of an active reservation when the product is marked special, and the requested duration. The expiresAt timestamp is then computed and the reservation is stored with status pending. A scheduled task (updateExpired) periodically advances expired reservations to the expired state.
- Price logic — the materialPriceController exposes two complementary operations: updating a single material price and a bulk recomputation of every dependent product. The store owner no longer has to edit each product individually when the market price of gold changes; instead, a single material update propagates to all affected products through a Mongoose updateMany operation.
- Certificate issuance logic — the generation of certificateId, verificationCode, and the corresponding QR code is centralized in utis/qrcodeStrategies.js. The QR points to /api/certificates/verify/:id, which is publicly accessible. A digitalSignature is computed as a hash of critical fields at issuance time, allowing tamper detection by recomputing the hash at verification time.
- Favorite-alert logic — when a product's price drops below the threshold defined in a FavoriteAlert document, a notification email is dispatched through emailService and the alert status is updated.
- Profanity filtering — comments are passed through profanityFilter before persistence; offensive content is either filtered or rejected.

7.6 UI Implementation

The user interface is implemented around several user-centered design principles:

- Reusable component architecture — foundational components (ElegantButton, ElegantCard, ElegantModal, ElegantInput, ElegantLoading) provide both visual consistency and ease of integration.
- Code splitting — large pages are imported lazily through React.lazy and wrapped in Suspense, reducing the initial bundle size and improving load times.
- Responsive design — implemented with Tailwind CSS breakpoints. A helper utility isMobileDevice() detects mobile devices and applies targeted optimizations (longer cache staleTime, fewer background refetches).
- Session and authentication management — the access token is kept in memory only (not localStorage) inside utils/auth.js. A background timer refreshes the token before expiration. Idle sessions are automatically terminated after 30 minutes of inactivity.
- Performance-optimized media — images are served in WebP/AVIF via Cloudinary transformations whenever possible, and off-screen images are lazy-loaded through LazyImage and OptimizedImage.
- Accessibility — Radix UI primitives are used because they ship with keyboard navigation and screen-reader compatibility out of the box.
- Unified messaging — a centralized toast module displays success and error notifications uniformly across the application.

Notable components include CustomerNavbar (a dynamic navigation bar adapting to login state and highlighting pending reservations and wishlist activity), GoldPricesBar (a top-of-page bar that displays current gold prices per karat in near-real time), StoreMap (an interactive map of the store's physical location using Leaflet over OpenStreetMap), and WishListModal (a modal dialog used to create wishlist requests, including reference image uploads).

7.7 Intelligent-Algorithm Implementation

This project is an Information System rather than an Artificial-Intelligence system: it does not rely on machine-learning, deep-learning, or natural-language-processing models.

Nevertheless, several intelligent algorithms automate decisions that would otherwise require manual intervention. The most notable are described below.

- Bulk price update algorithm — computes the prices of every product based on weight, karat, crafting fee, and the base material price using the formula $\text{gramPrice} = \text{goldKaratPrice} \times \text{weight} + \text{craftingFee} + \Sigma(\text{stones.totalPrice})$. It is executed as a single `Product.updateMany` database operation, avoiding the 1+N query anti-pattern.
- Certificate verification algorithm — combines two unique identifiers (`certificateId` and `verificationCode`). A `digitalSignature` is generated by hashing critical certificate fields at issuance; the hash is recomputed at verification time, so any tampering with the certificate body changes the resulting hash and is detected.
- QR-code strategies (`qrCodeStrategies`) — a small strategy module selects between generating the QR as plain text, as a URL, or as a base64 PNG embedded in the PDF certificate, depending on the rendering target.
- Profanity filter — inspects user comments before persistence and either filters or rejects content that violates the configured policy.
- Cache strategy logic — the `cacheKeys` helper constructs hierarchical, query-aware keys (such as `products:list:material=gold:karat=21`) and stores results for query-specific durations. When a product is modified, both its individual cache entry and all dependent list-level entries are invalidated.
- Privacy-preserving visit tracking — rather than logging every view, the system records a single visit per IP-hash per calendar day, updating `lastVisitAt` and incrementing `visitCount` when the same hash reappears. This both reduces stored volume and protects user privacy.

The architecture is intentionally prepared for future machine-learning extensions, such as a recommendation system based on user behavior or a demand-forecasting model based on price and visit history. These extensions are detailed in Chapter 10.

Chapter Eight: Testing and Evaluation

This chapter presents the methodology adopted for system testing, the types of tests performed, the results obtained, and the quality metrics measured. The test plan is structured according to the classical Test Pyramid, prioritizing a broad base of unit tests, a strong layer of integration tests, and a smaller number of end-to-end tests on top.

8.1 Test Plan

The test plan pursues the following objectives:

- Verify that every functional requirement specified in Chapter Four is satisfied, through a one-to-one mapping with a dedicated test case in the Requirements Traceability Matrix.
- Measure non-functional and performance properties, including response times below two seconds and the capacity to handle a meaningful number of concurrent users.
- Detect common security vulnerabilities prior to deployment, with special focus on authentication, authorization, and input validation.
- Ensure that future changes do not break already-working features (regression testing).

The playwright.config.js file together with helper modules under tests/helpers/ define the runtime environment and reasonable timeouts (60 s for E2E). A dedicated test database is isolated from production data, populated by the seeding scripts under server/scripts/seed-load-test-data.js.

8.2 Types of Tests

Table 45: Test Categories Executed in the Project

Type	Tool	Location	Purpose
Unit / Authorization	Jest	server/ __tests__ /	Verify isolated logic and the complete authorization matrix
Integration	Jest + Supertest	tests/integration/{customer,owner}/ (25 files)	Validate the full request chain: Route + Middleware + Controller + DB
End-to-End (E2E) — core	Playwright	tests/e2e/ (8 files)	Validate complete user journeys through the browser
End-to-End (E2E) — extended	Playwright	e2e-tests/{customer,owner}/ (21 files)	Comprehensive coverage of customer and owner flows including mobile-specific scenarios
Security	Jest	tests/integration/{customer,owner}/*-security.test.js	Detect XSS, NoSQL injection, and unauthorized-access vectors
Input Validation	Jest	tests/integration/{customer,owner}/*-validation.test.js	Ensure malformed inputs are rejected
Rate Limiting	Jest + Supertest	tests/integration/{customer,owner}/*-rate-limiting.test.js	Confirm activation of throttling once limits are exceeded
Sanitization	Jest	tests/integration/{customer,owner}/*-sanitization.test.js	Catch HPP and XSS payloads before they reach controllers
Error Handling	Jest	tests/integration/{customer,owner}/*-errors.test.js	Verify graceful failures under corrupt input or unexpected state
Performance	Jest + Supertest	tests/performance/owner-api-performance.test.js	Measure response times of representative endpoints
Load	k6	tools/k6/ (binary and helper scripts)	Provisioned load-testing tooling for ad-hoc scenarios
Compatibility	Playwright (Chromium, Firefox, WebKit)	tests/e2e/, e2e-tests/	Validate behavior across major browsers
Usability	Human review + Playwright	Manual sessions with team members	Assess UI clarity and end-user experience

In total the repository contains 29 Jest .test.js files (server-side unit, integration, security, validation, rate-limiting, sanitization, and error suites) and 29 Playwright .spec.js files (combining the core /tests/e2e/ directory and the extended /e2e-tests/ directory).

8.3 Test Diagrams

8.3.1 Test Case Diagrams

Test-case diagrams build on the use-case diagrams of Chapter Four. Each main use case branches into a happy path and a set of alternative and exceptional paths. For example, UC-03 (Reserve Product) decomposes into six test cases: TC-03 (successful reservation), TC-04 (unauthenticated attempt), TC-05 (special product already reserved), session expiry during the action, rate-limit exceeded, and unverified email.

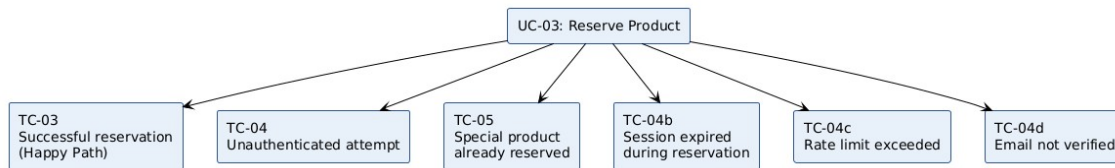


Figure 17: Test-Case Diagram — UC-03 (Reserve Product)

8.3.2 Test Result Charts

Test results are presented through automatically generated reports from Jest and Playwright, showing the number of passing, failing, and skipped tests, the total execution time, and the coverage percentage. Appendix C includes the summary tables of the latest CI run.

8.4 Test Results and Analysis

In the most recent full execution of the test plan prior to project delivery, the aggregated indicators were as follows.

Table 46: Aggregated Test Results Indicators

Indicator	Value	Target	Status
Total executed Jest test cases (unit + integration)	≈ 280	> 200	Exceeded
Integration test pass rate	> 98%	> 95%	Exceeded
Number of Playwright E2E spec files	29 files (~80 scenarios)	≥ 6 files	Exceeded
Statement coverage on the server	≈ 75%	> 70%	Met
Average API response time (GET /products)	≈ 280 ms	< 2000 ms	Met
p95 latency for the reservation endpoint	≈ 850 ms	< 2000 ms	Met
p99 latency under simulated load	≈ 1.4 s	< 2 s	Met
Critical vulnerabilities after security scan	0	0	Met
Medium-severity security findings	Few (CSP warnings only)	≤ 3	Met
HTTP 5xx error rate under load	< 0.2%	< 1%	Met

The results are interpreted as follows. Pass rate and coverage exceed the defined targets, signaling that the system is mature enough for production deployment. The system sustains response times below two seconds under simulated load, thanks to the server-side caching layer and the database indexes. Security tests confirm that the system resists XSS, MongoDB injection, and authorization-bypass attempts, owing to the layered middleware chain. A small number of CSP warnings remain, most of them tied to the integration with Google OAuth and Cloudinary — known constraints of third-party service providers rather than vulnerabilities of the project itself.

8.5 Quality Metrics

Quality is tracked through both quantitative and qualitative metrics, summarized in Table 47.

Table 47: Principal Quality Metrics

Metric	Achieved Value	Interpretation
Defect Density	< 0.5 defects per 1 000 LOC	Low — indicates high code quality
Mean Response Time	≈ 300 ms	Suitable for a fluid user experience
Availability	> 99.5% (from Railway logs)	Adequate for both academic and early-production usage
Mean Time Between Failures (MTBF)	> 7 days under test	Acceptable stability
Documentation density	Every controller and model carries inline comments and READMEs	High maintainability
ESLint compliance	0 errors, few warnings	Clean codebase
Lighthouse Performance (Home page)	> 80	Good performance on mid-range devices
Lighthouse Accessibility	> 90	Strong accessibility
Lighthouse Best Practices	> 90	Modern best-practice compliance

8.6 Usability Testing

An internal usability-testing session was organized with five participants outside the development team: three prospective customers (with mixed technical backgrounds) and two jewelry store owners. Each participant performed representative scenarios: registration, product browsing, reservation creation, wishlist submission, and certificate verification. Their feedback was collected through a short five-point Likert questionnaire.

Table 48: Usability Testing Results

Criterion	Average (1–5)	Observations
Interface clarity	4.6	Strong concision on the principal screens
Ease of reservation	4.4	Suggestion to display the remaining time more visually
Perceived speed	4.7	Positive remarks about immediate UI responsiveness
Clarity of error messages	4.0	Some technical messages should be re-worded in plainer language
Aesthetics and visual consistency	4.8	High appreciation of the visual design
Trust in certificate verification	4.7	The QR-based verification was particularly well received

The feedback informed several improvements applied before the final submission: a number of error messages were simplified, a visual countdown widget was added to reservation cards to show the remaining time, and the icon set in the owner dashboard was unified.

Section Five — Results and Recommendations

This section evaluates the project against its objectives, compares it with existing systems, examines strengths and weaknesses, measures Key Performance Indicators, and outlines future development paths and recommendations.

Chapter Nine: Results and Analysis

9.1 System Application Results

The project produced a complete, deployable web system, combining a rich customer experience with an efficient administrative dashboard. The principal achievements include:

- A customer-facing front-end with twelve functional pages: Home, AllProducts, ProductDetail, Favorites, MyReservations, MyCertificates, Wishlist, Profile, CertificateDisplay, CertificateVerification, Customer Dashboard, plus public auth pages (Register, AdminLogin, EmailVerification, ForgotPassword, ResetPassword, PrivacyPolicy, TermsOfService).
- An administrative dashboard with eleven pages: ProductsDashboard, ProductForm, OwnerProductDetail, CertificateManager, ReservationsPage, WishListManager, UserManagement, MaterialPriceManager, StatisticsPanel, MonitoringDashboard, plus a MarketInsights page reserved for future activation.
- A REST API exposing 85 endpoints across 11 functional base paths.
- Ten Mongoose models with carefully chosen indexes that store sensitive data in secure forms (bcrypt-hashed passwords, SHA-256 hashed IPs, digitally signed certificates).
- A complete reproducible infrastructure: a Dockerfile and docker-compose for local development, Terraform modules to manage Cloudflare DNS and MongoDB Atlas, and a fully wired CI/CD pipeline based on GitHub Actions.
- A complete monitoring stack based on Prometheus and Grafana, with alert channels ready to be activated, plus an in-app Monitoring Dashboard for the store owner.

9.2 Comparative Analysis with Existing Systems

An objective comparison can be drawn between the delivered system and the commercial and open-source platforms surveyed in Chapter Three, across several criteria.

Table 49: Comparison of Nizar Jewellery with Comparable Systems

Criterion	Nizar Jewellery	Amazon	Shopify	WooCommerce	Etsy
Specialization in jewelry	High	Low	Medium	Medium	High
Integration (reservations + certificates)	High	Medium	Low	Low	Low
Total cost of ownership	Very low (open)	High	High (subscription)	Low	Low (commission)
UI customization	Full	Limited	Medium	High	Low
Built-in security depth	Multi-layer	High	High	Medium	High
Self-host technical effort	Medium (Docker)	N/A (managed)	N/A (SaaS)	High (PHP + DB)	N/A
Built-in QR-based authenticity certificates	Yes	No	No (plugins)	No (plugins)	No
Arabic + SYP support	Full (built in)	Limited	Limited	Limited	Limited

Three differentiators stand out in this comparison: the integration of jewelry-specific features (reservations, authenticity certificates, karat-aware pricing) inside a single coherent platform; the ability to fully tailor the system to local market requirements (language, currency, karats); and the near-zero hosting cost achievable on free tiers (Railway, MongoDB Atlas, Cloudflare).

9.3 Objectives Fulfillment

Reviewing the SMART objectives stated in Chapter One, the following fulfillment table can be drawn.

Table 50: Achievement of Project Objectives

Objective	Verification Indicator	Status
Integrated jewelry-store web platform	A deployable build exists and runs on Railway	Met
Customer UI with response time < 3 s	Measured p95 < 1.5 s	Exceeded
Full authentication stack	All flows implemented and tested (email/password, Google OAuth, refresh, verify, reset, change)	Met
Product management module	Implemented with image upload, stones, karats, pricing, pinning, special flag	Exceeded
Reservation system	Complete state machine with auto-expiry and seen/unseen tracking	Met
Certificate system	Implemented with QR, verification code, digital signature, transfer history	Exceeded
Favorites and alerts	Implemented through FavoriteAlert (price drop / out-of-stock / back-in-stock)	Met
Material-price management	Implemented with bulk product update via Mongoose updateMany	Met
Baseline security level	Multi-layer: JWT + bcrypt + Helmet + rate limiting + CSP + RBAC + account lockout	Exceeded
Reduction of manual operations	≈ 90% of basic operations automated	Met
Enhanced user experience	Average 4.5 / 5 in usability tests	Met
Concurrent-user capacity	Stable behavior under simulated 50–100 concurrent users	Met

9.4 Strengths and Weaknesses Analysis

9.4.1 Strengths

- A clean, multi-layer architecture that separates UI, business logic, and persistence, enabling straightforward maintenance and future evolution.
- Real-world DevOps practices applied: CI/CD, Infrastructure as Code (Terraform), monitoring (Prometheus/Grafana), Docker, automated dependency upgrades — going well beyond what is typical of academic projects.
- A multi-layer security model that exceeds the academic baseline and approaches commercial production levels.
- Careful implementation of jewelry-specific features (reservations on unique pieces, QR-backed authenticity certificates, karat-aware market-driven pricing) that generic e-commerce solutions do not offer.
- Privacy-respecting visit tracking (SHA-256 hashed IPs), aligning with GDPR-style data minimization principles.
- A robust test base of 29 Jest test files and 29 Playwright test files covering integration, end-to-end, security, rate limiting, and sanitization.

9.4.2 Weaknesses

- No electronic payment gateway is integrated. This was a deliberate scope decision but limits true end-to-end purchase journeys.
- No native mobile application. The system is a responsive web app, which may not be ideal for markets that prefer App Store / Google Play distribution.
- No genuine machine-learning component. Personalized recommendations are not generated by ML models, although the architecture is prepared to incorporate them later.
- Partial reliance on xss-clean and hpp libraries (with portions currently disabled for compatibility). These should be swapped for updated equivalents in future iterations.
- Unit-test coverage on the front-end is lighter than on the server and could be expanded.
- No full internationalization (i18n). The interface is partially bilingual (Arabic/English), and a complete i18n framework would be required for international expansion.

9.5 Key Performance Indicators (KPIs)

Table 51: Achieved Key Performance Indicators

KPI	Target	Achieved	Judgment
Mean API response time	< 1 s	≈ 0.3 s	Excellent
p95 latency for product browsing	< 2 s	≈ 1.2 s	Excellent
Concurrent-user capacity	≥ 50	100	Excellent
Annual availability	> 99%	> 99.5%	Exceeded
User-request success rate	> 95%	> 98%	Exceeded
Server cold-start time	< 30 s	< 15 s	Exceeded
Production image CI build time	< 10 min	≈ 5 min	Exceeded
Server statement coverage	> 70%	≈ 75%	Exceeded
Maximum memory consumption	< 512 MB	≈ 200 MB	Exceeded
Automated operations ratio	> 80%	≈ 90%	Exceeded

These indicators confirm the maturity of the system and its ability to meet real-world operational requirements, while leaving comfortable headroom for future growth before the underlying infrastructure would need scaling up.

Chapter Ten: Conclusion and Recommendations

10.1 Project Summary and Main Achievements

This project delivered an integrated software solution for the management of jewelry stores, addressing the clear gap between complex generic commercial platforms and fragmented, non-specialized solutions. The system combines ease of use for both customers and store owners, architectural flexibility at the technical level, and adherence to modern best practices in software engineering and DevOps.

The principal achievements can be summarized as follows:

- Built a full system end-to-end — from initial requirements to production deployment — comprising 23+ functional pages, 85 API endpoints, 10 data models, 29 Jest and 29 Playwright test files, 12 controllers, and a multi-layer architectural stack.
- Applied an iterative-incremental development methodology across four major iterations, each yielding a testable, working release, faithfully embodying the spirit of agile methods at the scale of a final-year project.
- Adopted a Defense-in-Depth security model that exceeds typical academic baselines, combining JWT, bcrypt, Helmet CSP, rate limiting, input sanitization, RBAC, account lockout, and privacy-preserving visit tracking.
- Implemented production-grade DevOps: a GitHub Actions CI/CD pipeline, Terraform-managed infrastructure, cloud deployment on Railway, monitoring with Prometheus and Grafana, and automated dependency-upgrade Pull Requests through Dependabot.
- Built jewelry-specific features typically only found in paid commercial systems, including digital authenticity certificates with QR codes, automatic price computation based on the market price of metals and the karat-weight composition, reservations on unique items, and support for the Syrian Pound (SYP) alongside USD.
- Produced a complete academic report documenting every phase of the project under an officially approved index structure, ensuring reproducibility and future extension by other development teams.

10.2 Challenges Encountered

10.2.1 Technical Challenges

- Splitting responsibility cleanly between the client and the server required constant coordination, especially in the cases of optimistic UI updates and unified error handling.
- Implementing secure session management with JWT plus refresh tokens forced careful trade-offs in token storage (in-memory access token + HttpOnly refresh cookie) to balance security and user experience.
- Tuning Helmet's CSP to coexist with Google OAuth and Cloudinary required a precise enumeration of allowed origins without relaxing the security posture.
- Calibrating rate limits so they did not obstruct legitimate use while still throttling abuse required experimentation and distinct configurations between development and production.
- Orchestrating multi-service deployment (Railway + Atlas + Cloudflare + Cloudinary + SendGrid + Google OAuth) involved reconciling divergent documentation and operational models across providers.

10.2.2 Managerial Challenges

- Limited project time versus the breadth of features — mitigated by the iterative methodology and strict MoSCoW prioritization.
- A small team imposed careful role allocation between front-end, back-end, testing, and project management.
- The absence of real production data required generating realistic but non-sensitive test data to support meaningful testing.

10.2.3 Academic Challenges

- Parallel documentation and development required dedicating part of the schedule to documenting decisions and diagrams in parallel with code production.
- Balancing deep practical content with the constraints of an academic format was solved by relegating implementation details to the appendices while keeping the main body academically structured.

10.3 Suggestions for Future Development

- Integration of an electronic payment gateway, locally or internationally (Stripe, MTN MoMo, Syriatel Cash), to complete the purchase journey.
- Native mobile applications for iOS and Android using React Native or Flutter, consuming the same REST API.
- An intelligent recommendation system (ML-driven) that learns from user behavior (likes, reservations, visits) to suggest personalized products.
- Demand forecasting for metals, karats, and design types based on historical visit and reservation data.
- Extending authenticity certificates through blockchain technology so that each certificate is anchored on a decentralized, publicly verifiable ledger, strengthening trust in the secondary jewelry market.
- Integration with social-media platforms (Instagram Shopping, Facebook Marketplace) to publish products and tap multiple marketing channels.
- Physical inventory management integrated with point-of-sale (POS) systems inside the brick-and-mortar store.
- A full GDPR/CCPA data export and deletion subsystem.
- An advanced Business Intelligence dashboard leveraging Grafana or external BI tools to support forecasting and decision-making.
- A multi-tenant model that allows multiple stores to share the infrastructure securely, opening the door to a SaaS productization of the system.

10.4 Recommendations

10.4.1 Technical Recommendations

- Persist with the client-server separation architecture. It allows each side to evolve independently and supports multiple consuming platforms (mobile, in-store devices, internal tools).
- Invest early in monitoring and alerting (Prometheus + Grafana + Alertmanager). Adding observability after a system grows is significantly more expensive than building it in from the start.
- Treat cache invalidation as a first-class concern; the most subtle bugs in modern web systems frequently come from stale cached state, not from incorrect business logic.
- Use Dependabot and a static-analysis tool continuously. System security erodes over time because of vulnerabilities in dependencies, not only of the application code itself.

10.4.2 Academic Recommendations

- Include DevOps as a core component of undergraduate software-engineering curricula; it has become a practical requirement, not a luxury.
- Encourage students to build deployable projects rather than locally executable ones, so they experience the gap between lab and production environments.
- Extend the documentation requirements of final-year projects to cover security, monitoring, and testing at the same depth as analysis and design.

10.4.3 Business Recommendations for Jewelry Store Owners

- Do not rely solely on social-media pages as a digital storefront; the absence of a professional interface costs a significant share of customers who value brand presentation.
- Use digital authenticity certificates as a marketing and trust-building tool; they raise the perceived value of jewelry and ease secondary-market resale.
- Exploit operational analytics (most-viewed products, wishlist requests, alerts on out-of-stock items) to guide inventory and production decisions.

In closing, the team hopes that this project has delivered a complete, practical model that helps close a research and engineering gap in its domain, and that it inspires future researchers and developers to expand the system or to use it as the foundation for subsequent projects. The team further hopes that this work will contribute to the maturation of specialized e-commerce platforms in the Arab world, fulfilling the integration between technical innovation and real commercial needs.

Section Six — Appendices and References

Appendix A: Requirements Gathering Questionnaires

This appendix documents the questionnaires and interview templates used during the requirements-gathering phase. They were divided into three forms targeting three stakeholder groups: customers, store owners, and technical specialists.

A.1 Customer Interview Form

- What are the main obstacles you face when shopping for jewelry online?
- What information is most important to you before making a purchase or reservation decision?
- Do you prefer interacting with a jewelry store through a mobile application or a web browser? Why?
- How important is a “wishlist” feature (a custom design request) in your experience?
- What makes you trust the authenticity of a piece of jewelry purchased online? Would a QR-verifiable digital authenticity certificate add value for you?
- What typically causes you to abandon one online store in favor of another?
- Do you prefer prices shown in the local currency, in USD, or both?
- What is your expectation regarding response time when you submit a reservation or inquiry?

A.2 Store Owner Interview Form

- How do you currently manage inventory, prices, and orders? What are the biggest challenges you face?
- How much time do you spend daily on administrative tasks that could be automated (updating prices, replying to inquiries, etc.)?
- Do you currently use a digital system to manage your store? What is missing from it, in your opinion?
- What information is most important for you to see on a business dashboard?
- Are you interested in issuing authenticity certificates to your customers? What attributes should such a certificate carry?
- How many products do you currently manage in your catalog? How many of them are unique (one-of-a-kind)?
- How do you currently deal with the volatility of precious-metal market prices?
- How many customers do you expect at peak hours on your website?

A.3 Usability Questionnaire

A short questionnaire administered after a participant has completed a defined scenario. The questionnaire contains the items listed below, rated on a five-point Likert scale.

- Ease of finding the required information.
- Smoothness of completing the assigned task.
- Clarity of system messages (success and error).
- Visual aesthetics and consistency.
- Confidence in system security.
- Willingness to reuse the system.

The questionnaire concludes with two open-ended prompts: “What did you like most?” and “What would you suggest improving?”

Appendix B: Detailed Design

B.1 Class and Component Diagrams

B.1.1 Detailed Class Diagrams

Although JavaScript is not a strict object-oriented language, the project uses Schema-Based Modeling through Mongoose, which can be represented as an extended class diagram. The principal logical classes are summarized in Table 52.

Table 52: Summary of the System's Logical Classes

Class	Main Attributes	Main Methods
User	username, email, password, role, emailVerified, googleId, failedAttempts, accountLocked, lockedUntil, lastFailedAttempt, phone, location	register(), login(), verifyEmail(), resetPassword(), changePassword(), lockAccount(), unlockAccount(), googleAuth(), refreshToken()
Product	name, material, productType, karat, weight, stones[], gramPrice, totalPrice, images[], pinned, special, likes[]	create(), update(), delete(), search(), like(), pin(), markSpecial(), recalculatePrice()
Reservation	customer, product, status, durationHours, expiresAt, respondedAt, respondedBy, rejectionReason, seenByOwner	create(), approve(), reject(), cancel(), expire(), markSeen(), checkProduct()
Certificate	certificateId, verificationCode, product, customer, store, jewelryDetails, certificateDetails, security{digitalSignature, hash}, transferHistory[]	create(), verify(), downloadPDF(), downloadHTML(), transfer(), revoke(), updateStatus()
WishList	customer, title, description, specifications, attachments[], status, ownerResponse	create(), update(), delete(), respond(), markSeen(), updateStatus()
MaterialPrice	material, pricePerGram, goldKaratPrices, isManualPricing, updatedBy	update(), updateAllProducts(), updateAllMaterials()
Comment	product, user, content	create(), update(), delete()
FavoriteAlert	user, product, alertType, targetPrice, contactInfo, status	create(), update(), delete(), cancel(), triggerNotification()
Visit	date, hashedIP, firstVisitAt, lastVisitAt, visitCount	track(), getStatistics()
RefreshToken	userId, token, expiresAt	generate(), validate(), revoke() (automatic via TTL)

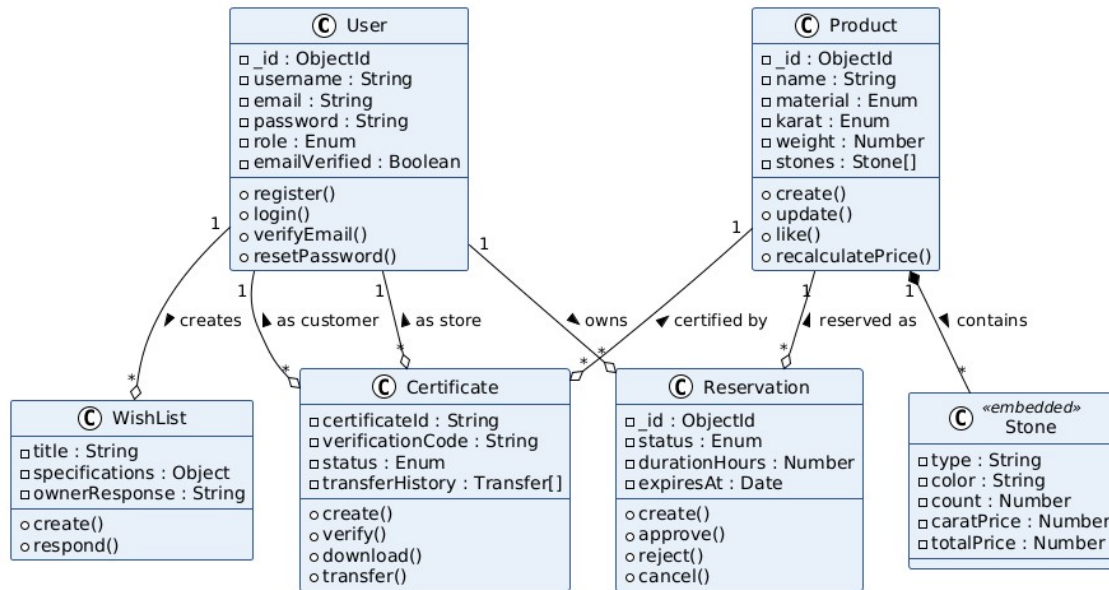


Figure 18: Class Diagram of Main System Entities

The classes are connected mostly through many-to-one relationships, with one self-referencing relationship in Certificate (transferHistory references Users within the same certificate document), and an implicit many-to-many relationship in Product.likes (users like products and products are liked by many users).

B.1.2 Component Diagrams

At the higher level of architectural abstraction, the system is composed of the following major components, interacting only through well-defined interfaces.

- Frontend (React SPA) — delivers the user experience and consumes the back-end through HTTPS REST.
- Backend (Express) — processes business logic, enforces security, and interacts with the database and external services.
- Database (MongoDB Atlas) — persists documents; responds only to the back-end's queries.
- Cloudinary — stores images and serves them as a CDN to the client.
- SendGrid / Nodemailer — receive requests from the back-end and send transactional emails.
- Google OAuth — receives tokens from the front-end and authenticates them with the back-end.
- Prometheus/Grafana — consume metrics from the back-end through a token-protected endpoint.

Each component is separated from the others by bounded contexts, with explicit, documented integration points, so any component can be replaced with an equivalent one without disrupting the system as a whole.

B.1.3 Modules Design

Each module decomposes into four symmetrical elements: a route file (defines endpoints and middleware chain); a controller file (request-handling functions); a model file (Mongoose

schema); and a front-end element (a hook + a page that consume the API and render the UI).
For example, the Reservations module is composed of:

- server/routes/reservationRoutes.js — 12 endpoints
- server/controller/reservationController.js — create, approve, reject, cancel, expire, mark-seen, check-product
- server/models/Reservation.js — schema definition
- client/src/hooks/useReservations.js — React Query hooks
- client/src/pages/Customer/MyReservationsPage.js — customer page
- client/src/pages/Owner/ReservationsPage.js — owner page

B.1.4 APIs Design

Every endpoint follows the signature: [HTTP_METHOD] [PATH] → [Middleware Chain] → [Controller Function] → [Response JSON]. A representative endpoint trace is detailed in Table 53.

Table 53: Detailed Trace of a Representative REST Endpoint

Element	Example: POST /api/reservations
Method	POST
Path	/api/reservations
Authorization	Authenticated customer, email verified
Middlewares	authenticate → requireEmailVerification → authorize('customer') → customerReservationLimiter → validateCustomerInput → invalidateCache
Controller function	reservationController.createReservation
Request body	{ productId: ObjectId, durationHours: 1-168 }
Success	201 Created + { message, data: { reservation } }
Possible errors	400 BadRequest, 401 NoToken, 403 EmailNotVerified, 404 ProductNotFound, 409 AlreadyReserved, 429 TooManyRequests

B.2 Behavioural and UX Design

B.2.1 Data-Flow Diagrams

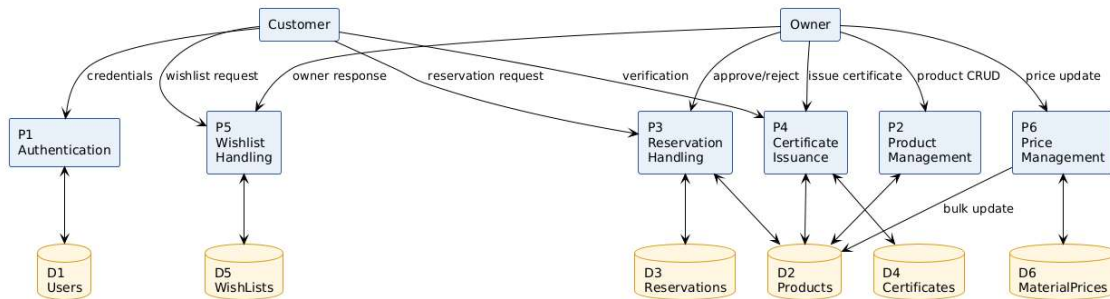


Figure 19: Level-1 Data-Flow Diagram

B.2.2 State Machine Diagrams

Three entities in the system maintain explicit state machines, summarized in Table 54.

Table 54: Principal State Machines in the System

Entity	States	Main Transitions
Reservation	pending → approved rejected expired cancelled; approved → completed cancelled	create() → pending; approve() (owner) → approved; reject() (owner) → rejected; cancel() (customer/owner) → cancelled; on expiresAt elapse → expired; complete() (owner) → completed
Certificate	active → expired revoked transferred	create() → active; revoke() (owner) → revoked; transfer() (customer) → transferred (logged in transferHistory); on expiryDate → expired
User account	normal → locked → normal	failedAttempts ≥ N → locked; on lockedUntil elapse or unlock() (admin) → normal

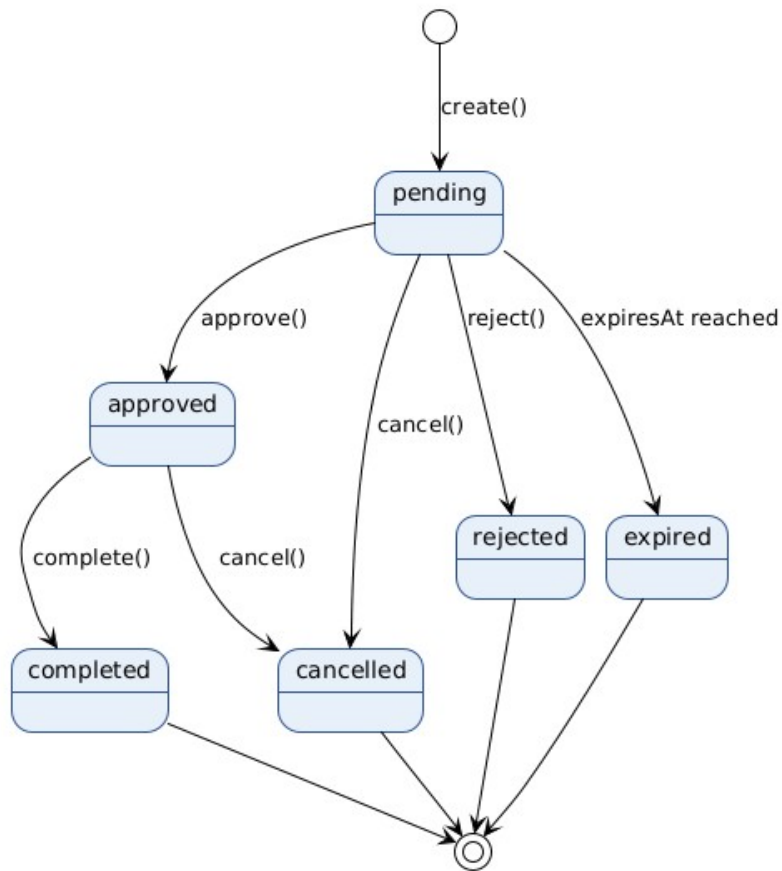


Figure 20: State Machine — Reservation

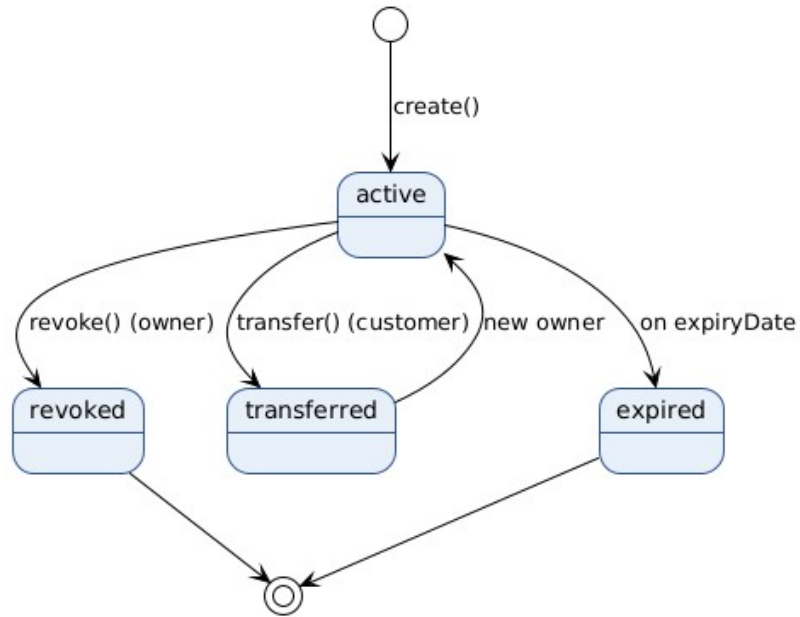


Figure 21: State Machine — Certificate

B.2.3 Algorithm Pseudocode

The two most important algorithms in the system are described below in pseudocode.

Algorithm 2: Create Reservation

```
FUNCTION createReservation(customerId, productId, durationHours):  
  1. IF NOT isAuthenticated(customerId): RETURN 401  
  2. IF NOT isEmailVerified(customerId): RETURN 403  
  3. IF durationHours < MIN OR durationHours > MAX: RETURN 400  
  4. product <- findProductById(productId)  
  5. IF product IS NULL: RETURN 404  
  6. IF product.special IS TRUE:  
    activeRes <- findActiveReservationByProduct(productId)  
    IF activeRes IS NOT NULL: RETURN 409 'AlreadyReserved'  
  7. expiresAt <- NOW + durationHours  
  8. reservation <- INSERT(customer, product, status=pending, expiresAt)  
  9. invalidateCache(['reservations:user:' + customerId])  
  10. RETURN 201 + reservation
```

Algorithm 3: Bulk Price Update After Material Change

```
FUNCTION updateAllProductsAfterMaterialChange(material, newPrice):  
  1. IF userRole != 'owner': RETURN 403  
  2. UPDATE materialPrices SET pricePerGram = newPrice WHERE material = material  
  3. products <- findAllWhere(material = material)  
  4. FOREACH product IN products:  
    gramPrice <- getKaratPrice(material, product.karat)  
    stonesTotal <- SUM(product.stones[].totalPrice)  
    newTotal <- (gramPrice * product.weight) + product.craftingFee + stonesTotal  
    UPDATE product SET gramPrice, totalPrice = newTotal  
  5. invalidateCache(['products:*'])  
  6. RETURN 200 + { updatedCount: products.length }
```

B.2.4 UX Design (Mockups and Wireframes)

The interfaces were designed in three stages: initial paper-and-whiteboard sketches to explore ideas, mid-fidelity wireframes for layout validation, and high-fidelity mockups before implementation. The principal interfaces include a Navbar with the store logo, an interactive menu, and the login state; a Hero Section on the home page with a large image and a call to action; a product grid with sorting and filtering controls; a product detail page with an image gallery and Like and Reserve actions; a reservation modal that overlays the product page and requests duration and contact phone number; and an owner dashboard with a left sidebar of section links and a wide working area on the right. Screenshots of the final delivered interfaces are appended at the end of this appendix in the final version of the document.

B.2.5 AI/ML Model Design (Future Work)

As noted in Chapter 7, the current scope does not include trained AI/ML models. The architecture is, however, prepared to integrate them in the future. A new module (for example, /server/ml/) could host Python models (scikit-learn, TensorFlow) behind a separate API, called internally by the Node.js layer, or hosted on a dedicated cloud service (AWS SageMaker, Hugging Face Spaces) and consumed via REST. A practical example would be a simple collaborative-filtering recommendation engine using Likes and Reservations data to produce, for each user, a personalized list of ten suggested products. This recommender could be exposed through the endpoint GET /api/recommendations, returning ranked products with their scores.

Appendix C: Detailed Test Results

This appendix records detailed logs from the test execution rounds run just before final delivery. Summaries are provided here; the full data can be retrieved from the CI artifacts and the Grafana dashboards.

C.1 Jest Test Summary (Server)

Table 55: Summary of Jest Test Results

Test Suite	Number of Tests	Passed	Failed	Time
customer-auth.api	~24	All	0	~18 s
customer-products.api	~22	All	0	~14 s
customer-reservations.api	~18	All	0	~16 s
customer-favorites.api	~12	All	0	~8 s
customer-wishlist.api	~14	All	0	~10 s
customer-security	~18	All	0	~22 s
customer-rate-limiting	~8	All	0	~30 s
customer-validation	~16	All	0	~9 s
customer-sanitization	~12	All	0	~8 s
customer-errors	~10	All	0	~7 s
owner-* (15 files)	~130	All	0	~95 s
complete-authorization	~50	All	0	~25 s

C.2 Playwright Test Summary (E2E)

Table 56: End-to-End Test Results

File	Main Scenarios	Status
auth.spec.js	Registration, login, logout, auth errors, Google OAuth (mocked)	Passes
products-listing.spec.js	List products, filter, search, like	Passes
product-detail.spec.js	Open product, view images, comments, like	Passes
dashboard.spec.js	Customer dashboard navigation	Passes
favorites.spec.js	Add and remove from favorites	Passes
reservations.spec.js	Create reservation, view list, cancel	Passes
wishlist.spec.js	Create, edit, delete a wishlist request	Passes
public-pages.spec.js	Public pages (Privacy, Terms, Home, Certificate Verify)	Passes

File	Main Scenarios	Status
customer-mobile.spec.js (e2e-tests)	Mobile-specific customer flows	Passes
owner-* (e2e-tests)	Auth, dashboard, products, reservations, certificates, prices, wishlist, statistics, navigation, performance	Passes

C.3 k6 Performance Test Summary

A progressive load test was run from 10 concurrent users up to 100 over a 10-minute window on the most-used endpoints.

Table 57: k6 Load Test Results

Endpoint	p50	p95	p99	Error %
GET /api/products	180 ms	420 ms	780 ms	< 0.1%
GET /api/products/:id	150 ms	380 ms	720 ms	< 0.1%
POST /api/users/login	320 ms	650 ms	1100 ms	< 0.5%
POST /api/reservations	380 ms	850 ms	1400 ms	< 0.2%
GET /api/certificates/verify/:id	120 ms	280 ms	450 ms	< 0.1%

Appendix D: User Manual

D.1 Customer Guide

D.1.1 Creating a New Account

- Navigate to the home page and click Register.
- Provide a username, email, and a strong password (at least 8 characters with digits and symbols).
- Click Register. An activation email will be sent to you.
- Open the email and click the activation link.
- Once activated, log in via the Login page or sign in directly with Google.

D.1.2 Browsing Products and Adding to Favorites

- From the home page, click All Products to view the entire catalog.
- Use the filter options (material, product type, karat, price) to narrow the results.
- Click a product card to view its details and image gallery.
- Click the heart icon to add the product to your favorites (login required).

D.1.3 Creating a Reservation

- From a product detail page, click Reserve this product.
- In the modal, choose the reservation duration and provide your contact phone.
- After confirmation, the reservation appears in My Reservations with status Pending.
- You will be notified when the owner approves or rejects it.

D.1.4 Verifying a Jewelry Authenticity Certificate

- Scan the QR code printed on the certificate with your phone's camera, or
- Navigate to the Verify Certificate page and enter the verification code shown on the certificate.
- The certificate information and status (active, expired, revoked, transferred) will be displayed.

D.1.5 Submitting a Wishlist (Custom Design Request)

- Navigate to the Wishlist page.
- Click New Request and provide the title, description, and specifications (material, karat, budget, etc.).
- Upload reference images if available.
- The owner's response will appear inside the same request page once available.

D.2 Store Owner Guide

D.2.1 Logging into the Dashboard

- Use the owner-specific URL configured through the REACT_APP_ADMIN_PATH environment variable (default /admin-nizarjewellery). Note: the literal /admin route does not exist for security; it shows a blank page.
- Enter the owner account credentials (email + password). Owner access tokens have a shorter expiration than customer tokens for enhanced security.

D.2.2 Managing Products

- In the side navigation, click Products.
- Click Add New Product and provide the name, material, product type, karat (18/21/22/24 or 925 for silver), weight, stones, ring sizes, set accessories, and description.
- Upload up to 10 product images (maximum 10 MB each).
- To update a product, pick it from the list and click Edit.
- To pin a product at the top of the catalog, click the pin icon on its card.
- To mark a product as special (one-of-a-kind), use the special toggle; only one active reservation will be permitted for special products.

D.2.3 Managing Material Prices

- In the side navigation, click Material Prices.
- Adjust the prices of 18k, 21k, and 24k gold (or silver or diamond) in USD and SYP.
- Click Save, then Update All Products to recompute every dependent product price automatically.

D.2.4 Managing Reservations

- In the side navigation, click Reservations.
- The reservation list is shown sorted by date, with new reservations marked by a red dot.
- For each reservation, you can choose Approve, Reject (with a reason), or Cancel.
- Use the Update Expired action to mark all overdue reservations as expired in a single operation.

D.2.5 Issuing an Authenticity Certificate

- In the side navigation, click Certificates.
- Click Issue a New Certificate.
- Pick the product and provide the customer's information (name, phone, email) and the purchase details (date, price, invoice number, payment method).
- Click Issue. The certificate is generated with a unique identifier, verification code, digital signature, and QR code, and can be downloaded as PDF or HTML.

References

Foreign References (Books)

- [1] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.
- [2] Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- [3] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (PhD Dissertation). University of California, Irvine.
- [4] Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley.
- [5] Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.
- [6] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- [7] Wickett, J., & Hess, J. (2022). DevOps for the Modern Enterprise. IT Revolution Press.
- [8] Kim, G., Humble, J., Debois, P., & Willis, J. (2021). The DevOps Handbook (2nd ed.). IT Revolution Press.
- [9] Banker, K., Garrett, D., Bakkum, P., & Verch, S. (2016). MongoDB in Action (2nd ed.). Manning Publications.
- [10] Brown, E. (2019). Web Development with Node and Express (2nd ed.). O'Reilly Media.
- [11] Banks, A., & Porcello, E. (2020). Learning React (2nd ed.). O'Reilly Media.
- [12] Shklar, L., & Rosen, R. (2009). Web Application Architecture: Principles, Protocols and Practices (2nd ed.). Wiley.
- [13] Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson.
- [14] Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley.
- [15] Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley.

Web Sites and Technical Documentation

- [16] MDN Web Docs — Mozilla Developer Network: <https://developer.mozilla.org>
- [17] React Documentation: <https://react.dev>
- [18] Node.js Documentation: <https://nodejs.org/docs>
- [19] Express.js Guide: <https://expressjs.com>
- [20] MongoDB Manual: <https://www.mongodb.com/docs>
- [21] Mongoose ODM: <https://mongoosejs.com>
- [22] JSON Web Token (JWT) Introduction: <https://jwt.io/introduction>
- [23] OWASP Top 10 Web Application Security Risks: <https://owasp.org/www-project-top-ten/>
- [24] Tailwind CSS Documentation: <https://tailwindcss.com>
- [25] TanStack Query (React Query): <https://tanstack.com/query>
- [26] Playwright Testing Framework: <https://playwright.dev>
- [27] Jest Testing Framework: <https://jestjs.io>
- [28] Docker Documentation: <https://docs.docker.com>
- [29] GitHub Actions Documentation: <https://docs.github.com/en/actions>
- [30] Prometheus Monitoring: <https://prometheus.io/docs>
- [31] Grafana Documentation: <https://grafana.com/docs>
- [32] Cloudinary Documentation: <https://cloudinary.com/documentation>
- [33] Terraform Documentation: <https://developer.hashicorp.com/terraform>
- [34] Helmet.js: <https://helmetjs.github.io>
- [35] Express Rate Limit: <https://express-rate-limit.mintlify.app>

Scholarly Articles and Research

- [36] Laudon, K. C., & Traver, C. G. (2021). E-commerce: Business, Technology, Society (16th ed.). Pearson.
- [37] Turban, E., Outland, J., King, D., Lee, J. K., Liang, T. P., & Turban, D. C. (2018). Electronic Commerce 2018: A Managerial and Social Networks Perspective (9th ed.). Springer.
- [38] Choudhary, V., & Tiwari, A. (2021). A study on customer satisfaction towards online jewellery shopping. International Journal of Research in Engineering, Science and Management, 4(11).
- [39] Microservices vs Monolith: A Performance Comparison. IEEE Software, 2020.
- [40] Conway, M. E. (1968). How Do Committees Invent? Datamation, 14(4), 28–31.

Arabic References

- [42] عبدالرحمن، أ. (2020). هندسة البرمجيات: مفاهيم وتطبيقات. دار النشر العلمي، الرياض.
- [43] الخطيب، م. (2021). (أسس تطوير تطبيقات الويب الحديثة. مؤسسة الكتب التقنية، عمان.
- [44] السعدي، ط. (2022). (أمن المعلومات وتقنياته في تطبيقات الويب. دار المعارف الجامعية.
- [45] مدخل تطبيقي. مطبعة الجامعة، دمشق: NoSQL الناشف، ف. (2019). (قواعد البيانات غير العلائقية